

Verification Continuum™

Verdi® SVTB Interactive Simulation Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

1. Preface	8
Audience	8
Customer Support	8
Synopsys Statement on Inclusivity and Diversity	9
2. Introduction	10
Overview	10
3. Usage	12
Before Invoking Interactive Simulation Debug Mode	12
Set Environment Variables	12
Generate the Simulator Executable	13
Import SystemVerilog Design and Testbench to Verdi	13
Configure Simulation Selection	13
Passing Tcl Commands at Runtime	14
Invoking Verdi Interactive Debug	14
Launching Simulation and Verdi at the Same Time	14
Launching Verdi After Performing Simulation	16
Handling FSDB_GATE When Invoking Verdi Interactive Mode Using Attach and Command Line Flows	17
Interactive Simulation Debug Mode	18
Debugging During Initialization of SystemVerilog Static Functions and Tasks	19
Limitations	19
Viewing Force/Release Information in Interactive Debug Mode	20
Invoking Verdi With Existing Simulation Process	24
4. Simulation Control and Local/Watch Variables Observation	26
Simulation Control Commands	26
Local Tab	27

Trace the Local Data	30
Watch Tab	33
Variable Values	35
Annotation Value	35
Force Values	37
Breakpoint Usage	41
Breakpoint Management	42
Set Breakpoint for Macro/Protect Code	47
5. Consistent Radix Setting and Value Format	49
Consistent GUI Style for Radix and Notation Settings	49
Consistent Radix and Value Format for Primitive Data Types	50
Consistent Value Format for Composite Data Types	53
Changing the Radix of Data Member	54
Support for Radix and Value Format Synchronization in Different Views	54
Change Sequence to Query Value When Cursor Time Equals Simulation Time . . .	55
Displaying Leading Zero for Signals in Interactive Mode	55
6. Browsing Classes and Dynamic Objects	56
Class Browser	56
Viewing Class Member Data in Member View	59
Navigating Contents in Class Browser	61
Show More Information in Class Browser	63
Object Browser	64
Viewing Object Member Data in Member View	65
Navigating Contents in Object Browser	66
Show Reference Information in Object Browser	68
Viewing Inherited Class Objects Hierarchically in the Local and Watch Panes . . .	69
Option Button on the Local/Watch Panes	71
Filter and Search Behavior in the Local/Watch Panes	72
7. Stack and Thread View	73
Active Mode	73
Thread Mode	74

8. Dynamic Dumping	76
HDL Example	76
Dump and Get Signals	77
Dumping Options	80
Dumping Dynamic Objects	81
Dump Batch Signals	83
9. Rebuild and Restart	84
Restart	84
Rebuild and Restart	85
Configurations of Rebuild KDB and simv	87
Configurations of Load KDB after Design Rebuild	88
Configurations of Restart Simulator after Design Rebuild	89
10. Record and Rewind the Simulation Status	90
Add Checkpoints Manually	90
Add Checkpoints Automatically	92
Add Checkpoints Automatically for Last Simulation Stops	92
Add Time Checkpoints Automatically for Last Simulation Stops	93
Rewind to Checkpoint	94
11. Save/Restore Simulation Session/State	96
Save/Restore Session	96
Saving a Session or Layout	96
Restoring the Saved Session	99
Save/Restore State	99
12. Constraint Debug	102
Local View	103
Turn On Constraint Mode	103
Create a New Constraint	104
Member View	106
Breakpoint Management	107

Contents

Simulation Control Commands	109
Step in Constraint Solver	109
Redo Randomize Call and Step in Constraint Solver	109
Distribution Re-randomize and Step in Constraint Solver	110
Solver View	111
Trace Solver Data	113
Relation View	115
Inconsistent Constraints	118
Re-randomize	119
Distribution Re-randomized Analysis	120
Supported Platforms and Simulator	123
13. Reverse Debug	124
Introduction	124
Prerequisites	125
Use Model	125
Before Invoking Verdi Interactive Simulation Debug Mode	126
Enabling Verdi Reverse Interactive Simulation Debug Mode	126
Using Reverse Simulation Control Commands	127
Going to Previous/Next Value Assignment	132
Using Undo and Redo Last Simulation Control Command	134
Going to Next or Previous Message in Interactive Console	134
Going to Next or Previous Function/Task Invocation	136
Going Back When an Object is Created	136
Active Point Behavior	137
Discarding Future Simulator Status	138
Discarding Future Automatically	139
Saving and Restoring Session	139
Specifying Compression Level of a Save Session	141
Adding Virtual Checkpoints to a Session	141
Support for Debug Points	142
Reverse Debug Commands	145
Run/Continue Reverse	146
Step Reverse	146
Next Reverse	146
Step in Thread Reverse	147

Contents

Next in Thread Reverse	147
Step in Testbench Reverse	147
Step out Reverse	147
Go to Active Point	148
Checkpoint Distribution	148
Checkpoint Memory Profiling	149
Reverse Debug Support for VHDL	150
Limitations	151
14. Tracing Testbench Driver in Temporal Flow View	152
Introduction	152
Viewing Testbench Frame Node on TFV	154
Viewing Signal Information when Tracing on Existing Path	159
Property Window for Testbench Node/Folder	163
Limitations	165
15. Appendix A: SystemVerilog Example	166
16. Appendix B: VHDL Support	168
17. Appendix C: Tcl Dumping Command	170
18. Appendix D: Known Problems and Limitations	171

1

Preface

This book is an user guide of Verdi SystemVerilog Testbench (SVTB) Interactive Simulation Debug and provides detailed description of every view and usage of Verdi SVTB Interactive Simulation Debug.

Audience

The audience for this manual includes engineers who look for the details of the view and usage available of Verdi SVTB Interactive Simulation Debug.

- *Linking Novas Files with Simulators and Enabling FSDB Dumping* - Provides information on linking Novas object files with supported simulators for FSDB dumping and the related dumping commands.
-

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:_...:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

2

Introduction

Overview

The core of Verdi SystemVerilog Testbench (SVTB) Interactive Simulation Debug is a tight integration with the simulator for communication between the Verdi platform and the simulator. Verdi SVTB Interactive Simulation Debug is a technology that allows you to setup the simulation environment and bring the Interactive Simulation Debug mode up easily to debug SVTB in the Verdi platform.

The new Interactive Simulation Debug solution in the Verdi Automated Debug Platform includes a set of capabilities that provide software interactive debugging that is especially critical for SystemVerilog testbench. At its core is a tight integration with the simulator for communication between the Verdi platform and the simulator. After setting up the simulation environment, the *Interactive Simulation Debug* mode can be easily brought up and the features used to debug SVTB in the Verdi platform.

The Interactive Simulation Debug solution includes the following advantages:

- Simulation control commands to operate simulation execution.
- Managing Breakpoints for managing the setting and usage of breakpoints.
- *Stack* and *Thread View* panes for viewing the call stack and executed threads.
- *Watch* and *Local* panes for checking signals, member data, and program variables.
- *Class Browser* and *Object Browser* panes for viewing the member data of class and dynamic objects.
- Constraint debug for analyzing the solver and distribution data.
- Rewind for going back to a specific simulation point.
- Save and Restore session for the convenience of continuing the debug job.

The remaining portion of this document provides further details on how to use these enhanced functions.

Note:

The new features and information described in this document are available with VCS 2013.06 or later versions.

- For IUS simulator, FSDB dumping might be incorrect in IUS10.2 and IUS11.1 versions. It is recommended to use other IUS simulator versions such as IUS12.1 for replay simulation FSDB dumping. Alternatively, the `FSDB_IUS_NO_OPTIM` environment variable is provided for IUS10.2 and IUS11.1 versions to avoid this issue:

```
setenv FSDB_IUS_NO_OPTIM 1
```

- For the protected code, replay simulation analysis does not support the target scope and its sub-scope that includes the protected code.

3

Usage

To use the interactive features, *Interactive Simulation Debug* mode needs to be enabled first. The following sections describe how to invoke *Interactive Simulation Debug* mode.

- [Before Invoking Interactive Simulation Debug Mode](#)
- [Invoking Verdi Interactive Debug](#)
- [Interactive Simulation Debug Mode](#)
- [Debugging During Initialization of SystemVerilog Static Functions and Tasks](#)
- [Viewing Force/Release Information in Interactive Debug Mode](#)
- [Invoking Verdi With Existing Simulation Process](#)

Before Invoking Interactive Simulation Debug Mode

Before using the *Interactive Simulation Debug* features, the following criteria must be met:

- Design should include a SystemVerilog testbench. The design under test consists of Verilog, SystemVerilog, or VHDL. Refer to the *VHDL Support* appendix for details about the difference between VHDL and Verilog support.
- You must have a supported VCS simulator as listed in the Supported Platforms section.
- Your Interactive Simulation Debug session focuses on the SVTB portion.

After the above criteria are met, you must generate the simulation executable. Then, set a couple of environment variables before invoking the Verdi platform and configuring the *Simulation* page of the *Preferences* form. Finally, you are ready to start running Interactive Debug mode. You can use the SystemVerilog example appended in *Appendix A* to run through the features described in this document.

Set Environment Variables

The following environment variables must be set before compilation, elaboration, and invoking the Verdi platform to enable Interactive Simulation Debug mode.

```
setenv VCS_HOME <VCS_INST_DIR>
setenv VERDI_HOME <VERDI_INST_DIR>
setenv LD_LIBRARY_PATH $VERDI_HOME/share/PLI/VCS/
<PLATFORM>:$LD_LIBRARY_PATH
```

If the version of VCS you used is prior J-14.12 VCS and your design includes FSDB dumping commands, you can also set these variables to help facilitate the script:

```
# Set environment variables for FSDB dumping tasks in design
setenv tab $VERDI_HOME/share/PLI/VCS/LINUXAMD64/novas.tab
setenv pli $VERDI_HOME/share/PLI/VCS/LINUXAMD64/pli.a
```

Generate the Simulator Executable

You must generate the simv simulator executable with the command line debugging option `-debug_access+all`. For example:

```
% vcs -sverilog example.sv -debug_access+all <compilation options>
```

Import SystemVerilog Design and Testbench to Verdi

Import your SystemVerilog design and testbench into the Verdi platform on the Verdi command line. For example:

```
% verdi -sv example.sv &
```

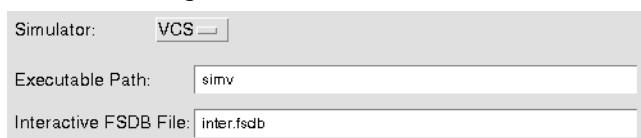
Refer to the *Import SystemVerilog Files* section of the *Verdi and Siloti Command Reference manual* (`$VERDI_HOME/doc/reference.pdf`) and the *Compile and Import SystemVerilog Code* application note for details about importing SystemVerilog code.

Configure Simulation Selection

Invoke the **Tools -> Preferences** command in the *nTrace* main window and select the **Simulation** page on the *Preferences* form.

1. Select VCS as the Simulator.
2. Type the full path of the simulator executable in the **Executable Path** text field.
3. Specify the interactive FSDB file name. The default value is `inter.fsdb`.

Figure 1 Configure Simulation Selections



Simulator:	VCS
Executable Path:	simv
Interactive FSDB File:	inter.fsdb

Note:

Alternatively, specify the `-simType`, `-simBin`, and `-simOpt` options on the Verdi command line to specify the simulation options. Refer to the Utilities: verdi section of the *Verdi and Siloti Command Reference manual* (`$VERDI_HOME/doc/reference.pdf`) for details.

Passing Tcl Commands at Runtime

You can pass Tcl commands from the simv command line using the `-verdi_opts "-cmd"` option at runtime. Following is the syntax:

```
% simv -verdi -verdi_opts "-cmd '<tcl_command>'"
```

The `-cmd` option allows you to pass Tcl commands from the simv command line at runtime. The Tcl commands can be UCLI or Verdi commands.

Example:

```
% simv -verdi -verdi_opts "-cmd 'puts {Hello world}'"
```

Invoking Verdi Interactive Debug

The following sections describe different ways to invoke Verdi Interactive Debug:

- [Launching Simulation and Verdi at the Same Time](#)
- [Launching Verdi After Performing Simulation](#)
- [Handling FSDB_GATE When Invoking Verdi Interactive Mode Using Attach and Command Line Flows](#)

Launching Simulation and Verdi at the Same Time

You can use either of the following ways to launch the simulation and Verdi at the same time:

- [Using simv Command Line](#)
- [Using Verdi Command Line](#)
- [Using Verdi GUI](#)

Using simv Command Line

In Unified Front End flow, use the `-verdi` runtime option to run the simulation and enable Verdi interactive mode. Following is the use model:

```
% vcs -debug_access+all -kdb [compile options]
% simv -verdi [runtime options] [-verdi_opts <verdi_options>]
```

Using Verdi Command Line

In the Unified Front End flow, you can use the `-i` runtime option to run the simulation and enable Verdi interactive mode while invoking Verdi. Following is the use model:

```
% vcs -debug_access+all -kdb [compile options]
% verdi -i [verdi options] -simDelim "[vcs runtime options]"
```

Example:

```
% vcs -full64 -sverilog -debug_access+all+reverse -kdb=common_elab
  -ntb_opts uvm -timescale=1ns/1ns
% verdi -dbdir ./test_simv.daidir -i +UVM_VERDI_TRACE=HIER -simDelim "
  +UVM_TESTNAME=test_2m_4s +UVM_TR_RECORD"
```

Using Verdi GUI

You must use the `-debug_access+all` and `-kdb` options at compile time.

Perform the following steps:

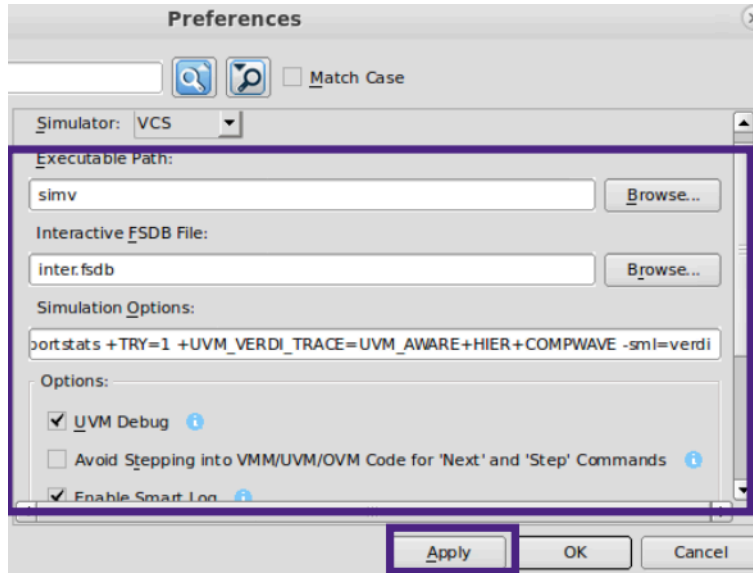
1. Invoke Verdi with `dbdir` as follows:

```
% verdi -dbdir simv.daidir
```

2. In the Verdi GUI, click **Tools -> Preferences**.

- In the *Simulation* page of the *Preferences* dialog box, specify simv path and simulation options as shown in the following figure:

Figure 2 Preferences Dialog



3. Click **Apply**.
4. Click the Run button  in the toolbar to invoke the Verdi simulator.

Launching Verdi After Performing Simulation

If your simulation is executing, you can enable Verdi Interactive debug by attaching Verdi to the running simulation using the command line or Verdi GUI.

The following are the three ways to attach Verdi to a running simulation:

- [Attaching Verdi to a Running Simulation](#)
- [Invoking Verdi Using UCLI Command](#)
- [Invoking Verdi Using Dbdir](#)

Note:

To enable these flows, you must specify the `-debug_access+all -kdb` options at compile time and `-ucli -ucli2Proc` options at runtime.

Attaching Verdi to a Running Simulation

You must use the `-attachSim <simv_pid>` runtime option to attach Verdi to a simulation with the Process ID (PID). There is no need to specify the `-dbdir` option to import the design.

Example: `% verdi -attachSim 16733`

You cannot use this option with the following usages while invoking Verdi interactive mode:

- `simv -verdi`
- `verdi -i`
- `start_verdi` in ucli2Proc mode

Invoking Verdi Using UCLI Command

You must use the `-start_verdi` UCLI command as follows to launch Verdi in UCLI mode:

```
ucli% start_verdi
```

Invoking Verdi Using Dbdir

You can invoke Verdi with `dbdir` and then attach Verdi to a selected simulation using Verdi GUI command as follows:

1. In Verdi GUI, select the **Simulation -> Attach to Simulation...** command to open the **Attach to Simulation** dialog box.
2. In the **Attach to Simulation** dialog box, select a simulation process and click **Attach**.

Handling FSDB_GATE When Invoking Verdi Interactive Mode Using Attach and Command Line Flows

Attach Flow

The Attach flow attaches Verdi to the simulation process. In this flow, the simulation provides the `FSDB_GATE` setting. By default, this setting is enabled when the dump signal is `simv`. You can use the `setenv FSDB_GATE 0` command to disable the default `FSDB_GATE` setting.

Verdi also provides the **Enable FSDB Gate** preference option (**Preferences -> Simulation**) to disable the `FSDB_GATE` setting; you must clear the checkbox of this option to disable the setting. However, a simulation restart is required to use the preference setting to disable `FSDB_GATE`.

Command Line Flow

This flow invokes Verdi interactive mode while executing `simv` in the command line.

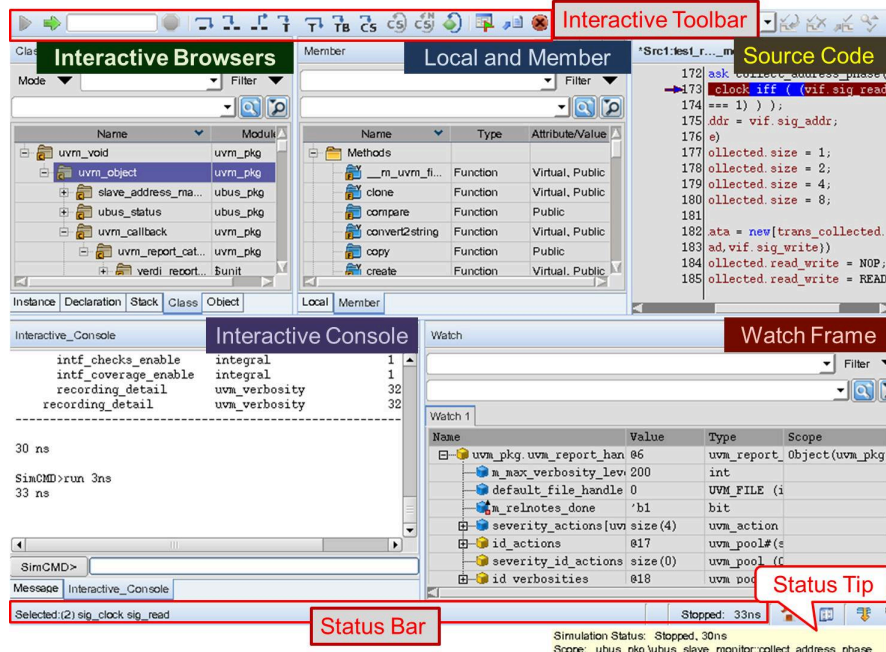
In this flow, the `FSDB_GATE` setting keeps the original behavior. That is, `FSDB_GATE` is auto-disabled when invoking Verdi interactive mode.

You can use the `setenv INT_FSDB_GATE 1` command to enable the `FSDB_GATE` setting in the interactive mode in the command line flow. Verdi also provides the **Enable FSDB Gate** preference option (**Preferences -> Simulation**) to enable the `FSDB_GATE` setting; you must select the checkbox of this option to enable the setting. However, a simulation restart is required to use the preference setting to enable `FSDB_GATE`.

Interactive Simulation Debug Mode

After loading your SystemVerilog design and testbench, select the **Tools -> Run Simulation** command to start *Interactive Simulation Debug mode*.

Figure 3 Interactive Simulation Debug Mode in Verdi Platform



The following panes can be viewed in the Interactive Simulation Debug Mode:

- The Interactive Toolbar places the simulation command buttons and the Interactive Browser provides the testbench architecture view.
- The Local tab automatically shows the local information in the current simulation scope and the Member tab shows the member data of the object.
- The Interactive Console pane provides the interface to input the UCLI commands and shows the results of simulator execution. You can change the text font of the

Interactive Console pane in the **Source Code -> Color/Font** page of the *Preferences* form (invoked with the Tools ' Preferences command).

- The variables, signals, or objects can be added to a Watch tab of the *Watch* pane. This pane contains one Watch tab by default and the maximum number of Watch tabs that can be added using the **Debug -> Add New Watch** command is five.
- The simulator status is shown in the Status bar. You can place the cursor over the **Stopped** field of the Status bar and hold for a couple of seconds to display a tip window.

Debugging During Initialization of SystemVerilog Static Functions and Tasks

You can enable debugging when initialization begins for static SystemVerilog tasks and functions in module definitions by using the `-ucli=init` runtime option as follows:

```
% simv -verdi -ucli=init
```

This debugging capability enables you to set breakpoints during initialization.

During initialization (init mode), the `-ucli=init` runtime option does not support the features mentioned in the [Limitations](#) section.

When you click the **run** button in the init mode, Verdi executes `run0` first and then the `run` command. The `run0` execution runs through and exits the initialization mode. Therefore, the FSDB file is created when the initialization mode finishes at time 0.

Limitations

The following are the limitations of the `-ucli=init` runtime option:

- FSDB commands
- The `change` and `value` commands
- Rewind/Reverse Debug
- Save/Restore simulator states
- Dumping mechanism. For example,
 - If you input `run 100` and the init mode is exited, then FSDB file is created at time 50 if there are dumping tasks in the design at that time, and the FSDB is empty during time 0~50. In addition, the FSDB filename is determined by the environment variable `FILE_NAME`, which is set during entering Verdi interactive mode.

Chapter 3: Usage

Viewing Force/Release Information in Interactive Debug Mode

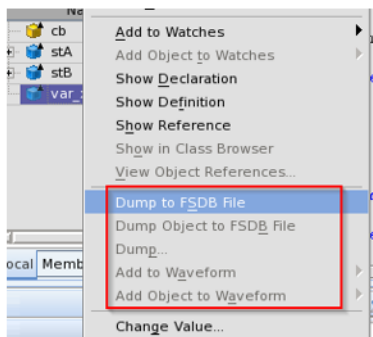
- If you input `run 100` and the init mode is exited, and there are no dumping tasks during this period, then the FSDB file is created at time 100 using the `fsdbDumpfiles` command from Verdi. The `FILE_NAME` and `fsdbDumpfiles` command use the same filename in the above scenarios. Hence, the difference is only when the time FSDB file is created.

The user interface related to the above restricted features is disabled in Verdi interactive mode as shown in the following figures. The FSDB based features (for example, `dump` command, `fsdbDumpvars`, and so on) are also disabled. These features are enabled only after exiting the init mode.

Figure 4 Reverse Debug Commands



Figure 5 Dump Commands and Add to Waveform Commands



Viewing Force/Release Information in Interactive Debug Mode

Verdi allows you to view all active forces at the current simulation time in interactive debug mode using the VCS `-force_list` option at compile time. For more information on the `-force_list` option, see *VCS User Guide*.

Use model

Compile the design as follows:

```
% vcs -debug_access+all <design file> -sverilog -kdb -force_list
```

Run the simulation as follows:

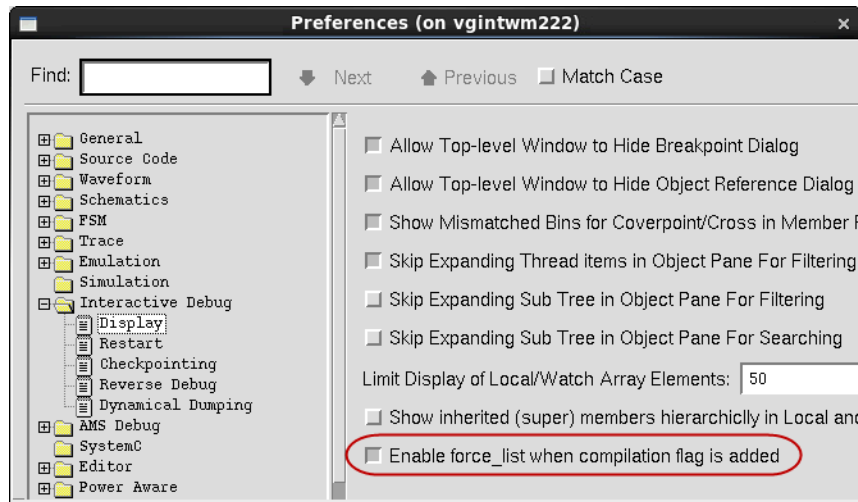
```
% simv -verdi +fsdb+force -force_list=uniqueID
```

or,

```
% setenv FSDB_FORCE 1  
% simv -verdi -force_list=uniqueID
```

Verdi automatically enables the force list feature at runtime when the *-force_list* option is used at compile time. You can use the **Enable force_list when compilation flag is added** preference option in the *Preferences* form (**Interactive Debug** folder -> **Display** page) to change this default setting.

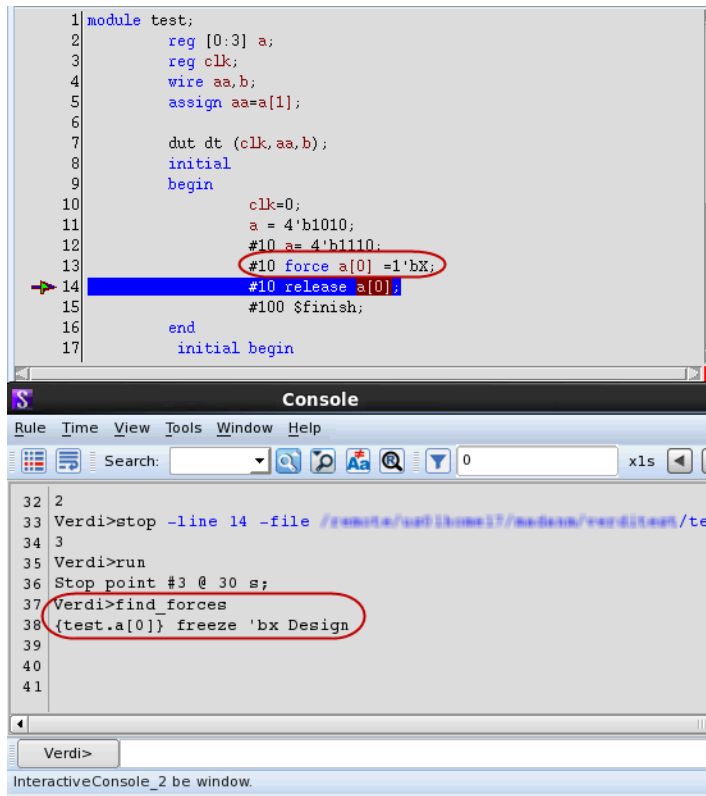
Figure 6 Preference Setting for *force_list*



You can use the `find_forces` UCLI command or the **Find Forces** dialog box to view the current active forces in the design or scope.

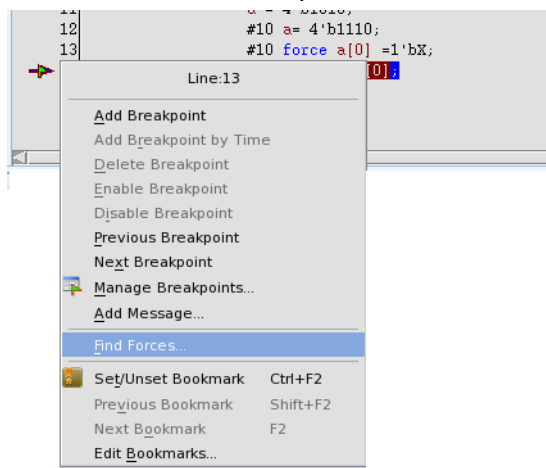
The following figure shows the active forces displayed using the `find_forces` UCLI command.

Figure 7 *find_forces in the Console Window*



You can also use the **Find Forces** dialog box to view the current active forces in the design or scope. To view the **Find Forces** dialog box, right click a line number in the *Source code view* and select the **Find Forces** option as shown in the following figure:

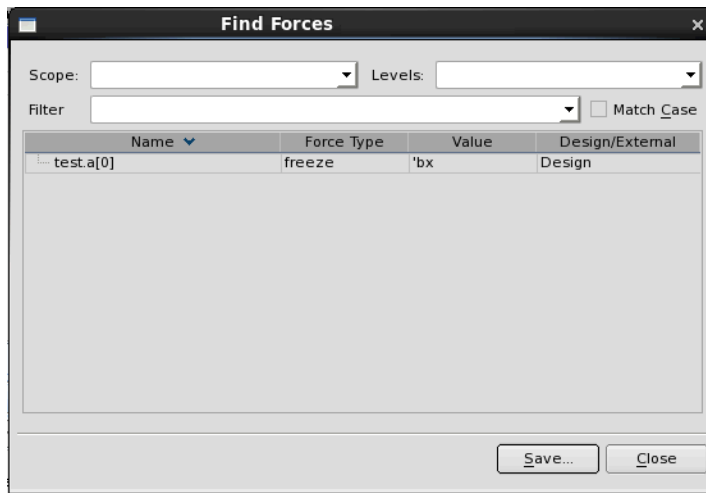
Figure 8 *Find Forces Option*



The **Find Forces** right-click option is enabled only when the `-force_list` option is used at compile time.

The **Find Forces** dialog box lists all the relevant forces which are still being forced in current time tick as shown in the following figure. By default, Verdi lists all forces.

Figure 9 Find Forces Dialog Box

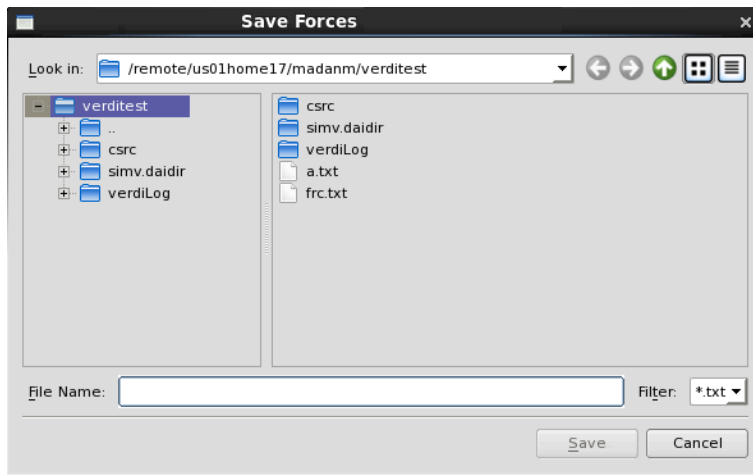


Where,

- **Scope** - Allows you to specify the full scope path name to find force. Verdi lists activate forces present only in this scope. If the scope value is empty, Verdi lists activate forces on all the scopes. Default behavior is to list all forces.
- **Levels** - Allows you to specify the level of hierarchy to do force search. Default value is 0 (search all sub-scopes). Value 1 searches only the signals at the given scope.

- **Filter** - Allows you to filter the active forces. If you select the **Match Case** check box, Verdi filters exact matches and saves the setting to the .rc file.
- **Save** - Allows you to specify a file in the **Save Forces** dialog box to report the result as shown in the following figure:

Figure 10 Save Forces Dialog Box



When simulation is running, the **Scope** and **Levels** text fields of the **Save Forces** dialog are disabled. When simulation stops, Verdi updates the view automatically.

Limitations

Following are the limitations of this feature:

- Hyper linking for design forces in the *Console* window is not supported.
- Double-clicking the design forces in the **Find Forces** dialog box do not jump to the *Source code view*.

Invoking Verdi With Existing Simulation Process

You can use the `-attachSim <sim_pid>` option to invoke Verdi with an attached simulation mode. This feature provides a simplified way to attach Verdi to the simulation without the need to specify the `-dbdir` option in the same machine. The `dbdir` is automatically obtained by Verdi connecting to the running VCS simulation using the process ID, `pid`.

Following is the syntax:

```
% verdi -attachSim < sim_pid >
```

`<sim_pid>`: The simulation process ID which you want to attach.

This option cannot be applied with the following usages while invoking Verdi interactive mode:

- `simv -verdi`
- `verdi -i`
- In `ucli2Proc` mode, `start_verdi`

Key Points to Note

- Use the `-debug_access+all` VCS elaboration option to enable the VCS debug features.
- The simulation must start with the `-ucli -ucli2Proc` VCS runtime option.
- The Restart and Restore session commands are not supported in Verdi after the simulation is attached.
- The design used in the simulation is used for invoking Verdi, and not to specify another design for the Verdi command.
- Using Ctrl+Z to suspend a `simv` process and then trying to attach Verdi to the simulation is not supported.
- The `-attachSim` option supports only the following additional options in the Verdi command line:

`-Apex, -Base, -Elite, -SX, -Ultra, -apex, -base, -lca, -rcFile, and -play`

4

Simulation Control and Local/Watch Variables Observation

The simulation control commands are provided to control the simulation progress and the **Local** and **Watch** tabs are used to observe the program execution and to trace data changes. You can also use the Manage Breakpoints to control the simulation stop locations with more variability and flexibility.

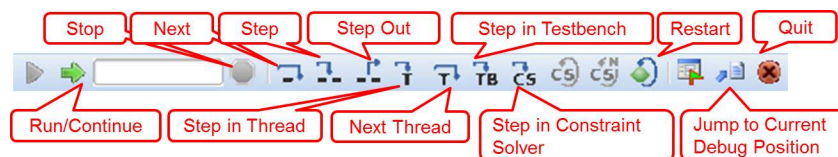
This chapter consists of the following sections:

- [Simulation Control Commands](#)
- [Local Tab](#)
- [Watch Tab](#)
- [Variable Values](#)
- [Breakpoint Usage](#)

Simulation Control Commands

The frequently used simulation control commands are listed in the Interactive toolbar as shown in the following figure to direct the simulator how to execute the program. Other commands, for example, the **Kill Simulator Process** command, are listed in the Simulation menu.

Figure 11 Simulation Control Commands in Toolbar



You can advance the simulation to the statement where a function is called using the **Step** command (by clicking the Step button in the toolbar or selecting the **Simulation -> Step** command). The **Local** and **Watch** tabs are refreshed accordingly.

To avoid confusion, simulation commands are allowed only when the simulator is not busy. Therefore, if the simulator is running, only the **Simulation -> Stop** and **Simulation -> Kill Simulator Process** commands are enabled. All other simulator related commands are disabled.

When the simulation is finished, the Verdi platform does not exit Interactive Simulation Debug mode and the data is kept in each frame until the **Simulation -> Quit or Tools -> Stop Simulation** commands are used to exit the Interactive Simulation Debug mode.

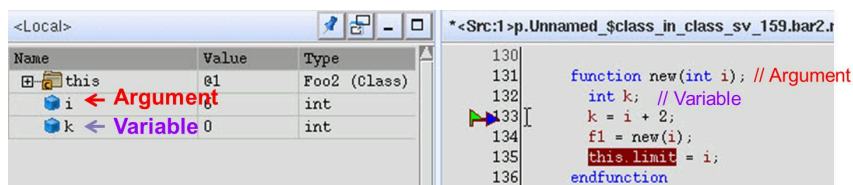
Refer to the Simulation Commands section of the *Verdi and Siloti Command Reference manual* ([\\$VERDI_HOME/doc/reference.pdf](#)) for details about the simulation commands and the Constraint Debug section for details about the constraints commands.

Local Tab

When the **Next** or **Step** simulation control commands are used and the scope of the current simulation belongs to a testbench scope or defined class object, all variables of the current simulation scope are shown automatically in the **Local** tab. Variables in the **Local** tab include the following categories:

- **this**: Represents the current class object. It is displayed only when the simulation stops inside a class object scope. The object ID and all member data are listed under the "this" node.
- **Local variables**: All local variables in the current scope are displayed, including function arguments, interface class objects, and constraints.
- **Arguments**: All arguments of the method are displayed if the simulation stops inside a function or task.

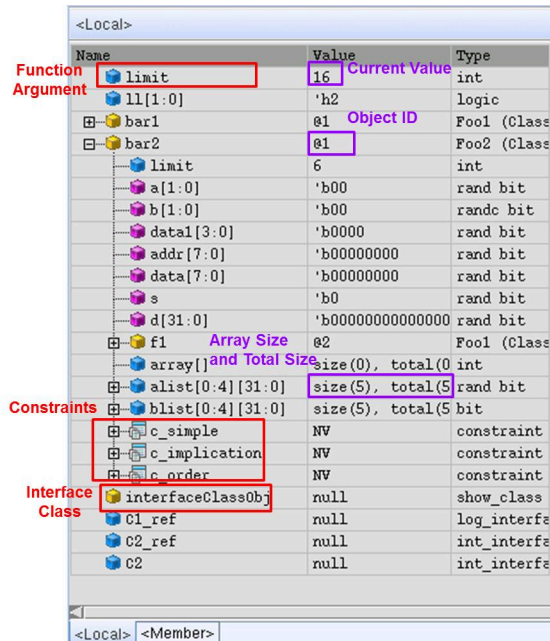
Figure 12 Local Tab



- **Signals**: If the simulation stops in an HDL scope and there is a class object in this scope, all signals of this scope are displayed, including class object.
- **Global Variables**: If the global variables are defined in the program, then they are always shown on the **Local** tab.
- **Package**: The imported packages for the current scope are displayed. You can expand a package node to see the global variables under this package.

The following figure shows the data information with a tree structure in the **Local** tab. Each tree node is a variable and you can expand it if the variable is an object, array, or constraint.

Figure 13 Local Tab Enhancements



The following table lists what value fields are shown in the **Local** tab for different corresponding nodes:

Corresponding Node	Value Field Shows	Example
Alive class object	Object ID	@1
Array	Child node number and leaf node number	size(4), total(128)
Constraint	No value displayed	NV
Other	Current value	16

Figure 14 Multi-Dimensional Array

Name	Value
s5	x
i_3Darray2[1:0][3:0][31:0]	size(2), total(256)
i_3Darray2[0][3:0][31:0]	size(4), total(128)
i_3Darray2[0][0][31:0]	size(32), total(32)
i_3Darray2[0][1][31:0]	size(32), total(32)
i_3Darray2[0][2][31:0]	size(32), total(32)
i_3Darray2[0][3][31:0]	size(32), total(32)
i_3Darray2[1][3:0][31:0]	size(4), total(128)

You can use the **Limit Display of Local/Watch Array Elements** option in the **Interactive Debug -> Display** page of the *Preferences* form (invoked with the **Tools -> Preferences** command) to change the display maximum number of array elements. After the number is specified and applied, all the expanded arrays in the **Local** and **Watch** tab are updated.

The constraint block status mode for constraint debugging can be turned on or off. Right-click a constraint block and choose the **Enable Constraint** command from the menu to enable the constraint mode, or choose the *Disable Constraint* command from the menu to disable the constraint mode. You can also use the **Enable Constraint/Disable Constraint** commands to turn on or off the random variable. The randomization of the variable is disabled when it is turned off. Refer to the [Constraint Debug](#) chapter for details about constraints debugging.

Figure 15 Turned-Off Random Mode Variable

b2[7:0]	'b0000011	rand bit
b3[31:0]	'b0000000000	rand bit
b4[69:0]	'b0000000000	rand bit

The following table and figure shows the icon attributes in the **Local** and **Watch** tabs. Icon attribute  represents the item that is derived from a base item. For example, an item with the icon represents a function that is derived from a base function. Refer to the following table and figure for more attributes and usages on the icons.

Icon Attributes:

Attribute Figure	Attribute
	The item is derived from base class
	The item is protected

	The item is private
	The item is static
	The item is virtual
	The item has been overridden

Figure 16 Icon Attributes in Local and Watch Tabs

	Default	From base	protected	private	static	virtual	overridden
Function							
Task							
Class Object							
Variable							
Rand variable							
Constraint Block							

Trace the Local Data

You can click one of column header from the

Name	Value	Type
------	-------	------

 columns to sort the data according to the related field in the **Local** tab. You can also use the following enhanced commands in the right-click menu to trace the selected variable in the **Local** Tab:

- **Show Navigation Text Field:** Filters or searches for the specified string for the current tree in the **Local** tab.



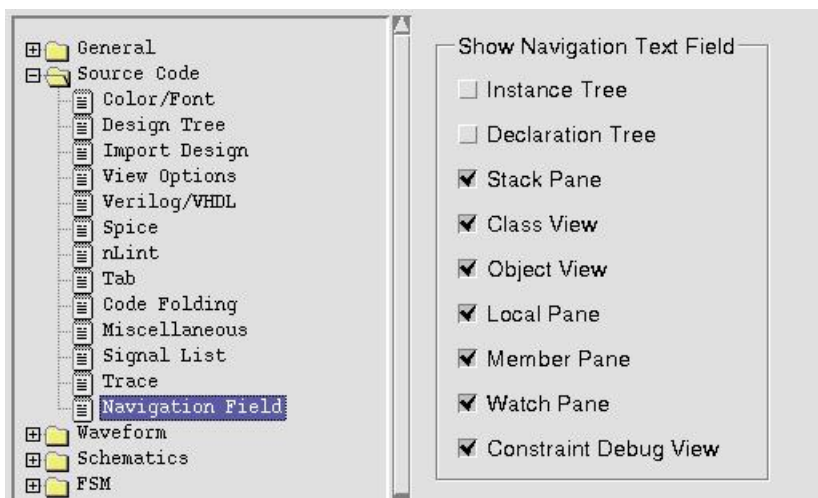
Input a filter string in the field and press the Enter key to filter the local data in the tree. If no matching object name with the specified string exists, the tree node is filtered out. You can select the filtering types in the **Filter** menu to filter the related items in the **Local** tab.

To search the tree, input a string to the field and then click the  icon to search backward or click the  icon to search forward. If a matched item belongs to a folded node, the node is expanded automatically to locate the item.

The *Navigation Text* field is shown by default. You can hide the fields using the following methods:

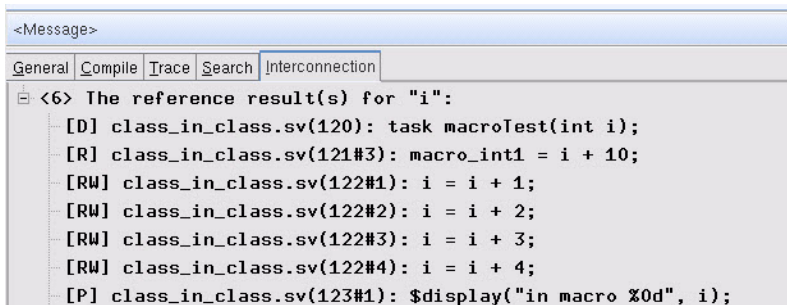
- Cancel the **Show Navigation Text Field** command in the right-click menu.
- Configure the setting in the *Preferences* form:
 - Use the **Tools -> Preferences** command to invoke the *Preferences* form.
 - Select the **Source Code -> Navigation Field** page.
 - Select the frames for which you want to show the *Navigation Text* field.

Figure 17 Show Navigation Text Field Options in Preferences Form



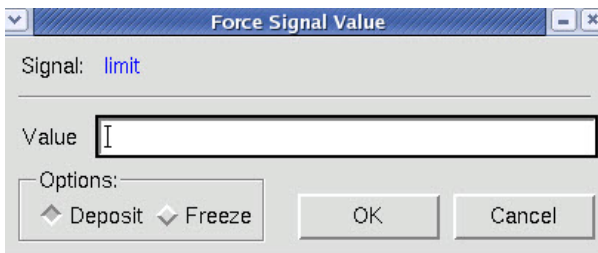
- **Show Declaration:** Shows the declaration of the selected node in the *Source Code* pane. Double-clicking the node shows the result in the *Source Code* pane.
- **Show Definition:** Shows the definition of the selected node in the *Source Code* pane.
- **Show Reference:** Lists all references of the selected node in the **Interconnection** tab of the *Message* pane. The following figure shows the result of all the references of the selected variable.

Figure 18 Show Reference in the Message Pane



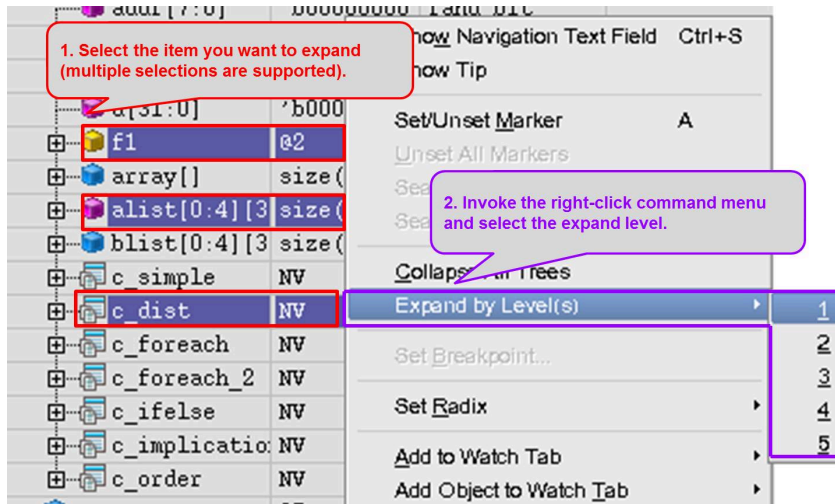
- **Add Object to Watch Tab:** Adds the selected object to a **Watch** tab for observing the value even when the simulation exits the object's living scope. This command works only when a tree node of a class object is selected. Refer to the [Watch Tab](#) section for details about the **Watch** tab features.
- **Show in Class Browser:** Finds the selected object in the **Class** tab for viewing the content of the selected class. This command works only when a tree node of a class object is selected. The class is highlighted in the **Class** tab. Refer to the Class Browser section for details about [Class Browser](#) features.
- **Change Value:** Changes the value of this variable in the *Change Value* form. This command only works when a primitive variable that is not a class, structure, construct, or array is selected. The changed value is overwritten by a subsequent driver when the **Deposit** option is enabled and is frozen when the **Freeze** option is enabled.

Figure 19 Force Signal Value Form



- **Expand Tree:** Expands all selected tree nodes by one level. If the selected nodes have already been expanded, there is no affect. This command can be applied to multi-selected items.

Figure 20 Expands All Selected Tree Nodes by One Level



As you switch to the **Local** tab, the **Stack** tab is displayed automatically.

Refer to the *Local Tab* section of the *Verdi and Siloti Command Reference manual* ([\\$VERDI_HOME/doc/reference.pdf](#)) for details about the supported data type and other commands provided in the **Local** tab.

Watch Tab

The **Watch** tab is used to observe the current values of variables in the current simulation scope. Both the values and current scopes are updated as simulation time changes. The supported data types of variables are objects, member data of objects, local variables, and signals that belong to a testbench scope. You can add them by dragging and dropping variables from the *Source Code* pane. You can see the added item with the following scopes in the **Watch** tab:

- **Local Scope:** The value of the added variable is the queried value in the current simulation scope.

Figure 21 Local Scope

Name	Value	Type	Scope
\$root.stream...dle[integer]	size(2)	string	Local
uvm_pkg.uvm_revision	"UVM-1.1d"	string	Local
uvm_pkg.uvm_resources	@1	uvm...ass)	Local
m_has_wildcard_names	'b0	bit	
rp	@1	uvm...ass)	

For example, in the figure below, the current simulation scope is "Foo1::print", drag and drop the "limit" variable to the **Watch** tab, then the **Watch** tab queries "limit" in the "Foo1::print" scope. Use the simulation control commands to keep the simulation inside

the "Foo2::print" scope. The **Watch** tab queries the value of the "limit" variable in the "Foo2::print" scope instead of the value in the "Foo1::print" scope.

Figure 22 Add "limit" Variable into Watch Tab

```

23  class Foo1;
73      task print(int limit);
74          $display("Limit %0d",limit);
75      endtask
27
78  class Foo2;
121      task print(int limit);
122          for (int i=0;i<limit;i++)
123              $display("%m: array[%0d]=%0d : %0d",i,array[i],limit);
124          f1.print(limit);
125      endtask
24

```

The following methods are provided to add variables with local mode to the *Watch* pane:

Drag a variable from the *Source Code* pane.

Drag a variable from the **Local** tab.

Use the **Add to Watches** command of the right-click menu in the *Source Code* pane.

Use the **Add to Watch Tab** command of the right-click menu in the **Local** tab.

Note:

The value of an HDL signal is queried in the instance where the added signal belongs.

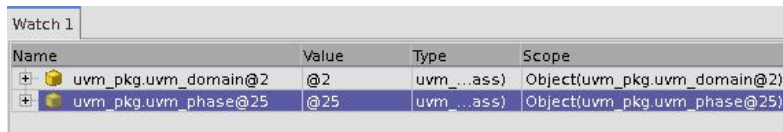
- **Static Scope:** The HDL signals belong to static scope. You can add HDL signals into the **Watch** tab anytime during the simulation.

Figure 23 Static Scope

Watch 1			
Name	Value	Type	Scope
ubus_req_master_0	'h0	Wire(Port In)	Static(ubus_tb_top.dut)
ubus_gnt_master_0	'h0	reg	Static(ubus_tb_top.dut)

- **Object Scope:** The **Watch** tab shows all members of the added object with a tree structure. Each tree node is a variable or signal. As shown in the following figure, the Value field shows the object ID (for example, @25) if the corresponding node is a class object. The root node scope is displayed in the **Scope** field to show where the object is defined.

Figure 24 Add an Object in the Watch Tab



Name	Value	Type	Scope
uvm_pkg.uvm_domain@2	@2	uvm_...ass)	Object(uvm_pkg.uvm_domain@2)
uvm_pkg.uvm_phase@25	@25	uvm_...ass)	Object(uvm_pkg.uvm_phase@25)

The following methods are provided to add an object ID to the *Watch* frame:

- Drag a class object ID from the *Class Browser* or *Object Browser* panes.
- Drag a class member from the **Member** tab.
- Select a variable of an object ID and use the **Add Object to Watches** command of the right-click menu in the *Source Code* pane.
- Select a tree node that has the object ID or is a class member in the **Local** tab, then use the **Add Object to Watch Tab** command of the right-click menu.
- Use the **Add to Watch Tab** command of the right-click menu in the *Class*, *Object*, or *Member* tabs.
- **Reference:** A reference can be added to the *Watch* tab by selecting the **Show Reference to Watch** button in the *Reference* form. Refer to the *Show Reference Information in Object Browser* section for more details.

Note:

The function, constraint, task, or class cannot be added to a *Watch* tab individually.

If the added variable no longer exists in the current simulation scope, the fields in the **Watch** tab are grayed-out.

The tracing commands introduced in the **Local** tab section are also provided in the **Watch** tab, refer to the *Local Tab* section and the *Watch Tab* sections of the *Verdi and Siloti Command Reference manual* ([\\$VERDI_HOME/doc/reference.pdf](#)) for more details.

Variable Values

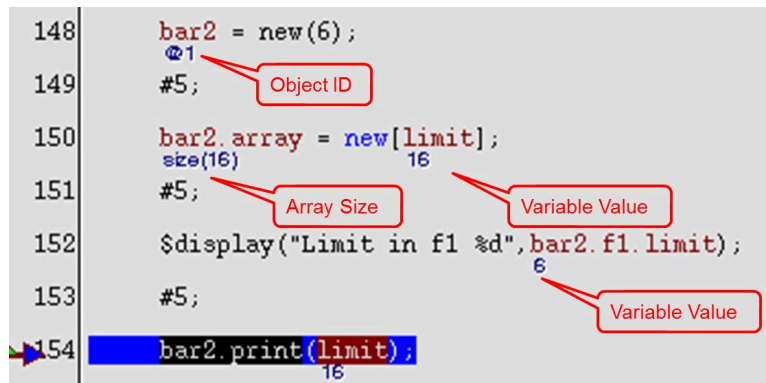
Annotation Value

Values can be annotated to the variables on the *Source Code* pane using the **Source -> Active Annotation** toggle command (the hotkey is 'x'). The **Source -> Active Annotation** outside Testbench Scope toggle command is also enabled to annotate the value of HDL signals by default.

- If variables belong to a testbench scope, the values of the variables are obtained from the simulator. If the variables are executed and match one of the following conditions:
 - belong to the active frame.
 - are the member variables of the current object.

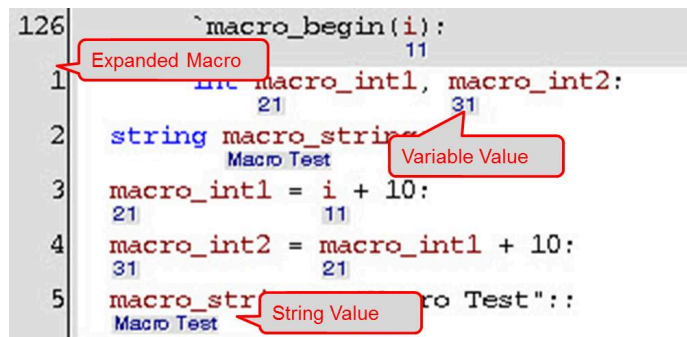
The values of the variables are displayed on the *Source Code* pane. For the variables with array type, the size of the array is shown. The radix of the variable value can be changed using the **Signal -> Signal Value Radix** command in the right-click menu.

Figure 25 Values of Variables Displayed in Source Code Frame



When a macro or protect code is in the current active scope and is expanded, the annotation values are also displayed.

Figure 26 Annotation Values of Variables Displayed in the Source Code Pane



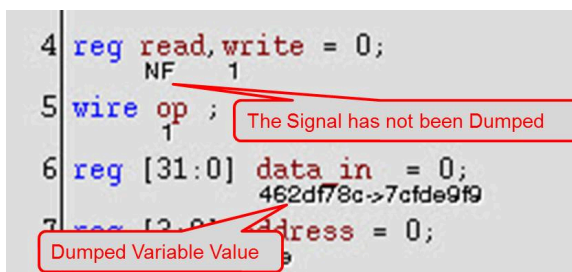
When the cursor time and simulator time are different, the active annotations that belong to the testbench scope need to be cleaned up. Move the cursor over the current simulator time and the annotation is shown again for the testbench scope variables.

- If variables belong to an HDL scope, the values of the variables are obtained from the active FSDB file first and then the simulator.

The annotation values are based on the cursor time in the active *nWave* window. The previous values can be shown by moving the cursor in the *nWave* window with an FSDB file loaded as follows:

- If the cursor time in the *nWave* window equals to the current simulation time, the values of the variables are obtained first from the FSDB file first and then the simulator.
- If the cursor time in the *nWave* window does not equal to the current simulation time, the values of the variables are obtained from the active FSDB file. In other words, the variables need to be dumped first, and then the values can be annotated to the variables on the *Source Code* pane.

Figure 27 Change Radix of Variable Value using Signal Value Radix Command



You can also set the following environment variable before invoking the Verdi platform to obtain the values only from the active FSDB:

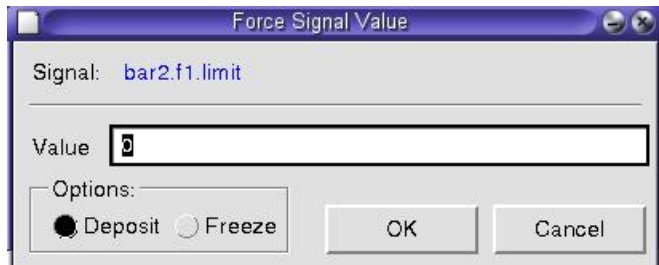
```
%> setenv NOVAS_SVTB_ANNO_MODE 0
```

Force Values

You can use the force value feature to observe the relationship between signals and see the effect for your design after the value is changed. The forced value can also be released during the interactive simulation.

For signals in the *Local*, *Watch*, and *Member* panes, use the **Change Value** command in the right-click menu. As shown in the following figure, the *Force Signal Value* form is displayed and the signal is shown in the **Signal** field. Specify the value in the **Value** field and use the **Deposit** or **Freeze** toggle options to deposit or freeze the forced value.

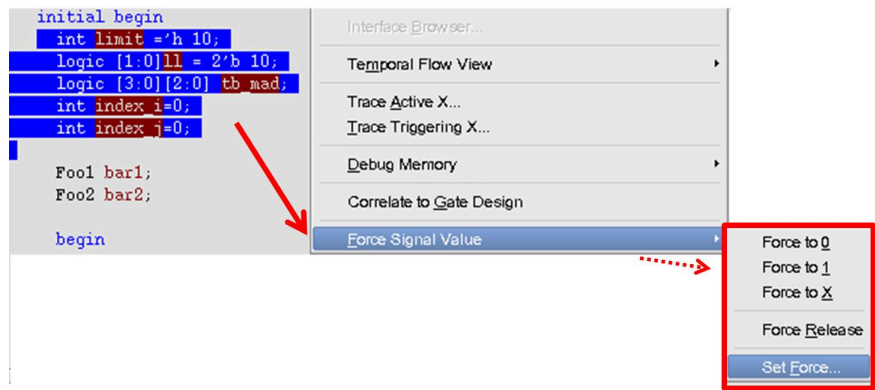
Figure 28 Change Value



You can also select the signals in the *Source Code* pane and use the **Force Signal Value** command in the right-click menu to force multiple signals values as follows.

- If the selected variables belong to initial blocks of a testbench scope, select the desired command to force the values to '0', '1' or 'X' or release the forced values (refer to the following figure). The force commands are applied to all selected signals.

Figure 29 Commands for Initial Blocks



Use the **Set Force** command to set more complex options for the forced value. As shown in the following figure, the *Force Values* form is displayed and all selected signals are shown in the **Signal** column. The setting is applied to the listed signals. The following table summarizes the fields descriptions and corresponding UCLI options.

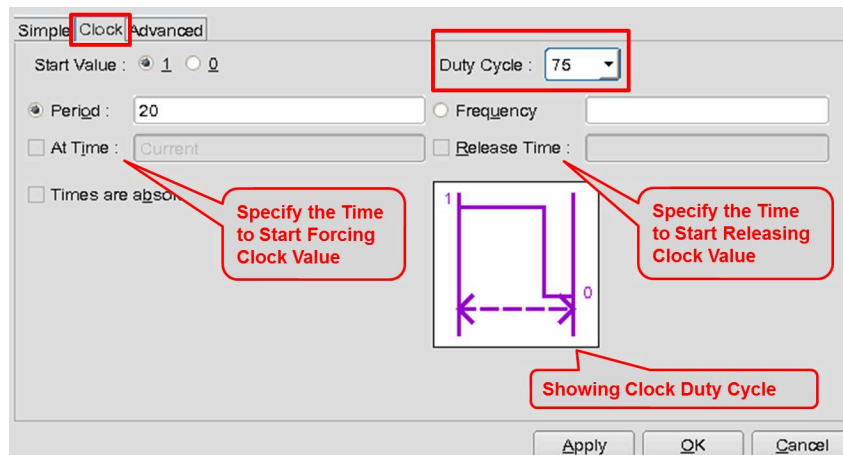
Figure 30 Force Values Form

Mapping UCLI Options in Force Values Form:

Fields	Mapping UCLI Options	Description
Value	<value>	Specify the value for the listed signal(s).
At Time	<time>[<unit>]	Specify the forcing time with time unit. "unit" can be s, ms, us, ns, ps or fs
Release Time	-cancel <time>	Release the forced value after the specified time.
Force Type: Freeze	-freeze	Freeze the value to the forced value.
Force Type: Deposit	-deposit	The value can be overwritten by a subsequent driver transaction.
Force Type: Drive	-drive	Attach a new driver to the signal (VHDL only).
Times are absolute	@	Identify an absolute time.

You can also force the oscillated value for the clock signals in the **Clock** tab. As shown in the following figure, select the Start Value with '1' or '0' and use the Period or Frequency field to specify the duration. When the **Duty Cycle** is selected, a refreshed figure shows the duty cycle waveform. If the **At Time** option is disabled, the current time is applied to start forcing clock values.

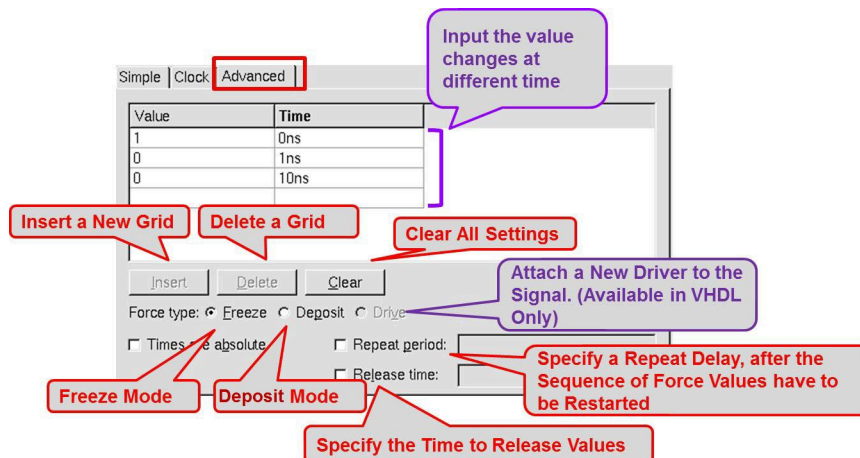
Figure 31 Force Clock Signal



In the *Advanced* tab, varying methods for forcing values are provided. You can specify the different values at different times in the **Value** and **Time** fields. Refer to the following figure for details and the following table for the corresponding UCLI options.

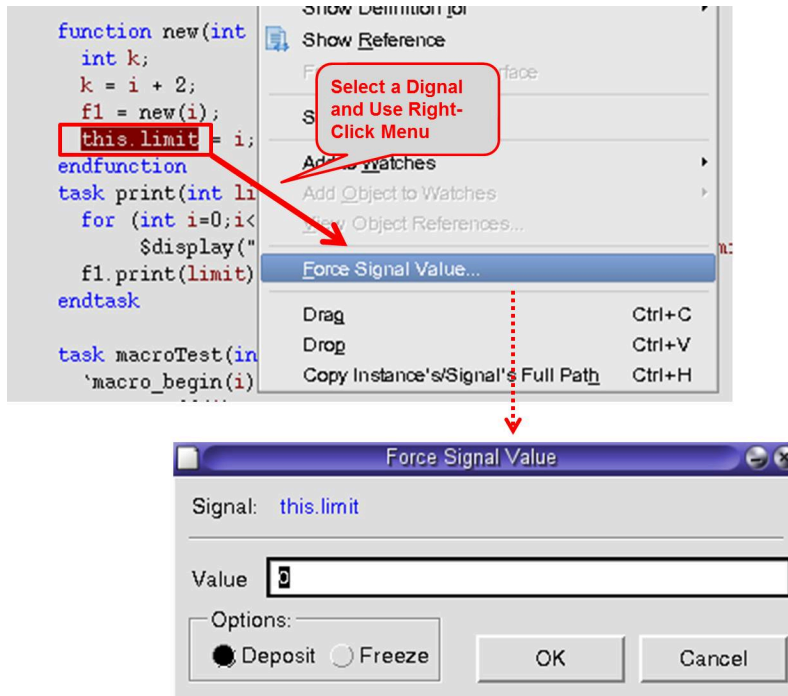
The Repeat period extra option maps to the `-repeat <interval>` UCLI option. This causes the signal(s) to repeat the waveform every relative time interval.

Figure 32 Advanced Forcing Value



- If the selected variables belong to blocks of a testbench scope, except initial blocks, the *Force Signal Value* form is displayed with the signal shown in the **Signal** field. Specify the value in the **Value** field and use the **Deposit** or **Freeze** toggle options to deposit or freeze the forced value.

Figure 33 Force Signal Value



- If the selected variables belong to an HDL scope, the usage is the same as signals belonging to the initial block of a testbench scope.

Breakpoint Usage

Breakpoints can be set by single-clicking or double-clicking the line number in the *Source Code* pane for the line-type breakpoints. Alternatively, use the **Manage Breakpoints** command to edit the breakpoints with more options in the *Breakpoints* form. The *Breakpoints* form keeps the breakpoints with enabled or disabled status, thus you can keep some breakpoints with disabled status temporarily and can enable the breakpoint later. Dynamic objects, inline macro and statement are also supported to set the breakpoints.

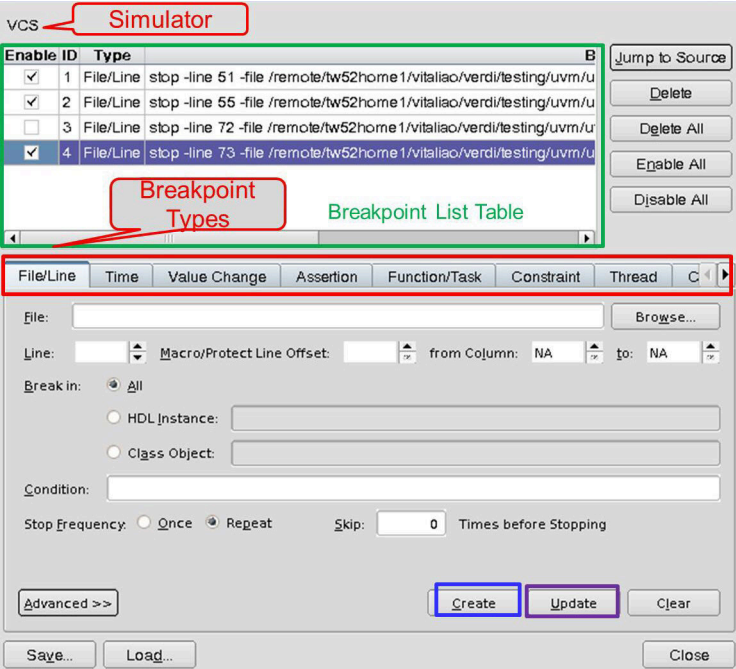
Breakpoint Management

After *Interactive Simulation Debug* mode is enabled, use the **Debug -> Manage Breakpoints** command to open the *Breakpoints* form. In [Figure 34](#), the Simulation Mode indicates the simulator type. The Breakpoint List table lists all existing breakpoints. Check or uncheck the row in the Enable column to enable or disable the breakpoint. The following icons are shown in the indicator area of the *Source Code* pane when a breakpoint is set on the line:

- : indicates the breakpoint is enabled.
- : indicates the function/task breakpoint is enabled.
- : indicates the breakpoint is disabled.

The **ID** column shows the breakpoint ID for simulator recognition and the **Type** column shows the breakpoint types. The UCLI commands for the breakpoint settings are listed in the **Breakpoint** column.

Figure 34 Break Point Form



VCS Simulator

Enable	ID	Type	Breakpoint
☒	1	File/Line	stop -line 51 -file /remote/tw52home1/vitaliao/verdi/testing/uvrn/u
☒	2	File/Line	stop -line 55 -file /remote/tw52home1/vitaliao/verdi/testing/uvrn/u
☐	3	File/Line	stop -line 72 -file /remote/tw52home1/vitaliao/verdi/testing/uvrn/u
☒	4	File/Line	stop -line 73 -file /remote/tw52home1/vitaliao/verdi/testing/uvrn/u

Breakpoint Types

File/Line Time Value Change Assertion Function/Task Constraint Thread C

File: Browse...

Line: Macro/Protect Line Offset: from Column: to:

Break in: ☒ All ☐ HDL Instance: ☐ Class Object:

Condition:

Stop Frequency: ☐ Once ☒ Repeat Skip: 0 Times before Stopping

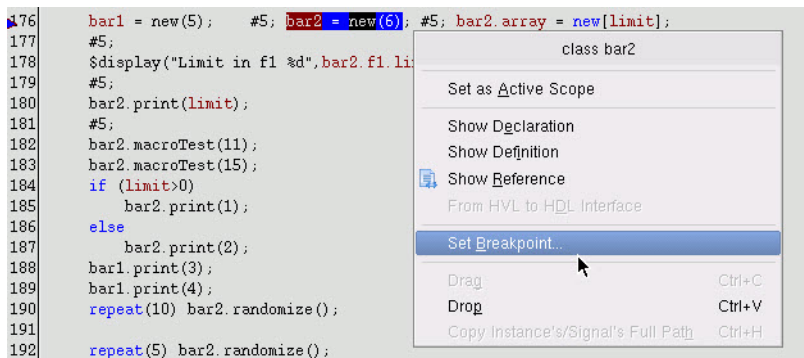
Advanced >> Create Update Clear

Save... Load... Close

The following are the types of breakpoints and corresponding tabs:

- **File/Line:** Add breakpoints at the line by specifying the line number of a file. Select the row and click the **Jump to Source** button in the *Breakpoints* form to jump the debug cursor to the line in the *Source Code* pane and highlight it. In addition, you can set the breakpoint for a specific statement in a line. Select the desired statement in the source line, as shown in the following figure, right-click and choose the **Set Breakpoint** command from the menu.

Figure 35 Setting Breakpoint for a Statement within a Line



The column numbers are automatically entered in the **from Column** and **to** fields of the *Breakpoints* form, as shown in the following figure. The  icon is shown at the line of the *Source Code* pane when the statement breakpoint is set successfully. The  icon is shown to indicate that there are breakpoints in a line.

Figure 36 Column Numbers of Breakpoints Form



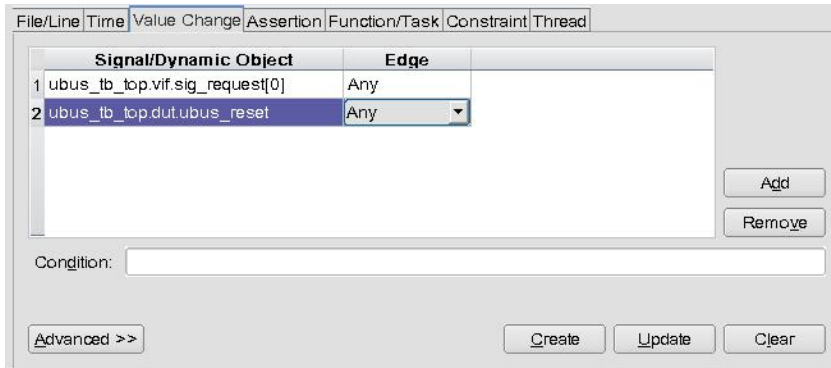
Refer to the following figure and table to see the options and mapping UCLI options.

Figure 37 Advanced Mode in Manage Breakpoints

Mapping UCLI Options:

Field	UCLI Option
File	-file, you can also click the Browse button to specify the file in the <i>Browse for Source File</i> form
Line	-line
Macro/Protect Line Offset	-moffset
Instance	-instance
Class Object	-object, -object_id
Condition	-cond
Skip Frequency	-skip
Executing Frequency: Once	-once
TCL	-once
Continue	-continue
Quiet	-quiet
Additional Options	Specify other UCLI options in this field

- **Time:** Add breakpoints at a time by specifying the simulation time. The time unit depends on the simulation definition.
- **Value Change:** Add breakpoint(s) at an event of a variable value changing.



For HDL Signals:

- i. Click the **Add** button to add a row in the **Signal/Dynamic Object** field.
- ii. Specify a signal with full hierarchy name in the **Signal/Dynamic Object** field.
- iii. Select one of the Any, Rising, or Falling options in the **Edge** field to trigger the simulation stop.
- iv. Click the **Create** button to add the breakpoint.

For Dynamic Objects:

- i. Click the **Add** button to add a row in the **Signal/Dynamic Object** field.
- ii. Specify a member data of an object with full hierarchy name in the **Signal/Dynamic Object** field. The object must belong to the current simulation scope and a testbench scope.
- iii. Only the **Any Change** option can be selected to trigger the simulation stop.
- iv. Click the **Create** button to add the breakpoint.

Alternatively, use the following steps to set a value change breakpoint for a variable of dynamic object:

- i. Select a variable in the **Local** or **Member** tabs or select variables in the *Source Code* pane. The variable must belong to the current simulation scope and a testbench scope.
- ii. Open the *Breakpoints* form using the **Set Breakpoint** command in the right-click command menu. Then, the selected variable name(s) are automatically entered in the **Signal/Dynamic Object** field.

iii. Click the **Create** button to add the breakpoint. If multiple variables are included in a breakpoint, the simulation is stopped while the value of any of the variables is changed.

- **Assertion:** Add breakpoints at an assertion event by specifying the event type to stop the simulation. The event types includes when the assertion event is starting, ending, failing, succeeding, or any above status. These options map to the `-start`, `-end`, `-failure`, `-success`, and `-any` UCLI options. The assertion name should be specified with a full hierarchical name.

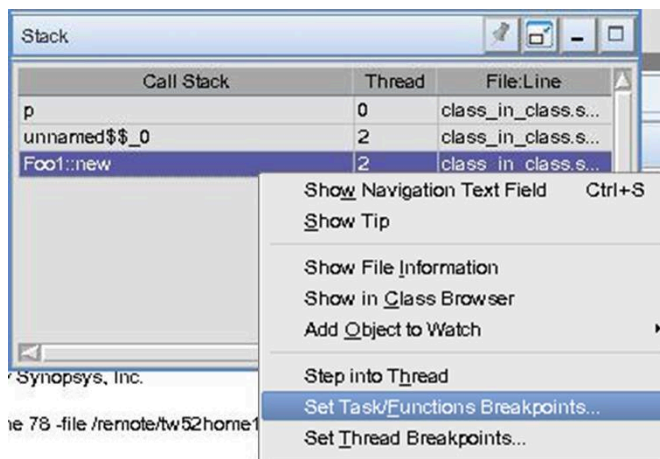
The screenshot shows the 'Assertion' tab of the breakpoint configuration dialog. It has tabs for 'File/Line', 'Time', 'Value Change', 'Assertion', 'Function/Task', 'Constraint', and 'Thread'. Under 'Event Type', there are radio buttons for 'Any', 'Start', 'End', 'Failure' (which is selected), and 'Success'. Below this is a text field labeled 'Breakpoint on Specified Assertion:'.

- **Function/Task:** Add breakpoints at the time of a function/task beginning or ending. Specify the function/task name in the **Break in Task/Function** field and select the timing of the simulation stop, begin or end of the function/task call. The **Break in Specified Task** field maps to the `-in` UCLI option.

The screenshot shows the 'Function/Task' tab of the breakpoint configuration dialog. It has tabs for 'File/Line', 'Time', 'Value Change', 'Assertion', 'Function/Task', 'Constraint', and 'Thread'. Under 'Stop at', there are radio buttons for 'Beginning' (selected) and 'End'. Below this is a text field labeled 'Break in Task/Function:' with the value 'clone'. To the right of this field is a dropdown menu labeled 'LVM'. Below this are two empty text fields labeled 'Class Object:' and 'Condition:'.

Alternatively, use the following methods to set a function/task breakpoint:

- Through the Member or Stack tabs:
 1. Select a member function in the Member or Stack tabs.
 2. Use the **Set Task/Functions Breakpoints** command in the right-click menu.



- Through the **Instance** tab of the *Hierarchy Browser* pane:
 1. Select a node of a function or a task in the Instance tab.
 2. Use the **Set Task/Functions Breakpoints** command in the right-click menu.

The *Breakpoints* form is displayed and the function name is automatically entered in the **Break in Task/Function** field. You can select the row and click the **Jump to Source** button in the *Breakpoints* form to jump the debug cursor to the line in the *Source Code* pane and highlight it.

Note:

Only tasks or functions that are not under a class or unnamed block can be added breakpoints.

After the settings are complete, click the **Create** button to create a new breakpoint. Click the **Update** button to update the setting of the selected existing breakpoint and the **Clear** button to remove the setting of the selected breakpoint. The breakpoint settings can be saved to a breakpoint file by clicking the **Save** button and can be loaded from the breakpoint file by clicking the **Load** button. The load operation replaces the existing breakpoints.

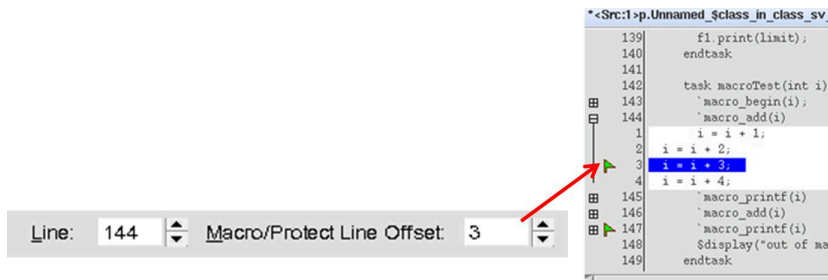
You can use the **Delete** button to delete one breakpoint and the **Delete All** button to delete all breakpoints. Use the **Enable All** or **Disable All** buttons to enable or disable all breakpoints. The breakpoint list remains after using the rewind features.

Set Breakpoint for Macro/Protect Code

The simulation can stop in the statement of a macro or protect code through the following methods:

- When a macro is expanded, advance the simulation to each statement in the macro using the **Step** or **Next** commands.
- Open the *Breakpoints* form using the **Debug -> Manage Breakpoints** command. Specify the line number where the macro or protected code is located in the source code in the **Line** field and the line offset of the macro/protect code in the **Macro/Protect Line Offset** field. Click the **Add** button to create the breakpoint (refer to the figure below).

Figure 38 Add Breakpoint to Macro Code



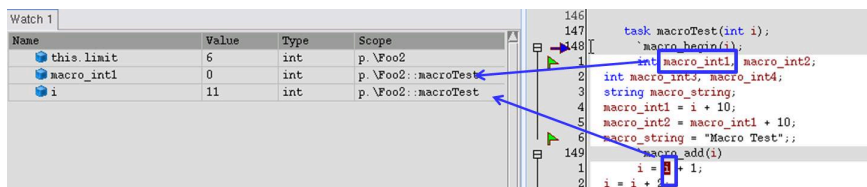
- The breakpoints can be added either before or after entering the *Interactive Simulation Debug* mode. Before entering *Interactive Simulation Debug* mode, turn on (check) the **Source -> Expand Macro toggle** command and add the breakpoint by double-clicking the offset line. After *Interactive Simulation Debug* mode is enabled, this option is automatically enabled and the breakpoint can be added directly by double-clicking the offset line.

Note:

Only protected code that can be expanded can be added as breakpoints.

When the simulation hits the breakpoint and stops in a macro/protected code scope, this macro/protected code is expanded and the debug cursor is placed in the offset of this macro/protected code in the *Source Code* pane. After the simulation enters a macro/protected code scope, the `-macro` option is automatically added to the UCLI commands and you can step-by-step trace the macro/protected code using the or buttons in the interactive toolbar. You can also see the local variables of the scope that the macro belongs to in the **Local** tab and can add variables of the macro into the **Watch** tab.

Figure 39 Add Macro Variable into Watch Tab



5

Consistent Radix Setting and Value Format

The following sections describe the consistent radix setting and value format feature:

- [Consistent GUI Style for Radix and Notation Settings](#)
- [Consistent Radix and Value Format for Primitive Data Types](#)
- [Consistent Value Format for Composite Data Types](#)
- [Support for Radix and Value Format Synchronization in Different Views](#)
- [Change Sequence to Query Value When Cursor Time Equals Simulation Time](#)
- [Displaying Leading Zero for Signals in Interactive Mode](#)

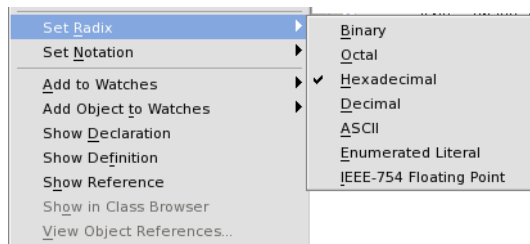
Consistent GUI Style for Radix and Notation Settings

Verdi provides consistent GUI style for radix and notation settings across all views (*nSchema*, *nTrace*, *nWave*, Local pane, Member pane, and Watch pane). The following figures show the new GUI style for radix and notation settings.

- **Set Radix**

The following figure shows the new GUI style settings for the **Set Radix** option:

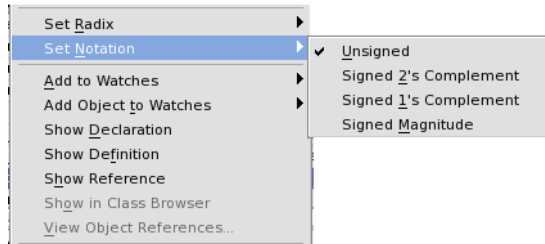
Figure 40 Set Radix Option



- **Set Notation**

The following figure shows the new GUI style settings for the **Set Notation** option:

Figure 41 Set Notation Option



Consistent Radix and Value Format for Primitive Data Types

Verdi supports consistent radix and value format for all primitive data types, for both dumped and non-dumped signals, across all views.

The default radix and notation types for dumped and non-dumped signals are hexadecimal and unsigned respectively.

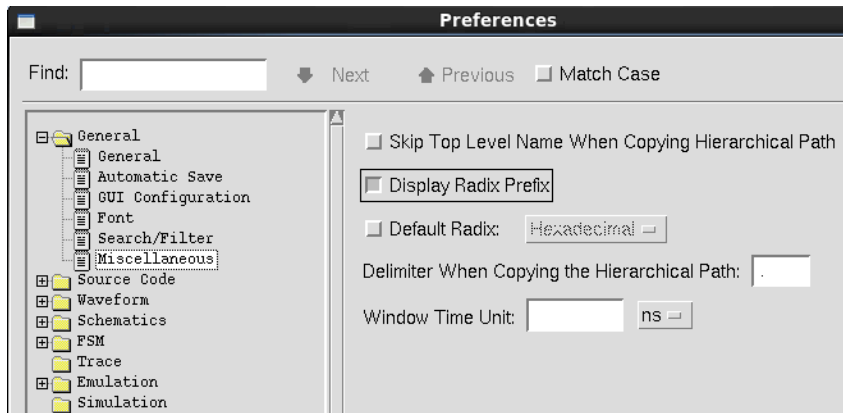
For both dumped and non-dumped signals or variables, signal value format does not have radix base specifier appended at the beginning by default.

Figure 42 Signal Value Without Radix Base Specifier

```
64 | signal dout10 : std_logic_vector(17 downto 0);  
    | 10ee7
```

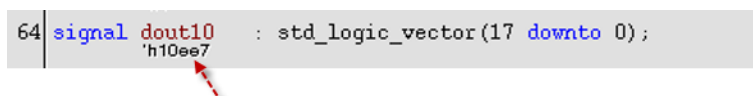
You can use the **Display Radix Prefix** preference option (**General -> Miscellaneous -> Display Radix Prefix**) to control the display of radix base specifier for all views (*nSchema*, *nTrace*, signal list, *nWave*, Local pane, Watch pane, Member pane, UVM Register view, and UVM Resource view).

Figure 43 Display Radix Prefix Option



When the **Display Radix Prefix** preference option is enabled, signal value format begins with preceding apostrophe ('), followed by radix base specifier, and then followed by signal value, as shown in the following figure:

Figure 44 Signal Value With Radix Base Specifier



The following table lists default radix and notation for different primitive data types:

Data Type	Default Radix	Default Notation
bit	Hexadecimal	Unsigned
logic	Hexadecimal	Unsigned
byte	Hexadecimal	Unsigned
short	Hexadecimal	Unsigned
int	Hexadecimal	Unsigned
long	Hexadecimal	Unsigned
integer	Hexadecimal	Unsigned
time	Hexadecimal	Unsigned
enum	Enumerated	Unsigned
VHDL_ENUM	Enumerated	Unsigned

Data Type	Default Radix	Default Notation
VHDL_INTEGER	Decimal	Signed 2's Complement
VHDL_BOOLEAN	Hexadecimal	Unsigned
VHDL_BIT	Hexadecimal	Unsigned
VHDL_TIME	Hexadecimal	Unsigned
VHDL_CHAR	ASCII	Unsigned
VHDL_BIT_VEC	Hexadecimal	Unsigned
VHDL_STDLOGIC	Hexadecimal	Unsigned
VHDL_STDLOGIC_VEC	Hexadecimal	Unsigned

The following table lists some examples of radix and notation:

Data Type	Radix/Notation in Previous Releases	Radix/Notation from 2018.09 Release
logic [7:0]	Hex, 2's Complement	Hex, unsigned
wire [1:2]	Hex, 2's Complement	Hex, unsigned
reg [3:0]	Hex, 2's Complement	Hex, unsigned
bit [1:0]	Bin, unsigned	Hex, unsigned
int/short/long/integer	Dec, 2's Complement	Hex, unsigned
int unsigned	Dec, unsigned	Hex, unsigned
real	Not applicable	Not applicable
time	Dec, 2's Complement	Hex, unsigned
enum	Not applicable	enum, unsigned

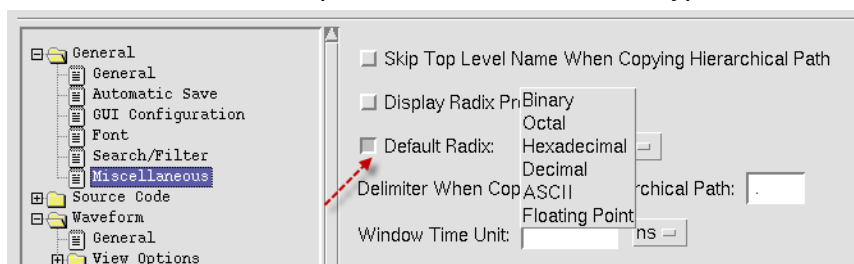
Note:

Starting with the 2018.09 release, the default radix for time, bit, int, short, long and integer data types is Hexadecimal. To disable consistent radix format and revert to the previous release behavior, refer to the *Verdi Migration Guide*.

Global Radix Setting

You can use the **Default Radix** preference option (**General -> Miscellaneous -> Default Radix**) to set global radix for the data types in all views (*nSchema*, *nTrace*, signal list, *nWave*, Local Pane, Watch Pane, Member Pane, UVM Register view, and UVM Resource view).

Figure 45 Default Radix Option to Set Global Radix Types



Consistent Value Format for Composite Data Types

Verdi supports consistent value format for composite data types, for both dumped and non-dumped signals.

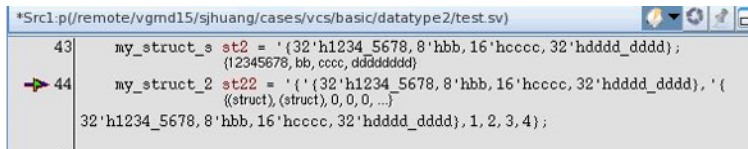
- For packet array, Verdi displays the whole value with hex notation. Verdi does not display value for each dimension of packed array.
- For unpacked array, Verdi displays the first five elements and end with "...".

The following table lists value format examples for composite data types:

Data Type	Value format in Previous Releases	Value format from 2018.09 Release
logic[7:0] macroram [255:0]	size(256)	{0,1,2,3,4...}
bit[3:0][1:0] bytes	'bxxxxxxxx	'hXX
bit[3:0][7:0] bytes [3]	size(3)	{0,1,2,3}
bit[3:0][7:0] bytes [2][3]	size(2)	{size(3),size(3)}

Verdi displays the value only up to the first level of the composite signal. Nested composite signals are not supported. In the following figure, Verdi does not display the values of the first and second data members of `my_struct2` struct type. Verdi displays the values as `{ (struct) , (struct) , 0, 0, 0, ... }`.

Figure 46 Nested Composite Signals



Changing the Radix of Data Member

Verdi applies the following changes if you change the radix for a data member of the selected object:

- Applies the radix change for the selected data member.
- Applies the radix change for the selected data member of other objects instantiated from the same class.

For example, consider the following code:

```
class A;
{
    int mem_a;
    int mem_b;
}
```

If there are three objects instantiated from *class A*, for example, `p.A@1`, `p.A@2`, and `p.A@3`.

If you change the radix for `p.A@1.mem_a` to binary, then Verdi changes the radices of `p.A@2.mem_a` and `p.A@3.mem_a` to binary. However, the radix of `p.A@1.mem_b` is not changed to binary.

Support for Radix and Value Format Synchronization in Different Views

Verdi supports radix and value format synchronization for the signals or testbench variables that are not dumped to FSDB across all views (*nSchema*, *nTrace*, signal list, *nWave*, Local Pane, Watch Pane, and Member Pane).

The radix type of the DUT signals is synchronized across all views.

For a testbench variable, the radix type is synchronized across all views except *nWave*. That is, the testbench variable displayed in *nWave* does not synchronize with other views.

For a testbench variable, radix setting is stored based on the class. Therefore, all objects of that class have the same radix for the member variable.

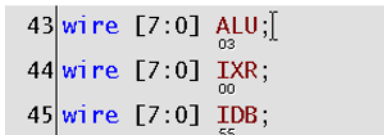
Change Sequence to Query Value When Cursor Time Equals Simulation Time

When cursor time equals simulation time, Verdi initially queries the annotation value from the simulator and then from the FSDB.

Displaying Leading Zero for Signals in Interactive Mode

Verdi displays leading zero for the selected signals in interactive mode. This feature is also supported for signals or variables not dumped to FSDB. This feature adds leading 0 based on signal size, as shown in the following figure.

Figure 47 Support Leading Zero



```
43 wire [7:0] ALU;[
44 wire [7:0] IXR;
45 wire [7:0] IDB;
```

6

Browsing Classes and Dynamic Objects

The Verdi platform provides the *Class pane* and *Object pane* to browse, filter, and search the definitions, member methods of classes, all existing class objects, and member values. After *Interactive Simulation Debug* mode is enabled, the *Class* and *Object panes* are automatically opened and docked as new tabs in the same location as the *Instance* tab of the *Hierarchy Browser* pane. The *Member pane* is also provided to observe the member contents of the selected class. It is opened and docked as a new tab in the same location as the *Local* tab.

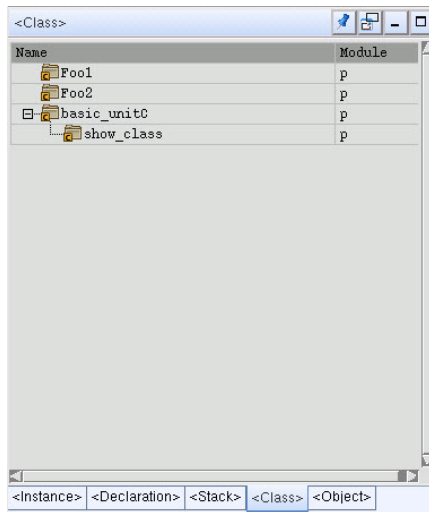
This chapter consists of the following sections:

- [Class Browser](#)
- [Object Browser](#)
- [Viewing Inherited Class Objects Hierarchically in the Local and Watch Panes](#)

Class Browser

The *Class pane* is provided to view all the defined classes and the instantiations with a hierarchical view. Referring to the following figure, all the defined class names are listed in the **Name** column and the scope where each class belongs is shown in the **Module** column.

Figure 48 Class Browser Layout



The instances can be viewed by expanding the class folder in the *Class* pane when the class contains active object instances during simulation. As shown in the following figure, expand the instance folder under the class folder to see the object IDs in the **Name** column. The number of the active object instances is also shown. For classes that are derived from `uvm_object` or `vmm_object`, the object names (value of `m_leaf_name` of UVM objects or name of VMM objects) are also shown.

Figure 49 View Objects in Class Pane

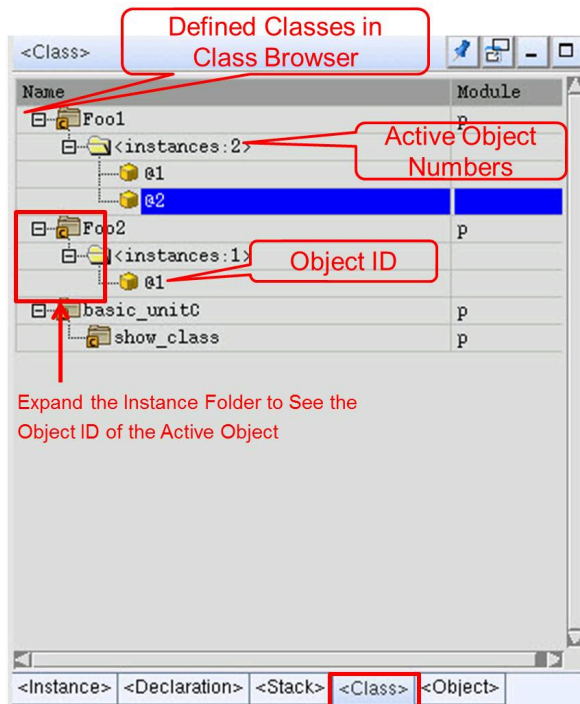


Figure 50 View UVM Object Names in Class pane

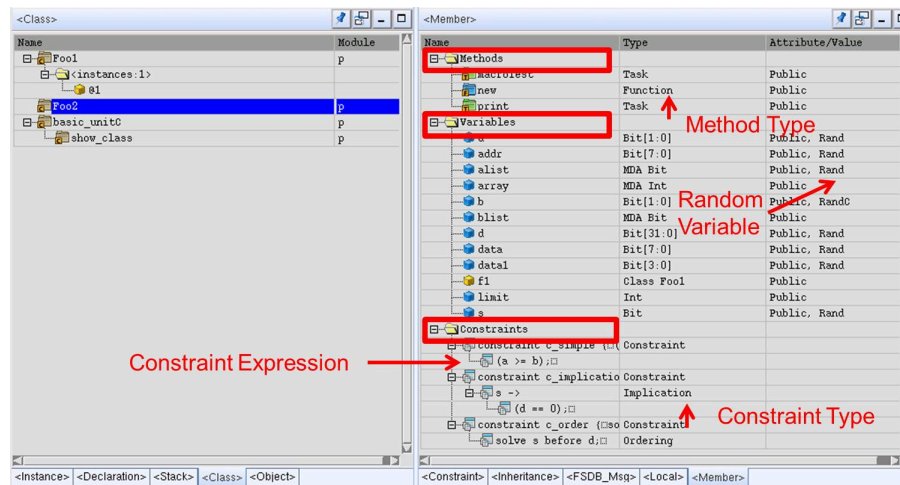
Name	Module
uvm_phase	uvm_pkg
<instances:25>	
@1 "common_end"	
@2 "build"	
@3 "connect"	
@4 "end_of_elabor"	
@5 "start_of_simu"	
@6 "run"	
@7 "extract"	
@8 "check"	
@9 "report"	
@10 "final"	
@11 "uvm_end"	
@12 "uvm_sched"	
@13 "uvm_sched_en"	

When a class or an object in the *Class pane* is selected, the content is automatically shown in the *Member pane*. When you double-click a class folder or an object in the *Class pane*, debug cursor jumps to the location where the class is defined and the class name is highlighted in the *Source Code pane*. Dragging and dropping a class folder or an object from the *Class pane* to the *Source Code pane* has the same effect.

Viewing Class Member Data in Member View

You can use the *Member pane* to see the defined variables of a selected class in the *Class pane*. As you switch to the *Class pane*, the *Member pane* is bound and displayed automatically. After selecting a class in the *Class pane*, the member methods, member variables, and constraints/constraint expression of the class are automatically shown in the *Member pane*. As shown in the following figure, there are three categories for the member data: Methods, Variables and Constraints.

Figure 51 View Class Member Data in the Member Pane



If a class implements interface classes, an Interface Classes category is also shown (refer to the following figure). If a class object is selected, an Information category is shown for displaying create stream, create time, and size of the object. Refer to the [Viewing Object Member Data in Member View](#) section for more details about viewing object members in the *Member pane*.

Figure 52 Member Interface Class

Name	Type
Interface Classes	
log_interfaceCl#(Logic)	
byte_interfaceBase#(logic)	
int_interfaceCl#(int)	
byte_interfaceBase#(int)	
Methods	
myTest	Function

The Methods, Variables and Constraints directories are used to place corresponding member data for the selected class. The member methods are listed under the Methods folder with their names, types, and attributes. The member variables are shown under the Variables folder and the defined data type are shown in the **Type** column. In

the Constraints folder, the constraint blocks can be expanded to see the constraint expressions. The following table shows the Type and Attribute field information for the directory.

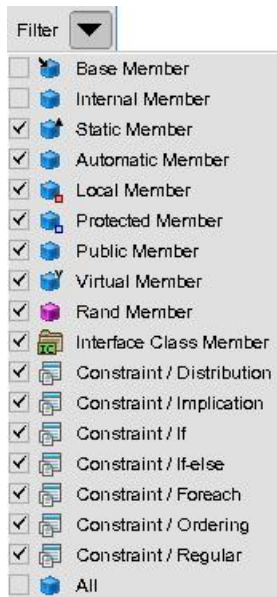
Directory	Type	Attribute
Methods	Task and Function	virtual, static, public, protected, and local
Variables	Defined data type	Attribute and "rand" or "randc" for random variables
Constraints	Types of constraint expression, including implication, constraint ordering, foreach, distribution, constraint if, constraint if else, and regular	NA

When you double-click a member method or a constraint block in the *Member pane*, the debug cursor jumps to the location where the member is defined in the *Source Code* pane.

When you double-click a member variable, the debug cursor jumps to the location where the variable is declared. Dragging and dropping a member data to the *Source Code* pane has the same affect.

You can use the **Navigation Text** field to filter the member data in the *Member pane* for focusing on the specified member data. The member types are provided in the **Filter** drop-down menu (as shown in the following figure). Multiple types can be selected.

Figure 53 Filter Types in Member Tab

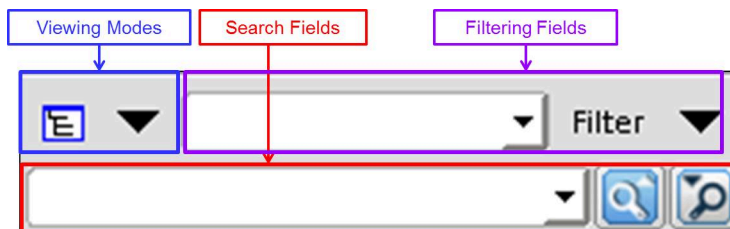


You can also enter the target string in the **Filter** and **Search** fields to filter out or search the member data. Refer to the [Navigating Contents in Object Browser](#) section for more details about using the filter and search features.

Navigating Contents in Class Browser

Content navigation is provided to set the viewing mode of classes and the searching or filtering results in the **Class** tab. The Navigation Text field is shown by default (refer to the following figure). The following paragraphs introduce these features separately.

Figure 54 Navigation Toolbar



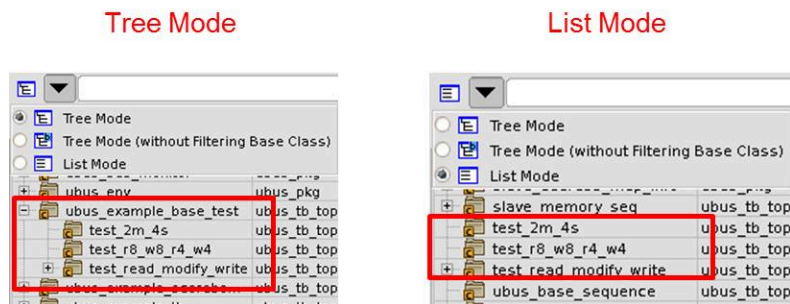
Viewing Modes

The Verdi platform provides three modes for you to view classes: Tree mode (default mode), Tree mode without filtering the base class, and List mode. Select one of these modes from the **Mode** menu.

In Tree mode, the inheritance relationship between all the defined classes is shown with an inheritance view. The root node classes are not derived from any class, the base classes are parent nodes, and the derived classes are child nodes. Tree mode without filtering the base class is similar to Tree mode. However, it keeps the base library classes (that is, VMM/OVM/UVM library classes) visible if there are derived user-defined classes to be shown.

In List mode, all defined classes are flattened. The hierarchical relationships exist only in those classes that have one or more object instances. As an example, refer to the following figure, the `test_2m_4s` derived class is the child of the `ubus_example_base_test` base class in Tree mode and is flattened in List mode.

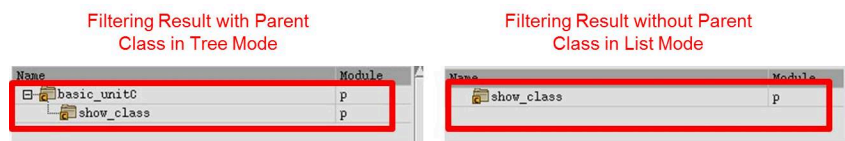
Figure 55 View Modes in Class Browser



Filter Displayed Classes

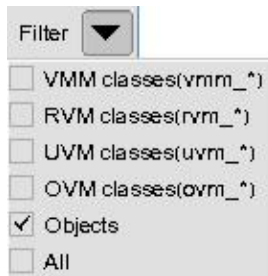
The Verdi platform supports filtering classes to hide the classes that you do not need to observe currently. Enter the target string in the field or select the history string from the drop-down menu. The wildcard (*) character and regular expressions are supported in the target string. Based on the viewing mode features, the filtering result includes the parent class in Tree mode and do not include it in the List mode. For example, enter the `*show*` string in the **Filter** field and press enter. As shown in the following figure, the `show_class` class is found and its parent class `basic_unitC` is also displayed in Tree mode and only the `show_class` class is displayed in List mode.

Figure 56 Filter Classes



The types of VMM, RVM, UVM, OVM library classes can be filtered by selecting the object types in the **Filter** menu (refer to the following figure). When the **All** option is enabled, all of the listed types are filtered. Multiple selections is supported.

Figure 57 Filter Menu



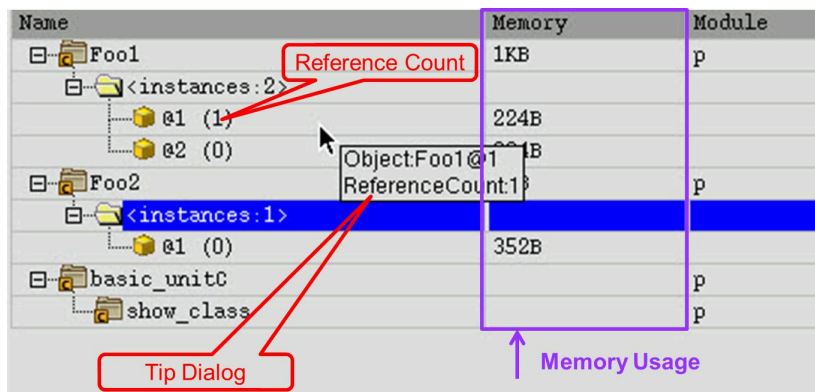
Search Classes

The Verdi platform supports searching classes in the **Class** tab. Enter the target string in the field or select the history string from the drop-down menu. The wildcard (*) character is supported in the target string and searching is case-sensitive. Click the or icons to show the search results. The class is selected and highlighted if there are matching strings.

Show More Information in Class Browser

The following options and commands are provided in right-click menu for showing more detailed information in the **Class** tab and for presenting the classes in different windows of the Verdi platform:

- **Show Reference Count:** Attach the total number of reference counts in parentheses following the object ID.
- **Show Tip:** Show a tip window when the cursor is placed over an object.
- **Show Total Memory:** Add a Memory column to display the memory usage of classes and objects.

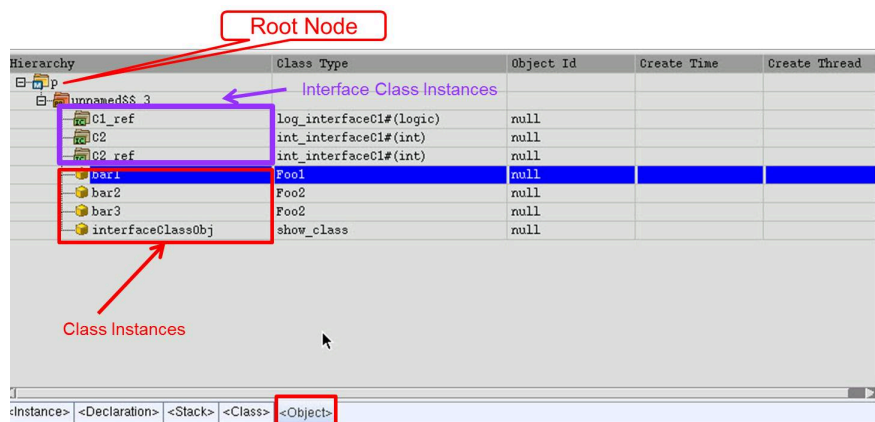


- **Show File Information:** Show the file information of the selected class or instanced object in the **Interconnection** tab of the *Message* pane.
- **Show Create Location:** Show the location where the selected object was instantiated in the *Source Code* pane.
- **View Object References:** Show all objects that reference the selected objects. See the details in the Show Reference Information in Object Browser section.
- **Add to Watch Tab:** Add the selected object ID to the *Watch* tab.
- **Set Constraint Breakpoint:** Invoke the Breakpoints form and add the constraint breakpoint of the selected class.
- **Expand Tree by Level:** Expand the hierarchical tree by specified levels.
- **Collapse All:** Collapse the entire hierarchical tree.
- **Display Source Code in New Tab:** Display the source code of the selected class in a newly opened tab of the *Source Code* pane.

Object Browser

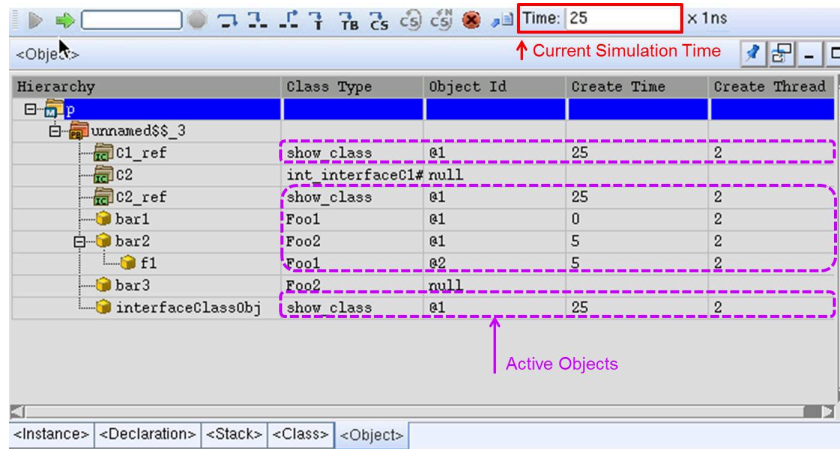
The Object tab is provided to observe the hierarchical and dynamic changes of objects in the instances trees. The root nodes are the modules or packages in the imported design that contain objects or references to objects. Refer to the following figure, the root node, instances of classes (with the  icon), and interface classes (with the  icon) within the instances tree in the **Hierarchy** column are displayed. The class names of the instances are shown in the **Class Type** column. Before executing the simulation, the object IDs listed in the Object ID column are null. The **Create Time** and **Create Thread** columns show the creation time and execution thread of the object.

Figure 58 Object Browser Before Simulation



Use the simulation control commands to advance the simulation. The fields change according to the current paused simulation time. The following figure shows the current active objects when the simulation time is in 25ns. The values of the active objects' ID, created time, and created thread are listed in the **Object Id**, **Create Time**, and **Create Thread** columns.

Figure 59 Object Browser in 25ns



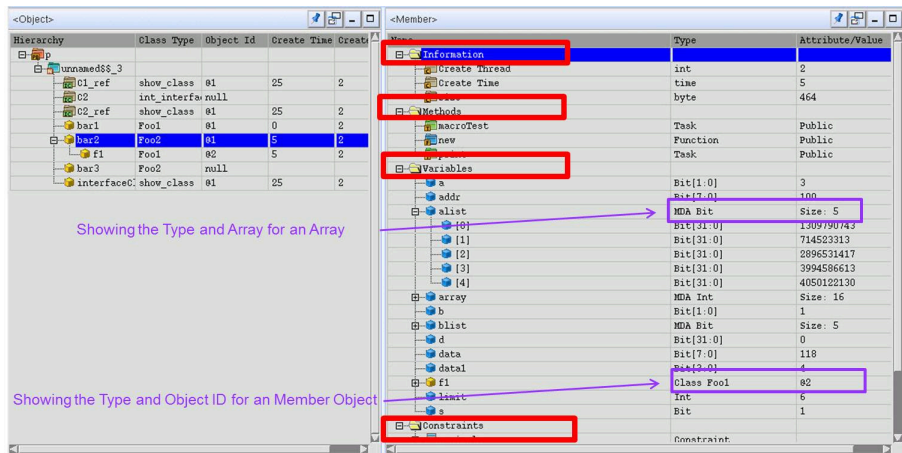
When an object is selected in the **Object** tab, the content of the selected object is automatically shown in the **Member** tab. See more details in the View Object Member Data in Member View section. When you double-click an object in the **Object** tab or drag and drop it to the *Source Code* pane, the debug cursor jumps to the location where the object is declared and the declaration is highlighted in the *Source Code* pane.

Viewing Object Member Data in Member View

Use the **Member** tab to see the contents of a specified object in the **Object** tab. As you switch to the **Object** tab, the **Member** tab is displayed automatically. After selecting an object, the creation information, member methods, member variables, and constraints/constraint expression of the object are automatically shown in the **Member** tab.

As shown in the following figure, the Information, Methods, Variables and Constraints directories are available for viewing the object members. The Information folder shows the information for the thread and creation time of the selected object. The Method folder shows the member methods. Local variables are shown in the Variables folder. The type and size are shown in the **Type** and **Attribute/Value** columns if the variable is an array and you can expand the array to see the array contents. The class type and object ID are shown in the **Type** and **Attribute/Value** columns if the variable is a member object and you can expand the object to check all member data of this member object. The presentation for the Constraints folder is the same as viewing class members (refer to the [Viewing Class Member Data in Member View](#) section for details).

Figure 60 View Object Member Data in Member Tab



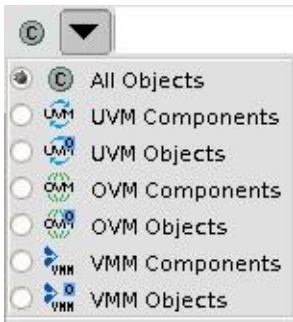
Navigating Contents in Object Browser

You can use the Navigation Text field to specify the view for the classes and to search or filter results in the **Object** tab. The appearance of the Navigation Text field in the **Object** tab and in the **Class** tab is the same.

Viewing Modes

The Verdi platform provides several viewing modes to show the UVM/OVM/VMM components or objects in the Object tab for focusing on the library group of interest. Click the **Mode** menu and select the object types to display. The following table summarizes the supported object group. The **All Objects** option can be selected to view all objects.

Figure 61 Mode Menu in Object Browser



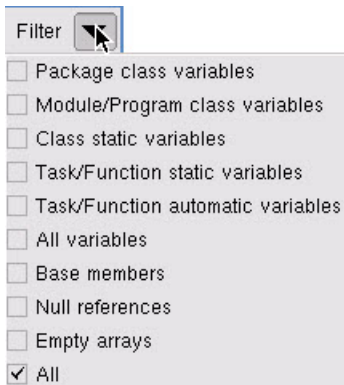
Filtered Type	Derived from	Under the Scope
All Objects (default)	Show objects of all types.	All scopes

UVM Components	Show objects derived from uvm_component	under uvm_pkg.uvm_top.top_levels
UVM Objects	Show objects derived from uvm_object	under uvm_pkg.uvm_top.top_levels
OVM	Show objects derived from	under ovm_pkg.ovm_top_levels
Components	ovm_component	
OVM Objects	Show objects derived from ovm_object	under ovm_pkg.ovm_top_levels
VMM Components	Show objects derived from vmm_group, vmm_env, vmm_simulation	under _vcs_vmm.vmm_object.roots
VMM Objects	Show objects derived from vmm_object, vmm_ral_block_or_sys	under _vcs_vmm.vmm_object.roots

Filter Displayed Objects

The following figure shows the objects types for the VMM, RVM, UVM, OVM library classes in the **Filter** menu. The filtering usage in the **Object** tab is the same as that in the **Class** tab. Refer to the [Navigating Contents in Class Browser](#) section for details.

Figure 62 Filter Menu



Search Objects

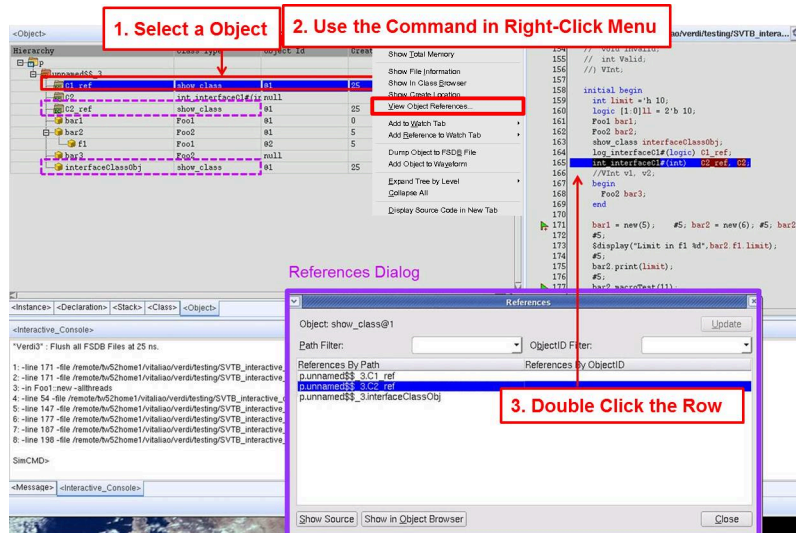
The searching usage in the **Object** tab is the same as that in the **Class** tab. Refer to the [Navigating Contents in Class Browser](#) section for details.

Show Reference Information in Object Browser

Refer to the [Show More Information in Class Browser](#) section for the details about the same commands in the right-click menu. The following commands are different from the **Class** tab for showing more detailed information in the **Object** tab.

- **Show in Class Browser:** Show the class of the selected object in the **Class** tab.
- **View Object References:** Show all objects that reference the same object as the selected object in the References form. As shown in the following figure, the References form lists the objects that have the same `show_class` class type and the same `@1` object ID. They all reference to the same object. You can use the following commands to see more information about the selected reference in the References form:
 - **Double-click a reference:** Highlights the line where the object is declared in the *Source Code* pane.
 - **Show Source:** Highlights the line where the object is declared in the *Source Code*.
 - **Show in Object Browser:** Highlights the reference in the **Object** tab.
 - **Update:** refresh and retrieve the references again after the simulation is advanced.

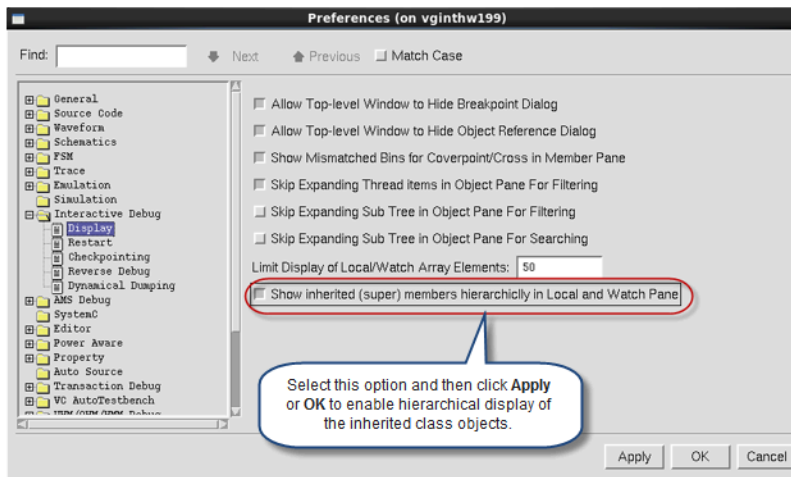
Figure 63 View References Data



Viewing Inherited Class Objects Hierarchically in the Local and Watch Panes

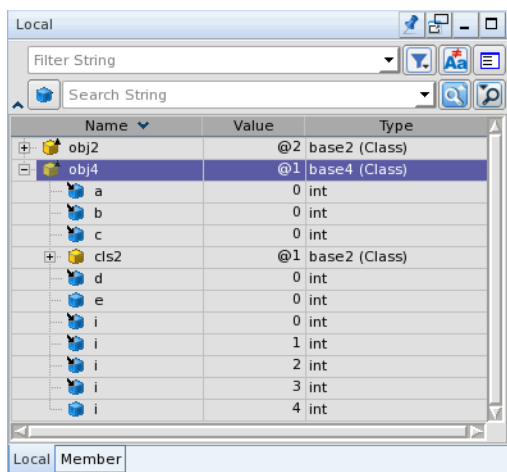
Verdi supports hierarchical display of the inherited class objects in the *Local* and *Watch* panes. To enable this feature, you must enable the **Show Inherited (super) members hierarchically in Local and Watch Pane** option in the *Preferences* form (**Interactive Debug > Display**) as shown in the following figure. This option is recorded in the `novas.rc` file so that Verdi can automatically apply it in next executions.

Figure 64 Enabling Hierarchical Display of the Inherited Class Objects



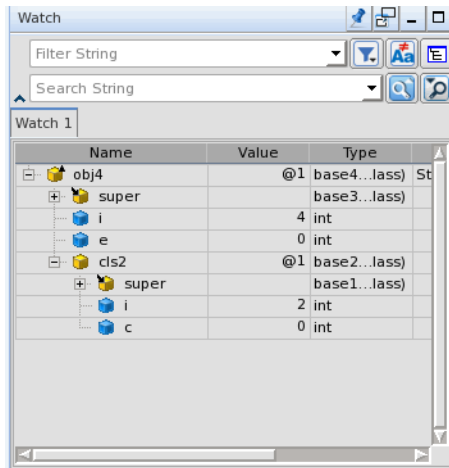
By default, this option is disabled, and the inherited members are displayed in the flattened view in the *Local* and *Watch* panes as shown in the following figure:

Figure 65 Flattened View of the Inherited Class Objects



When the **Show Inherited (super) members hierarchically in Local and Watch Pane** option is enabled, the inherited class members are grouped together as a node named *super*. When the inheritance is deep, the members are grouped recursively and displayed in the form of nested *super* nodes, as shown in the following figure:

Figure 66 Hierarchical View of the Inherited Class Objects



This feature allows you to focus on the desired base objects and have a clear view on members of that object itself.

The *super* node is not either a real member variable or a real class object. The following apply to the *super* node:

- The value is empty because it is not real objects and has no object id
- The type shows the base class type
- The *super* node always lists at the top despite of the sorting setting
- The **Show Declaration** option jumps to the class definition of the selected base class as there is no actual object for the *super* node.
- The **Show Definition** option jumps to the class definition of the selected base class

The following operations are not valid on the *super* node:

- Add object to Watch Pane/nWave
- Dump
- Reverse debug actions, including “Jump to Creation”

Tcl Command

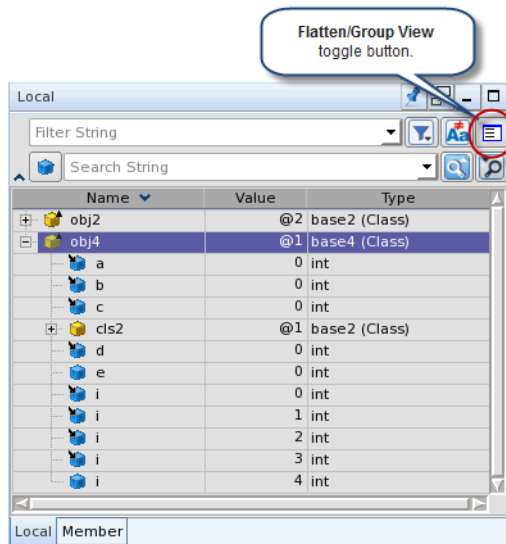
You can use the `-tbvDataViewSuperNode` option in the `tbvSetPreference` Tcl command to enable this feature. Following is the syntax:

```
tbvSetPreference -tbvDataViewSuperNode 1
```

Option Button on the Local/Watch Panes

To view the flatten or hierarchical display of the inherited class objects, the **Flatten/Group View** toggle button is added in the *Local* and *Watch* panes, as shown in the following figure:

Figure 67 Flatten/Group class base members Icon



By default, Verdi displays the **Flatten View** button . When the **Show Inherited (super) members hierarchically in Local and Watch Pane** option is enabled, Verdi enables this feature and displays the **Group View** button  in the *Local* and *Watch* panes.

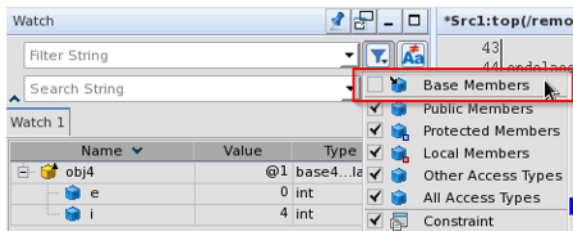
The initial value (**Flatten View/Group View**) of the button is determined by the *Preferences* setting, either changed by you or restored using the `novas.rc` file.

You can click **Flatten/Group View** button in the *Local* and *Watch* panes to toggle between the flatten or hierarchical display of the inherited class objects.

Filter and Search Behavior in the Local/Watch Panes

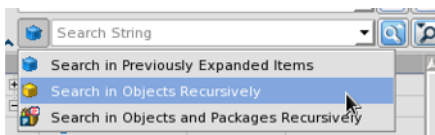
When the filter menu option **Base Members** is not selected, the *super* node is not displayed.

Figure 68 *Base Members Filter Option*



In the *Local pane*, the search options **Search in Previously Expanded Items** and **Search in Objects Recursively** treat the *super* node as a normal object node. You must use the **Base Members** filter option to decide whether the search excludes the *super* node.

Figure 69 *Search Options*



7

Stack and Thread View

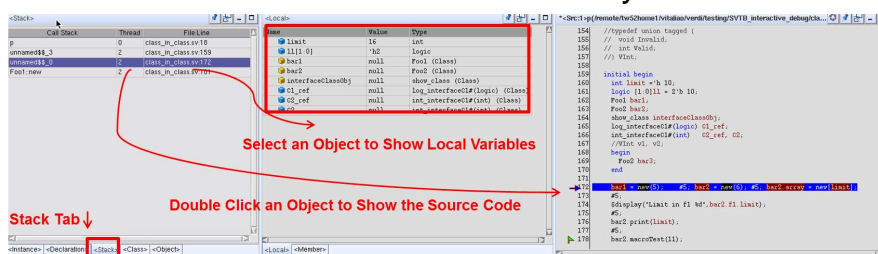
The Verdi platform provides the *Stack/Thread View* panes to show the testbench dynamic hierarchy tree along with all the testbench threads and their status. These panes are in the same location as the *Instance* pane as new tabs. There are two viewing modes for all the threads of the program execution in the *Stack* pane, Active mode and Thread mode. When Thread mode is enabled, the call stack of all current threads are shown. When Active mode is enabled, only active *Stack* panes are shown.

- [Active Mode](#)
- [Thread Mode](#)

Active Mode

The *Stack* pane appears empty at time 0. The dynamic objects are displayed as and when they are created in the testbench during simulation. As you switch to the *Stack* pane, the *Local* tab is displayed automatically. The thread ID is shown in the **Thread** column and the file name and line number are shown in the **File:Line** column. The call stack of the active thread is shown in the *Stack* pane with **Active** mode enabled by default (refer to the figure).

Figure 70 Stack View with Active Mode Enabled by Default



The debug cursor jumps to the line in the *Source Code* pane when you double-click the row of the dynamic object. You can repeat the operations to check the execution sequences of the program. The local variables are shown in the *Local* tab at the same time.

Thread Mode

You can switch to Thread mode from the default Active mode to observe all the threads:

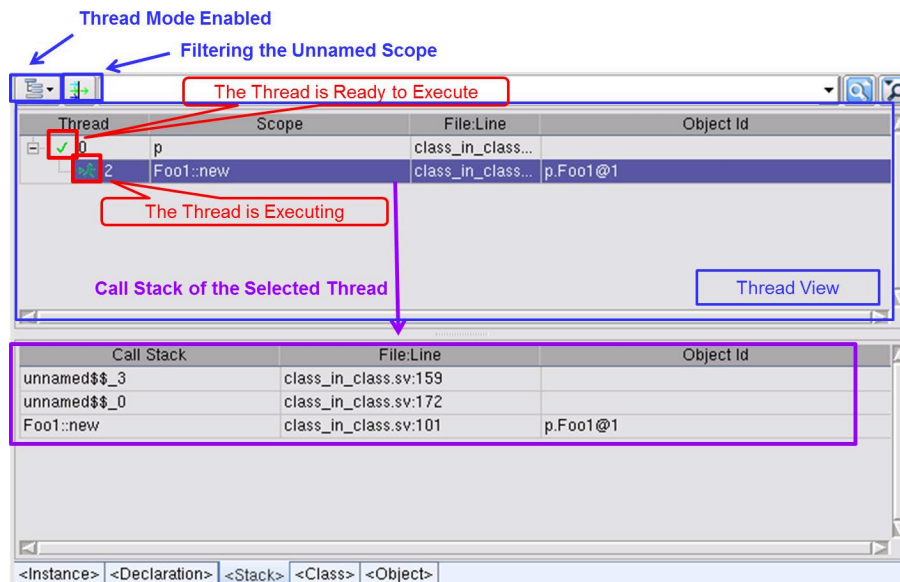
Click the  button in the  Navigation Text field and select **Thread Mode** option in the menu (as shown in the following figure) to enable Thread mode.

Figure 71 Enable Thread Mode



After Thread mode is enabled, the  button is changed to the  button and the *Stack* pane is separated into two panes. Threads can be viewed in the upper pane (the *Thread* pane). Selecting a thread in the upper pane displays its call stack in the lower pane (the *Stack* pane, as shown in the following figure). The unnamed scope can be hidden by clicking the  button.

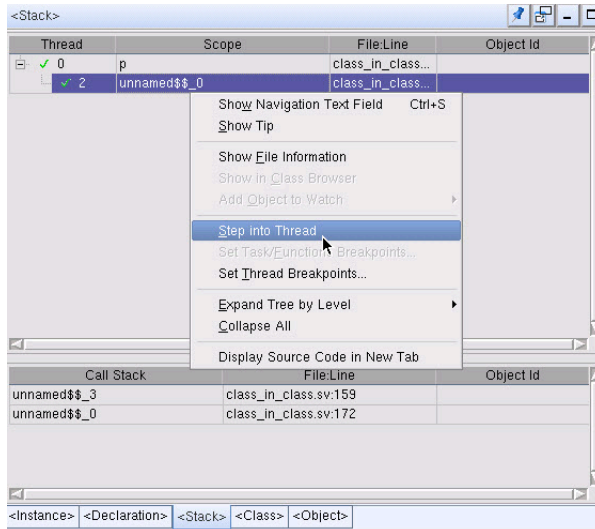
Figure 72 Thread Mode Layout



The full thread tree is shown in the *Thread* pane. You can see the threads with a hierarchical view. The thread with the  icon indicates the thread is executing and the thread with the  icon indicates the thread is ready to execute. Right-click a ready thread

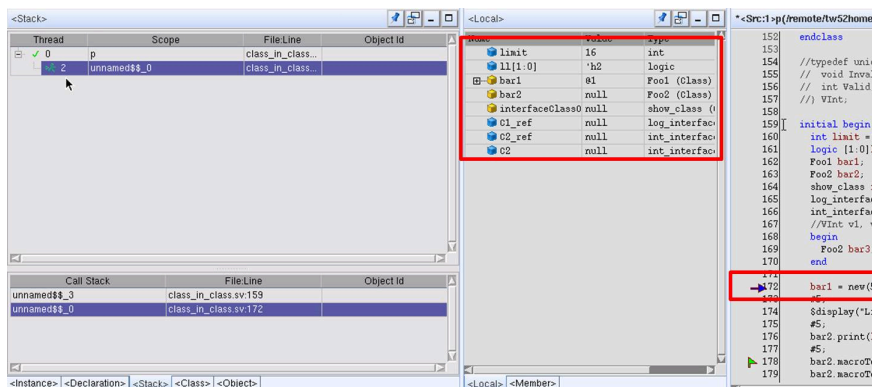
in the *Thread* pane and choose the **Step into Thread** command in the menu to advance the simulation until entering the selected thread (refer to the following figure).

Figure 73 Select Step into Thread Command



As shown in the following figure, the simulator stops when the simulation enters the selected thread. The local variables are updated to map the current scope in the *Local* tab and the debug cursor jumps to the location where the simulation stopped in the line of the *Source Code* pane.

Figure 74 Step into Selected Thread



8

Dynamic Dumping

The Verdi platform provides a dynamic dumping mechanism to dynamically dump the HDL signals to the `inter.fsdb` FSDB file by default. You can check the waveform of the dumped signals in the *nWave* window while Interactive Simulation Debug mode is enabled. In addition, objects and member data can also be dumped to the FSDB file.

This chapter consists of the following sections:

- [HDL Example](#)
- [Dump and Get Signals](#)
- [Dumping Options](#)
- [Dumping Dynamic Objects](#)
- [Dump Batch Signals](#)

HDL Example

```
`timescale 1 ns / 1ns

module memory (
    input wire read,
    input wire write,
    input wire [7:0] data_in,
    input wire [3:0] address,
    output reg [7:0] data_out
);

reg [7:0] mem [0:15];

always @ (*)
begin
    if (write) mem[address] = data_in;
end

always @ (read, address)
begin
    if (read) data_out = mem[address];
end
endmodule
```

```

module memory_example();

reg read,write = 0;
wire op = 0;
reg [31:0] data_in = 0;
reg [3:0] address = 0;
wire [31:0] data_out;
reg clk;
assign op = write?1'b1:(read?1'b1:1'bz);

memory U0 (read, write, data_in[7:0], address, data_out[7:0]);
memory U1 (read, write, data_in[15:8], address, data_out[15:8]);
memory U2 (read, write, data_in[23:16], address, data_out[23:16]);
memory U3 (read, write, data_in[31:24], address, data_out[31:24]);

always #1 clk = ~clk;

initial begin

    $monitor ("%g read=%b write=%b address=%b data_in=%h data_out=%h",
        $time, read, write, address, data_in, data_out);
    clk = 0;
    #1 read = 0; // why only for read
    #3 repeat (16) begin
        data_in = $random;
        write = 1;
    #1 address = address + 1;
    end
    write = 0;
    address = 0;
    #3 repeat (16) begin
        read = 1;
        #1 address = address + 1;
    end
    read = 0;
    #1 $finish;
end
endmodule

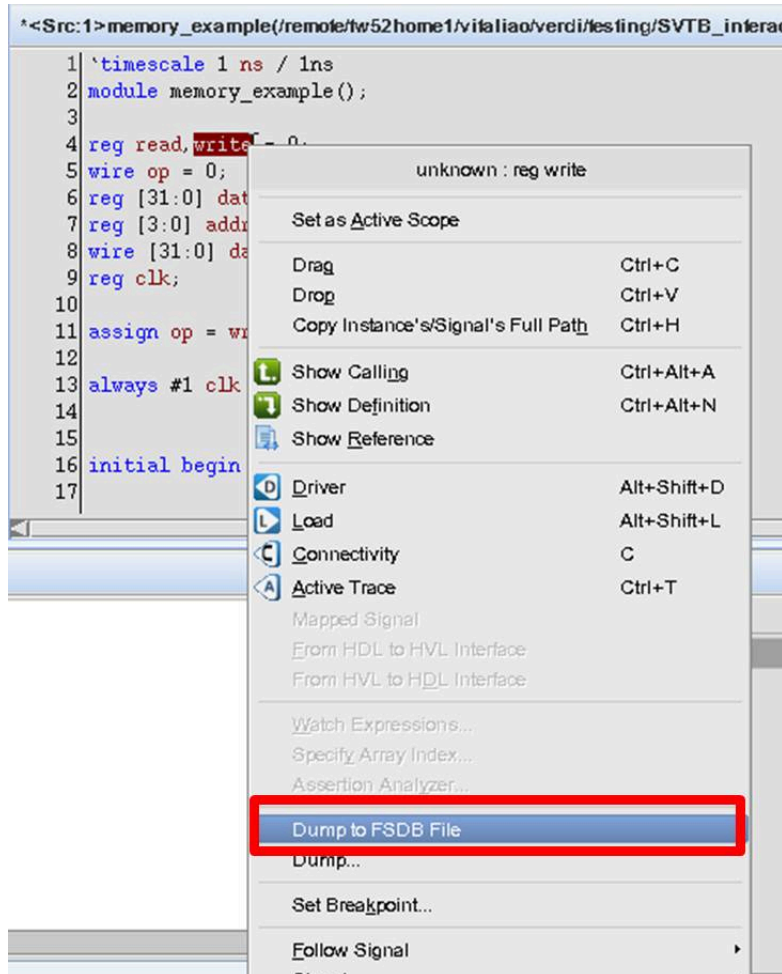
```

Dump and Get Signals

After Interactive Simulation Debug mode is enabled, the `inter.fsdb` FSDB file is automatically created by the Verdi platform. This file is used during Interactive Simulation Debug mode. Complete the following steps to dump signals to the FSDB file and view the waveform in the *nWave* window.

1. Select a signal in the *Source Code* pane and choose the **Dump to FSDB File** command from the right-click menu (refer to the following figure).

Figure 75 Dump Signal to FSDB File

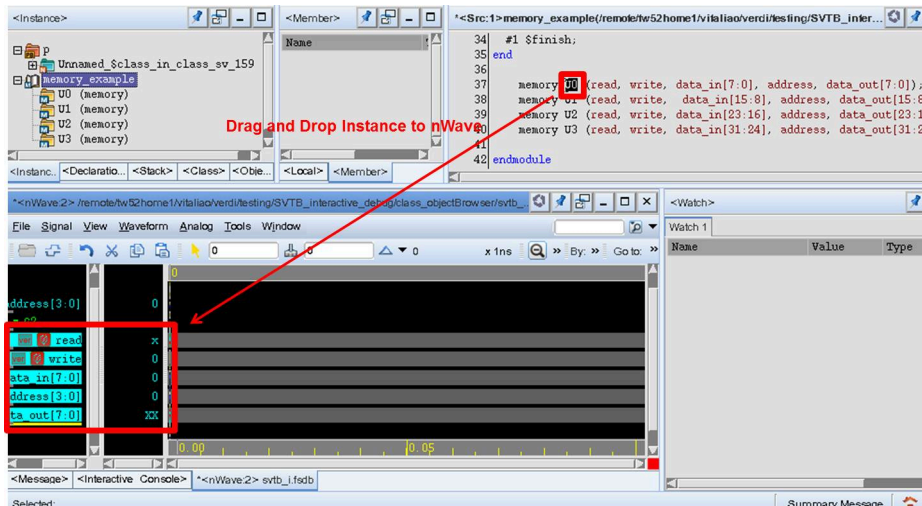


The Verdi platform submits the call to the `fsdbDumpvars depth {signal full-path name}` Tcl dumping command for the simulator to start recording the value of specified signals and the call to the `fsdbDumpflush` Tcl dumping command to write signals to the FSDB file. The values of the dumped signals are recorded from the current time. You can dump signals at any simulation time during Interactive Simulation Debug mode. Refer to the Tcl Dumping Command appendix for more details about the supported Tcl dumping commands.

2. Click the  button to open the `inter.fsdb` FSDB file in the *nWave* window. Use the **Signal -> Get Signals** command to show the dumped sign.

- Alternatively, directly drag and drop the selected signals from the *Source Code* pane to the *nWave window* with the loaded *inter.fsdb* FSDB file. The Verdi platform triggers the dynamic dumping mechanism to dump the signals if the signals cannot be found in the current FSDB file.
- The HDL instance can also be dragged and dropped to the *nWave window* to dump and add all the input/output ports of the instance to the FSDB file (refer to the following figure).

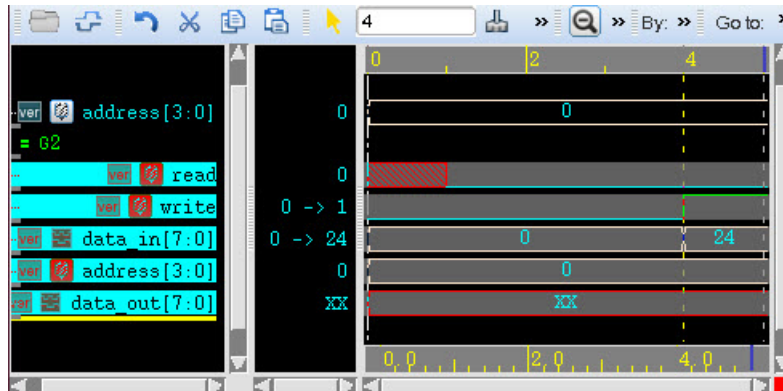
Figure 76 Dump and Add Instance Ports to FSDB File



You can also drag an HDL node from the **Instance** or **Declaration** tabs of the *Hierarchy Browser* pane to the *nWave window* to add one level of signals. In addition, you can select a node and use the **Dump to FSDB File** command in the right-click menu to add all level of signals.

5. Run the simulation 5ns by inputting the run 5ns command in the Command Console pane. Refer to the following figure, the signals shown in the nWave window are advanced 5ns.

Figure 77 Dumped Signals in nWave at 5ns



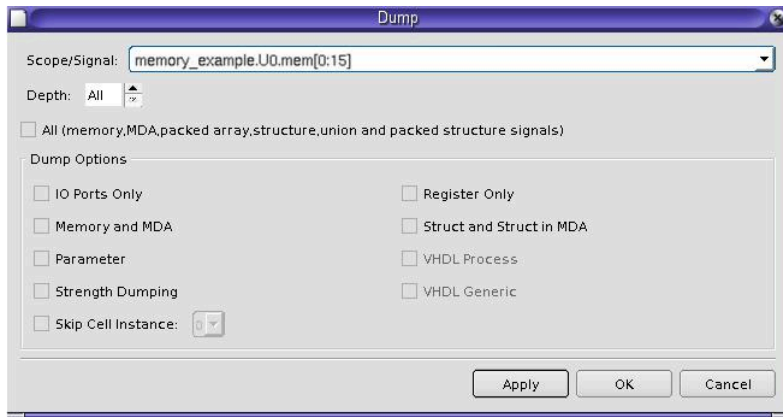
Dumping Options

You can use the **Dump** command to set the dumping configurations in the Dump form as follows.

1. Select an instance or signal in the *Source Code* pane.
2. Use the **Dump** command from the right-click command menu. The selected scope or signal is automatically shown in the **Scope/Signal** field as shown in the following figure. You can also directly enter the specified scope or signal on demand.
3. If the selected item is a scope, you can specify the dumping depth in the Depth field. Set '1' in the Depth field to dump one level of signals for the active scope. Set '2' in the

Depth to dump the two levels of signals for the active scope and so on. You can set "All" in the Depth field to dump the instance to the FSDB file.

Figure 78 Dump Options

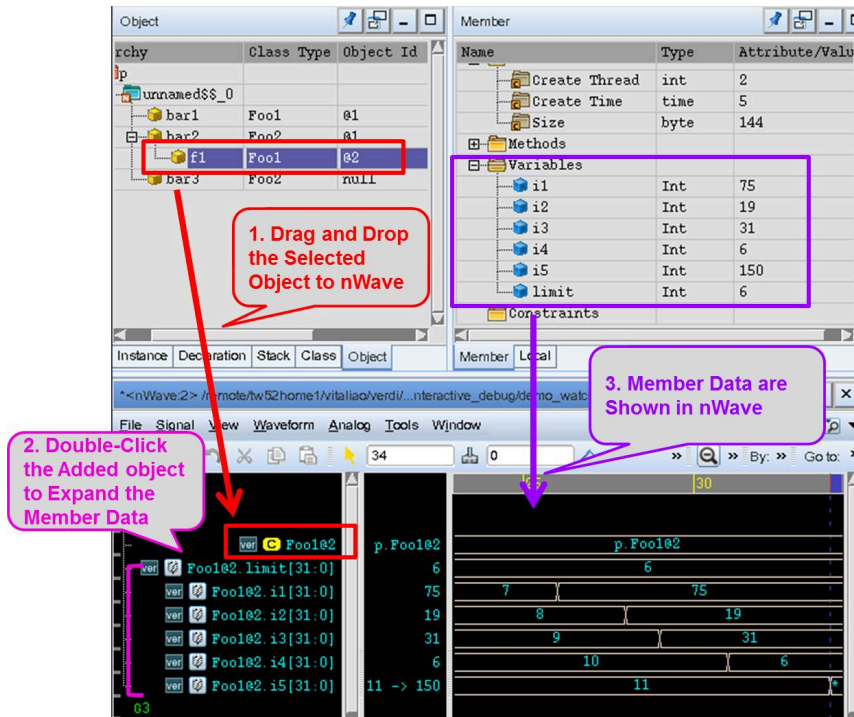


The data types of memory, MDA, packed array, structure, union, and packed structure signals can be disabled from dumping by turning off the **All** option to get better performance and prevent a larger waveform size. You can also enable the signal categories in the **Dump Options** field to dump the specified signal types in the scope.

Dumping Dynamic Objects

Dumping objects and data members to an FSDB file is also available. You can directly select a variable from the Class, Object, Member, Watch, or Local tabs and drag and drop the variable to the *nWave* window with the loaded FSDB file. This dumps the variable to *inter.fsdb* FSDB file and add it to the *nWave* window (refer to the following figure). When the select variable is a class object, the value of the added variable will be the value of the class pointer.

Figure 79 Object Dumping Including Data Member



After adding objects to the *nWave* window, double-click the object to expand it and see its member data (refer to the figure above). If there is a data member with class type, you can add it as an individual variable in the *nWave* window. As shown in the following figure, right-click the variable in the waveform and use the **Add Object** command to add this object to the *nWave* window.

Figure 80 Directly Add Object in nWave



After adding the data member with class type, you can expand it as the added object described above.

Dump Batch Signals

You can restore signals from the saved .rc file to automatically dump all batch signals in one time and then add them in the *nWave* window automatically as follows:

- In an opened *nWave* window with a loaded inter.fsdb file, use the **File -> Restore Signal** command to open the *Restore Signal* form
- Select a *saved .rc* file and click the **OK** button to load this file

After the .rc file is loaded to the *nWave* window, all the signals saved in the .rc file is dumped to the FSDB file and added to the *nWave* window.

Note:

Dumping batch signals does not support novas.rc file. You need to save a .rc file in the *nWave* window.

9

Rebuild and Restart

After *Interactive Simulation Debug* mode is enabled, you can use the Restart feature to restart the current simulation. If your design is modified, you can also use the Rebuild and Restart feature to re-build the simulation with your design and restart the simulation in the *Interactive Simulation Debug* mode.

This chapter consists of the following sections:

- [Restart](#)
- [Rebuild and Restart](#)

Restart

You can use the **Simulation -> Restart** command to restart the current simulation. After invoking the Restart command, the following actions are restarted:

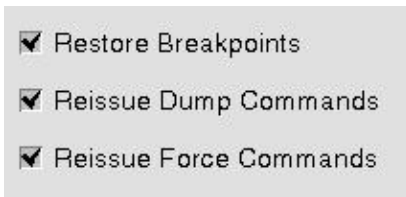
- Setting breakpoints.
- Dumping and adding signals in the *nWave* window (the signals that were previously dumped into the `inter.fsdb` file are automatically dumped after restarting the simulation).
- Issue force commands for HDL signals.

Note:

Only the force commands issued for HDL signals are applied. The force commands from script files or Interactive Console pane are ignored.

In the **Interactive Debug-> Restart** page of the *Preferences* form (invoked with the **Tools -> Preferences** command), you can also disable the options (refer to the following figure) before restarting simulation to disable restart of these actions.

Figure 81 Restart Options



Note:

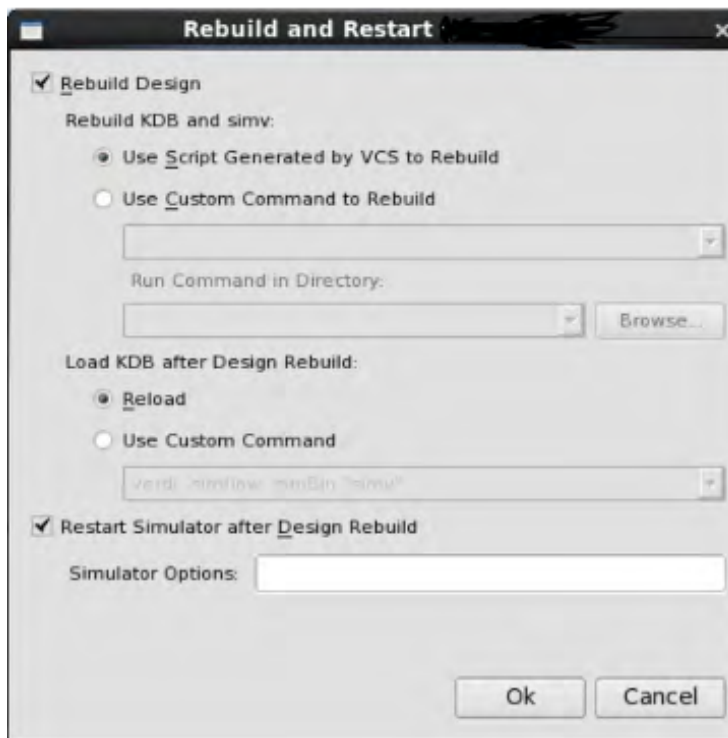
These settings for restart options are applied after invoking the **Restart** command.

Rebuild and Restart

When your design is modified, you can use the rebuild and restart features to rebuild the KDB and the simv simulator executable in the *Interactive Simulation Debug* mode by using the VCS script or your own custom script. Meanwhile, you can decide whether to restart the simulation after the design rebuilt. After rebuilding the KDB and the simv simulator executable, the KDB can be loaded automatically.

Invoke the Rebuild and Restart form by using the **Simulation -> Rebuild and Restart** command to specify the configurations of rebuild and restart features.

Figure 82 Rebuild and Restart Configurations



Where,

- **Rebuild KDB and simv:** Verdi uses the specified method to rebuild the KDB and the simv simulator executable files.
- **Load KDB after Design Rebuild:** Verdi uses the specified method to load the KDB after the design is rebuilt.
- **Restart Simulator after Design Rebuild:** When the option is enabled, Verdi restarts the simulation after the design is rebuilt and loaded.

After completing the setting of the configuration, click **OK**. Verdi performs the following actions:

1. Rebuild the KDB and the simv simulator executable files.
2. Load the KDB.
3. If the **Restart Simulator after Design Rebuild** option is enabled, restart the simulation.

After the simulation is rebuilt and restarted, the GUI layouts and the following data of *Interactive Simulation Debug* mode is restored:

- The objects dumped in the FSDB file
- The breakpoint commands, force and release commands, and signals shown in the *nWave* window.

Note:

This feature can be controlled in the **Interactive Debug -> Restart** page (refer to the Restart section for details).

Limitation

- Rebuild and Restart flow is not supported for Mixed Signal Verification flow.

Configurations of Rebuild KDB and simv

You can enable the **Use Custom Command to Rebuild** option and type the rebuild command directly in the field to rebuild the KDB and the simv simulator executable files.

Rebuild KDB and simv:

☐ Use Script Generated by VCS to Rebuild

☒ Use Custom Command to Rebuild

`vcs -lca -full64 -debug_access+all -sverilog -timescale=1ns/1ns`

The current working directory is used by default. If you want to run the rebuild commands in another directory, you can specify the path in the **Run Command in Directory** field.

Rebuild KDB and simv:

☐ Use Script Generated by VCS to Rebuild

☒ Use Custom Command to Rebuild

`vcs -debug_access+all -full64 [+UVM_TESTNAME=test_read_mod`

Run Command in Directory:

`:/integrated/ubus_evdp_ntb/examples/New Folder`

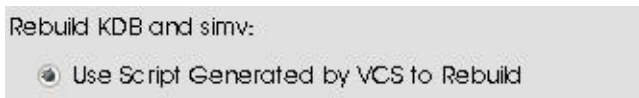
When you have your own custom script file, you can also enable this option and use the script file to perform the rebuild.



Note:

It is recommended to include only the compile and elaborate commands in the scripts.

VCS generates the `vcs_rebuild` rebuild script (located in the `simv.daidir/vcs_rebuild` path) automatically when VCS is invoked each time. You can use the commands recorded in this file to rebuild the KDB and the simv simulator executable files by enabling the **Use Script Generated by VCS to Rebuild** option.



Note:

The **Use Script Generated by VCS to Rebuild** method is recommended for Verilog-only single-VCS compilation flows. It does not support UUM, MX, slave mode, or multi-step builds. For complex compilation script, UUM, MX or multi-step builds, use the **Use Custom Command to Rebuild** method.

Configurations of Load KDB after Design Rebuild

You can use one of the following toggle options in the Rebuild and Restart form to load the KDB:



- **Reload:** When this option is enabled, the options which were used previously to load the KDB are used to reload the KDB. This option is enabled by default.
- **Load KDB from 'simv.kdb' Generated by VCS:** If your KDB file is generated by the Unified Compiler front-end flow, this option is provided to load the KDB with the options generated in the `simv.kdb` file.

- **Use Custom Command:** You can use custom commands or a script file to load the KDB.

Note:

If files are added from your design, you need to use new import commands or a new script to import those files.

Configurations of Restart Simulator after Design Rebuild

You can enable the **Restart Simulator after Design Rebuild** option to restart the simulation in *Verdi Interactive Simulation Debug* mode after the design is rebuilt. The option is enabled by default.



You can enable the **Restart Simulator after Design Rebuild** option to restart the simulation in *Interactive Simulation Debug* mode after the design is rebuilt. The option is enabled by default.

10

Record and Rewind the Simulation Status

You can capture several simulation times using the checkpoint feature and can go back to any of those simulation points (checkpoints) using the rewind feature during the *Interactive Simulation Debug* mode.

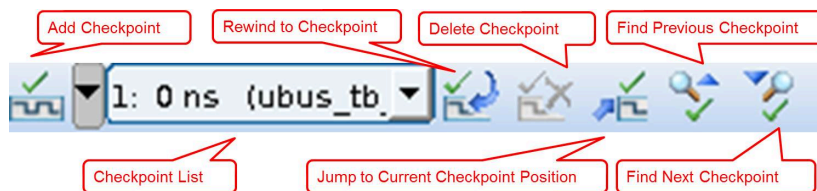
The Verdi platform provides the following methods to add checkpoints:

- [Add Checkpoints Manually](#)
- [Add Checkpoints Automatically](#)
- [Rewind to Checkpoint](#)

Add Checkpoints Manually

As shown in the following figure, the checkpoint commands are provided in the toolbar.

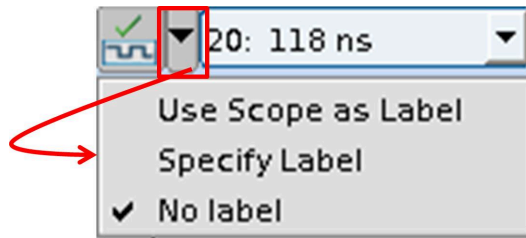
Figure 83 Checkpoint Toolbar



When the simulation proceeds to a stage you want to take a record, you can click the  icon to add a checkpoint. The checkpoint records the data based on the current simulation time and you can add several checkpoints to the same time.

As shown in the following figure, you can also click the  icon (next to ) to specify the label for a next new checkpoint.

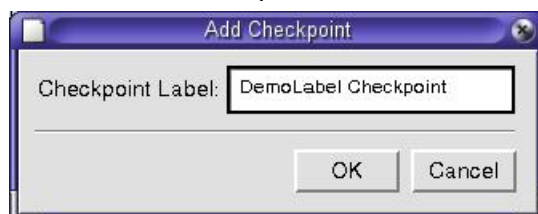
Figure 84 Checkpoint Label



Perform the following steps to add checkpoints manually:

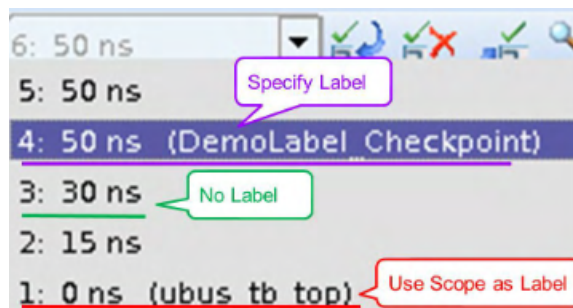
1. Select an option from **Use Scope as Label**, **Specify Label**, and **No Label** for naming checkpoints. The **Use Scope as Label** option is the default value.
2. Click the  icon to add a new checkpoint for the current simulation status.
3. If you select the **Specify Label** option in the first step, the *Add Checkpoint* form is opened. In this form, you can specify the label name in the **Checkpoint Label** field.

Figure 85 Add Checkpoint Form



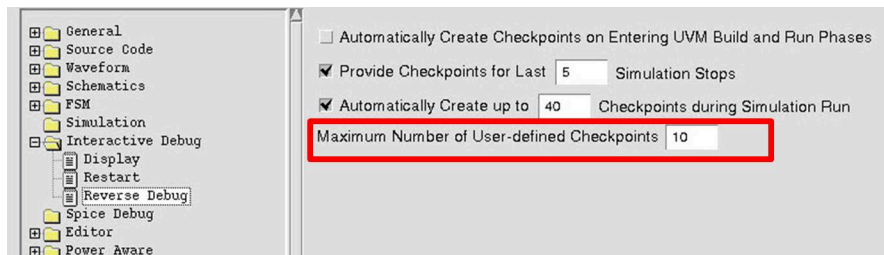
The added checkpoints are listed in the drop-down menu with the `index:time [label]` format. For example, the following figure shows the six added checkpoints. Some checkpoints have the same simulation time. The checkpoint `1:0ns (ubus_tb_top)` uses active scope as the label and the checkpoint `4:50ns (DemoLabel_Checkpoint)` has its specified label.

Figure 86 Checkpoint List



You can specify the maximum number for you defined checkpoints. As shown in the following figure, use the **Tools -> Preferences** command to open the *Preferences* form. In the *Preferences* form, select the **Interactive Debug -> Reverse Debug** page and specify the number in the **Maximum Number of User-defined Checkpoints** field. The default value is 10.

Figure 87 Specify Number of User-Defined Checkpoints



Add Checkpoints Automatically

The Verdi platform also allows you to create checkpoints automatically for simulation advancement caused by the simulation commands or by time. You can then go back to the previous simulation status to undo the last simulation commands or go back to one of the saved interval.

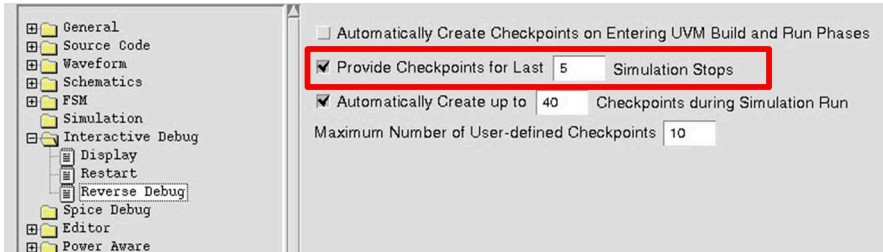
Add Checkpoints Automatically for Last Simulation Stops

Checkpoints can be saved automatically for the last simulation stops. You can enable this feature by the following steps (refer to the following figure):

1. Use the **Tool -> Preferences** command to open the *Preferences* form
2. Select the **Interactive Debug -> Reverse Debug** page and enable the **Provide Checkpoints for Last <number> Simulation Stops** option
3. You can also specify <number> in the field for the number of last checkpoints to be created automatically for the simulation stops. The default value is 5.

4. Click **OK** in the *Preferences* form.

Figure 88 Checkpoints for Simulation Stops



After the feature is enabled and a simulation control command is executed, the checkpoint of the previous simulation status is saved automatically. For example, when you:

1. Click the  **Next** icon twice.
2. Click the  **Step** icon twice.
3. Run 1ns simulation.

As shown in the following figure, the checkpoints are created automatically with the `index:time (after <simulation control command>)` format.

Figure 89 Automatically Created Checkpoints List

```
17: 21 ns (after run)
16: 20 ns (after step)
15: 20 ns (after step)
14: 20 ns (after next)
13: 15 ns (after next)
```

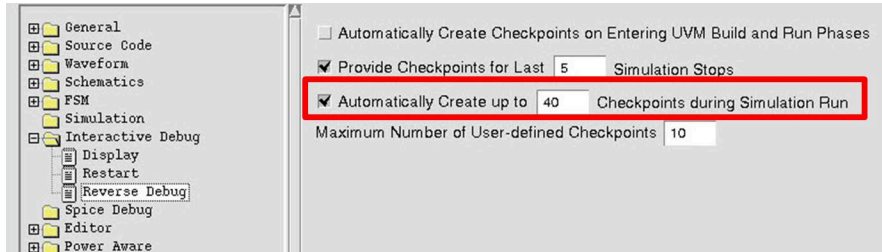
Add Time Checkpoints Automatically for Last Simulation Stops

Checkpoints can also be saved automatically during the simulation based on the specified interval. You can enable this feature by the following steps (refer to the following figure):

1. Use the **Tool -> Preferences** command to open the *Preferences* form.
2. Select the **Interactive Debug -> Reverse Debug** page and enable the **Automatically Create up to <number> Checkpoints during Simulation Run** option.
3. You can also specify <number> in the field for the maximum number of the checkpoints to be created automatically. The default value is 40.

4. Click **OK** in the *Preferences* form.

Figure 90 Checkpoints for Simulation Stops

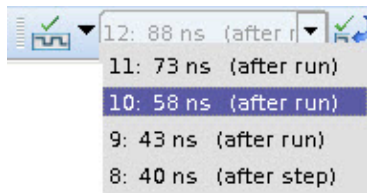


After the feature is enabled and a simulation is executed, the time checkpoints are created automatically with the `index:time (Automatic Time Checkpoint)` format.

Rewind to Checkpoint

After adding a checkpoint, the simulation can go back to the checkpoint from the current simulation time. Select one checkpoint from the drop-down menu and use the **Rewind to Checkpoint** command by clicking the  toolbar icon. If the simulation is rewound to a previous time, all the checkpoints after that point are removed. For example, referring to the following figure, the ninth and tenth checkpoints are removed after rewinding when the 8:40ns checkpoint is selected and the **Rewind to Checkpoint** command is used.

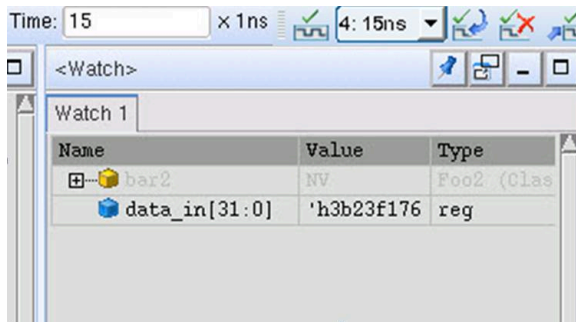
Figure 91 Checkpoint List



The following figure shows the `data_in[31:0]` signal and the `bar2` object in the *Watch* tab when the simulation time is 15ns. The value of the `data_in[31:0]` signal is 'h3b23f176'. The `bar2` object is not active and has no value.

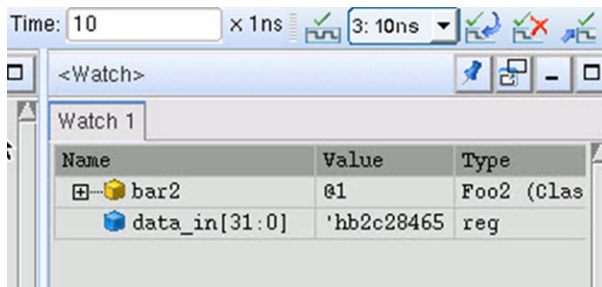
Select the checkpoint in time 10ns which is added previously and click the  button to rewind the simulation to time 10ns.

Figure 92 *data_in and bar2 in Watch Tab at 15ns*



The following figure shows the results from the rewind for the `data_in[31:0]` signal with 'hb2c28465' value and `bar2` having the @1 object id.

Figure 93 *Rewind Session to 10ns*



If there are too many checkpoints or you want to distinguish the different checkpoints with the same simulation time, you can jump to the source code where the checkpoint is set during the *Interactive Simulation Debug* mode. Select one of the checkpoints and use the **Jump to Current Checkpoint Position** command by clicking the  toolbar icon. The *Source Code* pane indicates the line of source code the checkpoint has been set to. You can also use the **Delete Check Point** command to delete checkpoints by clicking the  button.

All checkpoints exist only in this *Interactive Simulation Debug* mode session. That is, the checkpoints are removed when *Interactive Simulation Debug* mode is disabled.

11

Save/Restore Simulation Session/State

During Interactive Simulation Debug mode, the Verdi platform provides the following two types of save and restore features to save the currently ongoing debugging process or restoring the point to continue the unfinished debugging process:

- [Save/Restore Session](#)
- [Save/Restore State](#)

Save/Restore Session

Verdi allows you to save the current state of debugging process of Verdi (that is, display layout and design database) and restore the saved session file with a different design and different simv, KDB, and FSDB information in both interactive and post-processing debug mode.

The following sections describe this feature in detail:

- [Saving a Session or Layout](#)
- [Restoring the Saved Session](#)

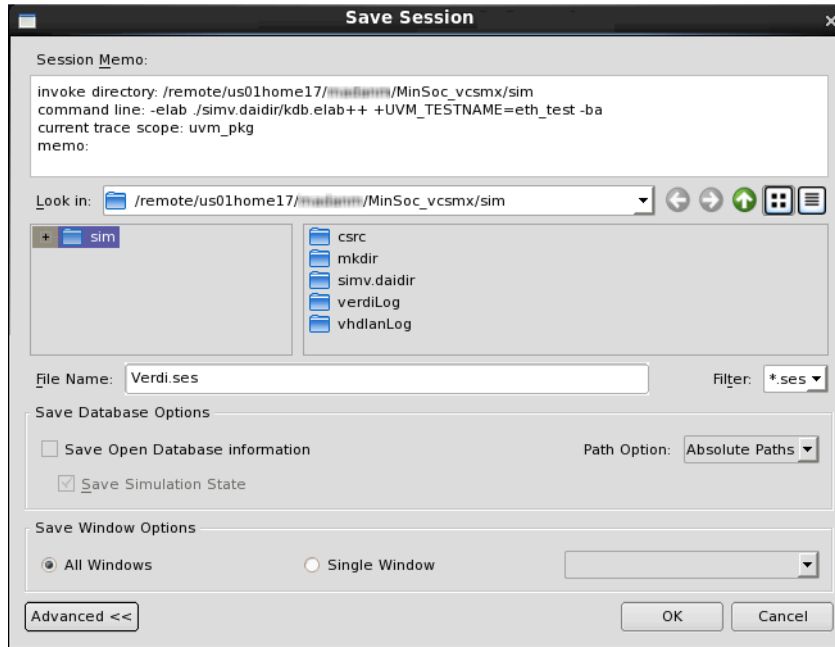
Saving a Session or Layout

You can save the current state of debugging process of Verdi, that is, display layout and design database path options, using the **Save Session** dialog box. The options that you select in the **Save Session** dialog box are restored after you restart Verdi.

To save a session,

1. Select **File -> Save Session** to open the **Save Session** dialog box, as shown in the following figure:

Figure 94 Save Session Dialog Box



In the post-processing mode, you must click the **Advanced** button to view the advanced settings.

2. To specify the file name, enter a file name in the **File Name** field or browse for the file in your directory. The default file name is `Verdi.ses`.

The session (`.ses`) resource file stores the current Verdi information to restore window layout, current scopes, current design, viewing options, and bookmarks. For more information about the session file, refer to the *Verdi and Siloti Command Reference Manual*.

3. Select the **Save Open Database Information** option if you want to save the design database (KDB) and FSDB to the session file. By default, this option is disabled in the interactive mode and saves only GUI status according to the **Save Window Options** setting. This option is enabled by default in the post-processing mode.

The **Save Simulation State** option is the sub-option of the **Save Open Database Information** option. This option allows you to save the simulation state to the session file and restore it when restoring the saved session. This option is enabled only when the **Save Open Database Information** option is enabled. You can clear the **Save Simulation State** check box to disable this option.

If the **Save Open Database Information** option is disabled or not selected, the design database information is not saved to the session file. The restore session can be restored with a different design, simv, FSDB and KDB. For more information, see [Disabling Database Information](#).

4. Select any of the following path option from the **Path Option** list box:

Absolute Paths — (Default) Specifies fully qualified path for opened design database. This allows you to reload the session file from any location, but if you move the simulation directory, the session file does not work.

Relative Paths — Specifies path for opened design databases (relative to the directory where the session file is stored) or interactive design. If you move the simulation directory, the session file still works.

5. Select any of the following window option from the options specified under the **Save Window Options** section:

All Windows — Saves the layout and contents of all windows in the current Verdi session to the session file. The restore session restores all windows to be same as the saved session. If the **Save Simulation State** option is not enabled, then the contents of the Stack pane, Local pane, Member pane, Object view, and Constraint view are not restored.

Single Window — Saves the layout and contents of only a single window in the session file and restores it in the restore session. This option is supported only for the single waveform window. All opened waveform views are listed in the list box located in this section. You can choose a waveform view to save from this list box.

6. Click **OK** to save the session.

You can place the mouse pointer over an option to view its information in a tool tip.

Disabling Database Information

If you disable the **Save Open Database Information** option, the design database information is not saved to the session file. The restore session can be restored with a different design, simv, FSDB and KDB. The following points describe this flow:

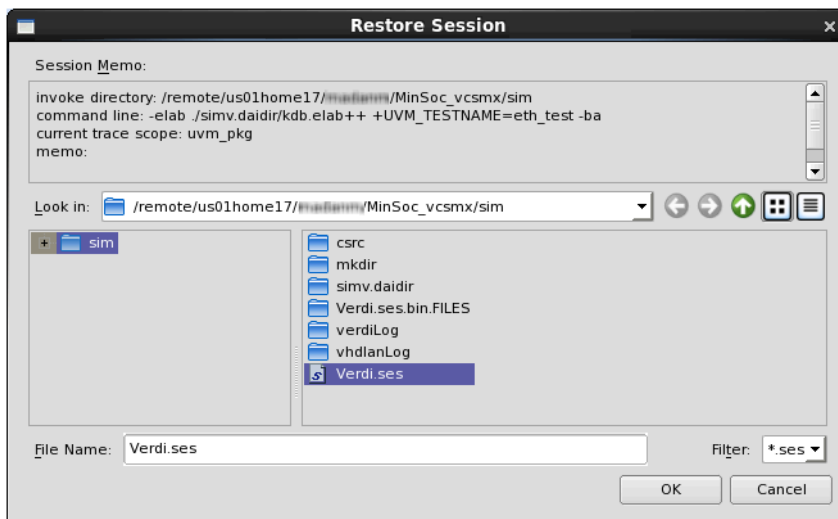
- Simulation state is not restored in this flow
- If you select the **All Windows** option, the content of all windows except the Stack pane, Local pane, Member pane, Object view, and Constraint view is restored.
- If you select the **Single Window** option, the layout and content of the selected waveform view is restored.

Restoring the Saved Session

To restore the saved session,

1. Select **File -> Restore Session** to open the **Restore Session** dialog box, as shown in the following figure:

Figure 95 Restore Session Dialog Box



2. Select a saved session and click **OK**.

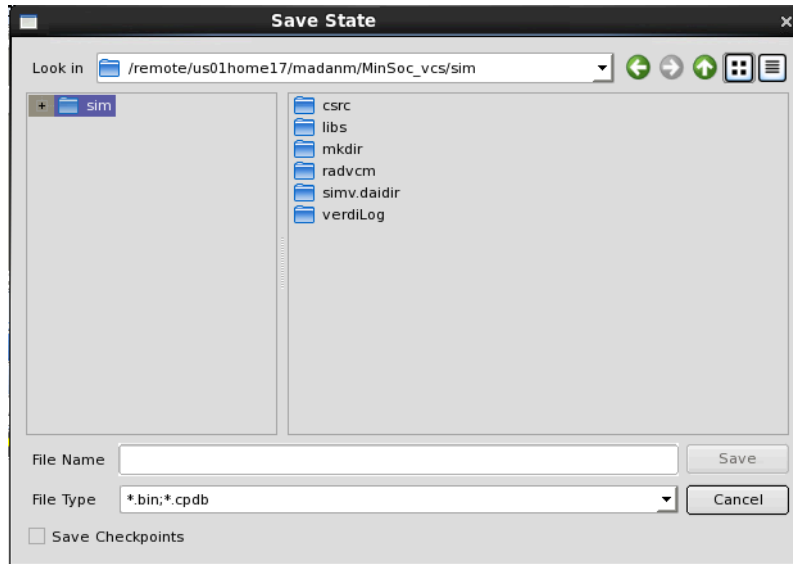
Save/Restore State

You can also use the **Simulation -> Save State** command to save the current simulation status and dumped FSDB file. After Interactive Simulation Debug mode is enabled, the **Simulation -> Restore State** command is provided for you to restore the saved simulation status without the effort to reload a huge design or reopen complex *nSchema* windows for gate-level designs. The FSDB file is also restored, and the signals that were dumped continue to be dumped to the FSDB file.

For example, perform the following steps to save the simulation state at 18ns:

1. Use the **Simulation -> Save State** command to open the *Save State* dialog box, as shown in the following figure:

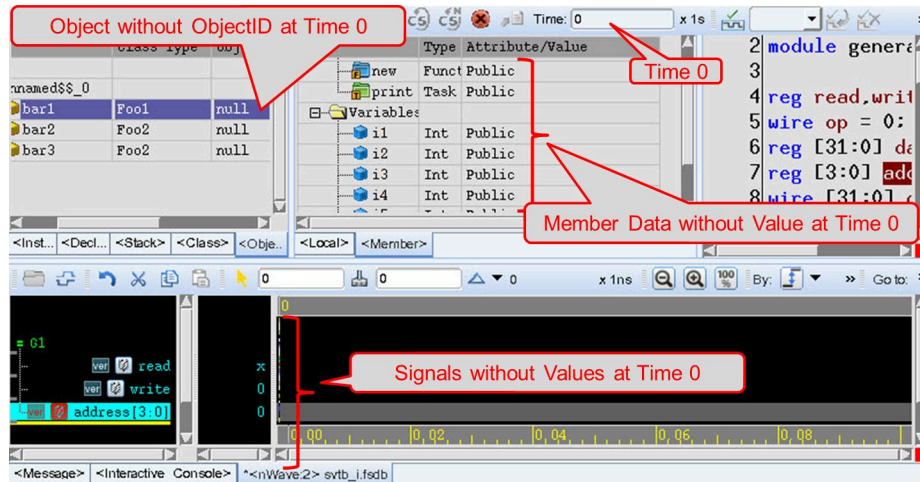
Figure 96 *Save State Dialog Box*



2. Specify the file name of the simulation state in the **File Name** field or browse for the file in your directory. For example, specify the file name as `save_18ns_ses.bin`.
3. Click the **Save** button.

When you restart the simulation or invoke the Verdi platform (and open the nWave frame) the next time, the default value of the objects, member data, and signals are all empty, as shown in the following figure:

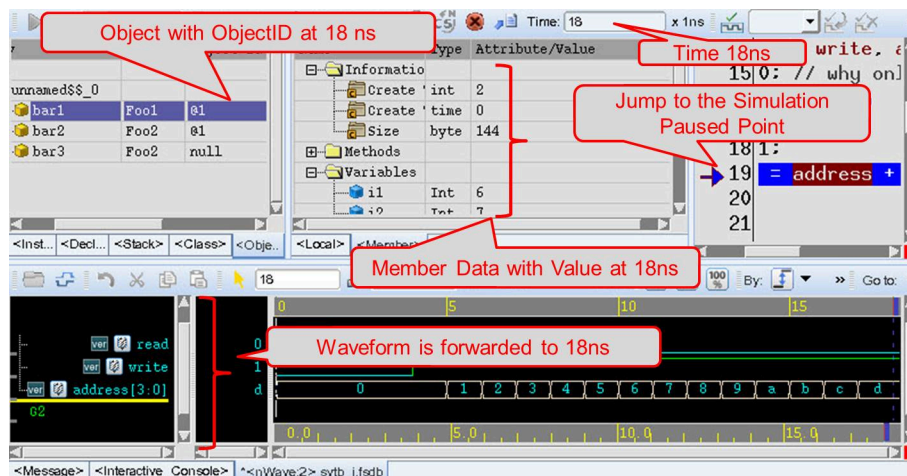
Figure 97 Enter Interactive Mode at Beginning



Use the **Simulation -> Restore State** command and select the file you saved previously (the file name is `save_18ns_ses.bin`) in the *Restore State* dialog box to restore the simulation status.

Referring to the following figure, the simulation time is restored forward to the saved time, and the variables values are represented at 18ns.

Figure 98 Restore Simulation State



12

Constraint Debug

The Verdi platform provides the Constraints Solver engine for randomization. You can analyze a randomized call after the randomization is finished and before the post-randomization is executed.

Constraints Solver is a complex engine of the simulation with VCS where good debug capabilities can greatly enhance user efficiency. Better debug capabilities can help you to understand the constraints better. In addition, it can help you to understand how a certain randomized call was solved, the sources of inefficiency, and the reason for a solver failure during the enabled Interactive Simulation Debug mode.

The Constraint Debug features are listed and described in the following sections:

- Display the constraints and random variables information in the *Local* pane. The Constraint Mode and Random Mode can be enabled and disabled in the *Local* pane.
- Display the static constraint information in the *Member* pane.
- Constraint breakpoints can be set and the simulation control command is provided to step into Constraint Debug mode.
- Show the details about how the constraints were solved in the **Solver** tab and show the relationship between the variables in the **Relation** tab.
- Re-randomization is supported.
- Interactive rewind is supported.
- Distribution Debug is supported.
- Add new constraints on-the-fly from the **Local** tab.

This chapter consists of the following topics:

- [Local View](#)
- [Breakpoint Management](#)
- [Simulation Control Commands](#)
- [Solver View](#)
- [Relation View](#)

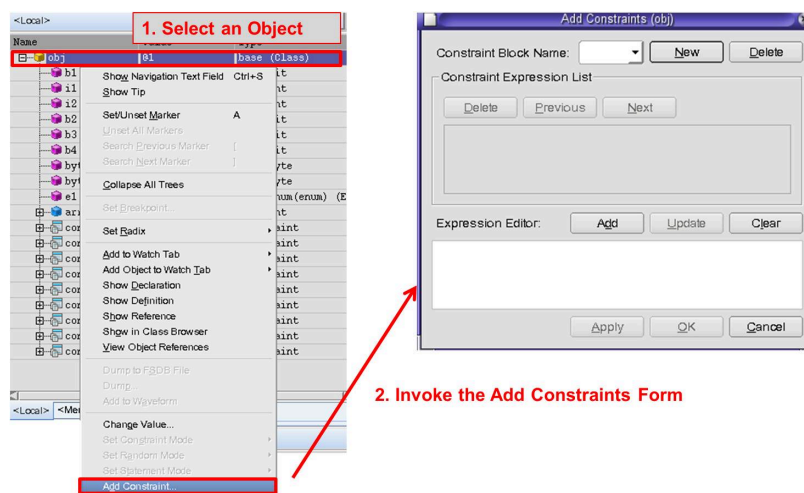
statement in a constraint block tree, the **Set Statement Mode** command is provided to enable or disable the statement.

Create a New Constraint

A new constraint can be added into an object for a runtime constraint. The following describes how to add a new constraint:

1. Select and right-click an object in the *Local* or *Member* panes and use the **Add Constraint** command to add a new constraint (as shown in the figure below).

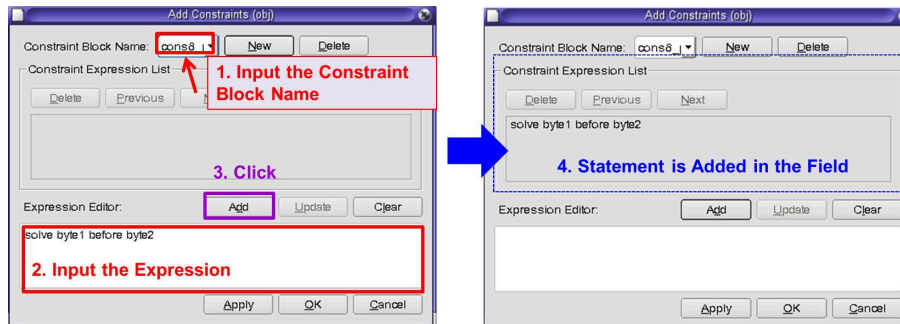
Figure 100 Invoke the Add Constraint Form



2. Type a constraint block name in the **Constraint Block Name** field for a new constraint block.
3. Edit the expression in the Expression Editor field (as shown in the figure below, type "solve byte1 before byte2" in the field). Then press the **Add** button to add the new expression to this new constraint. This new statement is shown on the **Constraint Expression List** field.

You can re-do this step to add more than one expression for a constraint block.

Figure 101 Add an Expression in New Constraint Block

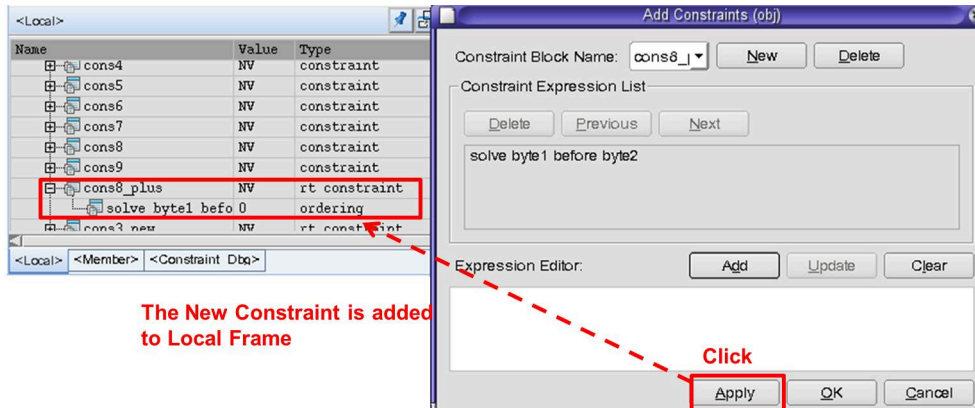


A statement listed in the **Constraint Expression List** field can be selected and edited in the **Expression Editor** field. When editing is complete, click the **Update** button to replace the original statement.

- When all settings are done, click the **Apply** button to ask the simulation to create a new constraint block.

The new constraint is created with "rt constraint" type and the *Local* and *Watch* panes are updated. As shown in the figure below, the new constraint is added with the rt constraint type in the **Local** tab.

Figure 102 New Constraint Block in Local Tab



The New Constraint is added to Local Frame

Limitations

The following types are not supported for creating constraint blocks in the *Local* Pane:

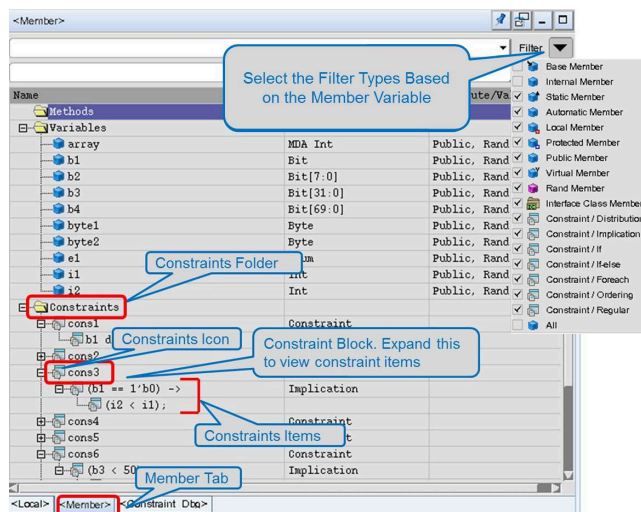
- Foreach
- If/If-else
- Nested implication

- Enumeration type variables
- Non-constant array index
- Function call in constraints
- Array reduction method call (array.size(), array.sum(), and so on)
- Typecast in constraints
- Parameterized class object
- Solver-before hard
- SV struct/union
- randc type variables
- NTB-OpenVera

Member View

The constraint blocks that are defined in the selected class or object in the Class or Object tabs are shown in the *Member* pane. You can double-click a constraint block node or a constraint item to see the source code in the *Source Code* pane. The filter types in the **Filter** menu are provided to filter base class variables, task, functions, class objects, non-class objects. The Base, Static and Automatic members can work in the Constraints folder.

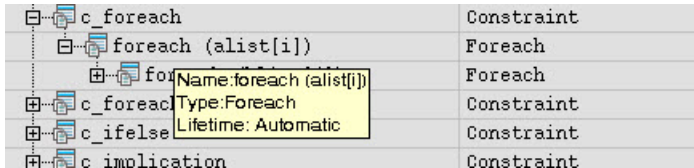
Figure 103 Constraints in Member View



If the Show Tip command has been invoked in the *Member* pane, the constraint information can be shown in a tip window. As shown in the following figure, the name,

type, and lifetime of the constraint block or statement that the cursor is placed over are shown in the tip window.

Figure 104 Show Tip for Constraints in Member View

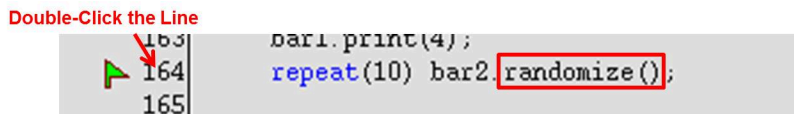


c_foreach	Constraint
foreach (alist[i])	Foreach
for Name:foreach (alist[i])	Foreach
c_foreach Type:Foreach	Constraint
c_ifelse Lifetime: Automatic	Constraint
c_implication	Constraint

Breakpoint Management

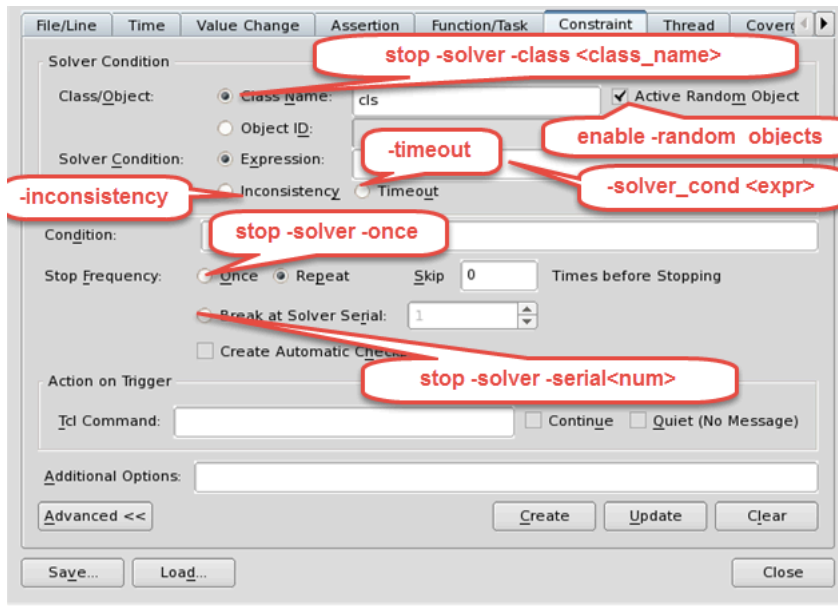
To be able to stop the simulation before any or certain randomize calls (for example, `obj.randomize();`), the breakpoint can be set by double-clicking the line where randomize is called (refer to the following figure) or in the **Constraints** page of the **Breakpoints** form (invoked with the **Debug -> Manage Breakpoints** command).

Figure 105 Set Breakpoint for randomization



As shown in the following figure, the **Break at Solver Serial** field maps to the `-serial` UCLI option, the **Executing Frequency: Once** option maps to the `-once` option, the **Class Name** field maps to the `-class` option and the **Solver Condition** field maps to the `-solver_cond` option for the `stop -solver` command.

Figure 106 Set Breakpoint for Randomization in Breakpoints Form



- You can use the **Active Random Object** option in Verdi to allow the randomization to stop when the `randomize()` method is called from the object derived from the specified class or from the object that contains the embedded objects of the specified class type.

Use the `-random_objects` option along with the `-class` option in the `stop -solver` UCLI command as shown in the following command:

```
stop -solver [-random_objects] [-class <class_name>]
```

The `-random_objects` option extends the `-class` functionality to match class names in the set of objects that are to be randomized as a whole. The matching of class names starts with the object that invoked the `randomize()` method. These class objects are referred as the active random objects.

- In the **Expression** field, type the expression for the `-solver_cond` option. If the expression specified by `-solver_cond` option is evaluated as true, then the breakpoint is hit when any random variable of the specified class is solved.
- The **Inconsistency** option button represents the `-inconsistency` option. If you select the **Inconsistency** option button, the breakpoint is hit when inconsistent constraints are detected during randomization.

Simulation Control Commands

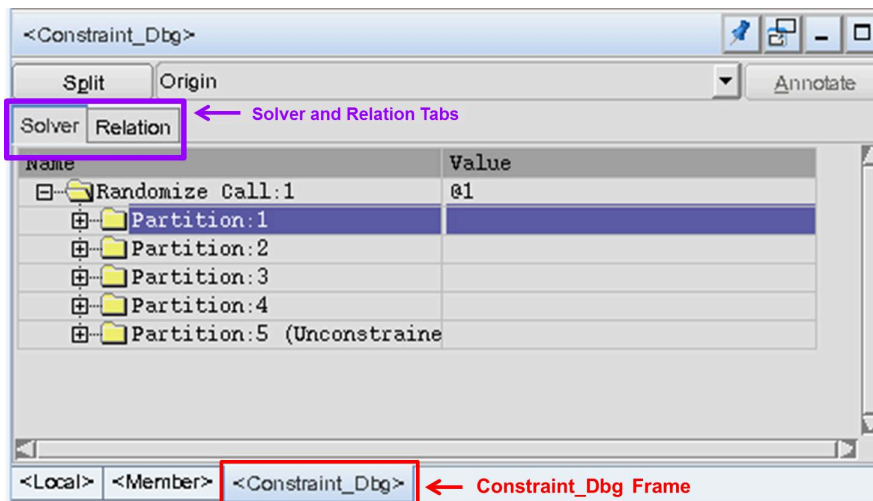
The following simulation control commands are provided to control the simulation progress for constraint debugging. The commands can be accessed through the simulation control toolbar or the Simulation menu.

Step in Constraint Solver

When the simulation stops before a randomize call (as described in the Breakpoint Management section), use the  icon or select the **Simulation -> Step in Constraint Solver** command from the menu to step in the Constraint Solver. If the current line does not have a randomize call, the result of using the **Step in Constraint Solver** command is same as using the **Next** command.

After stepping into the Constraint Solver (Constraint Debug mode is enabled), the details for the solved results are shown in the *Constraint_Dbg* pane, including the Solver and Relation tabs, that is docked in the same location as the Member pane as a new tab (as shown in the following figure).

Figure 107 Constraint Debug pane

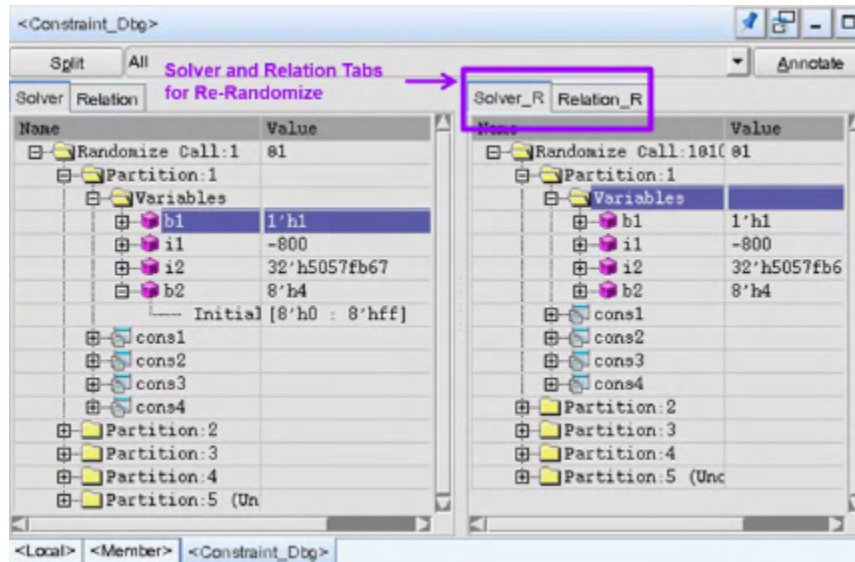


Redo Randomize Call and Step in Constraint Solver

When the simulation is already in Constraint Solver, the Re-randomize command may be invoked by clicking the  icon or using the **Simulation -> Redo Randomize Call and Step in Constraint Solver** command to re-randomize and step in Constraint Solver. After the re-randomization is done, the simulation enters Constraint Debug mode and stop before the post-randomize call. Meanwhile, another set of Solver and Relation

tabs (with the "Solver_R" and "Relation_R" tab names) are shown for the re-randomize solved results (refer to the following figure). Refer to the Re-randomize section for the re-randomization in details.

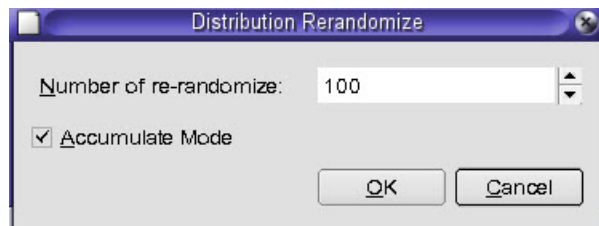
Figure 108 Redo Randomize



Distribution Re-randomize and Step in Constraint Solver

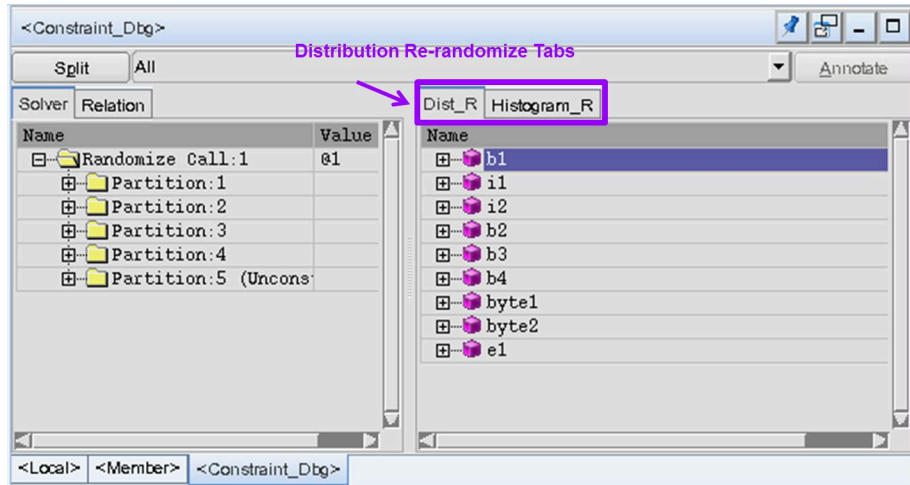
The Distribution Debug button will be enabled when the simulation is already in Constraint Debug mode, same as the normal re-randomize. To perform multiple re-randomizations, click the  icon or select the **Simulation -> Do Distribution re-randomize and Step in Constraint Solver** command and the Distribution Rerandomize form is displayed as shown in the following figure. The number of re-randomizations can be specified in the blank field and the default value is "100" and the valid number must be between "2" to "99999". The Accumulate Mode option is provided to accumulate the results of the previous n distribution randomizations for the current randomize call. This option is enabled by default. Click the **Ok** button to perform the distribution re-randomization.

Figure 109 Specify Number of Distribution Re-randomize



The randomization is performed the specified number of times, and then the simulation is stopped in the Constraint Solver. The *Constraint_Dbg* pane displays the *Dist_R* and *Histogram_R* tabs, as shown in the following figure. Refer to the Distribution Re-randomized Analysis section for details.

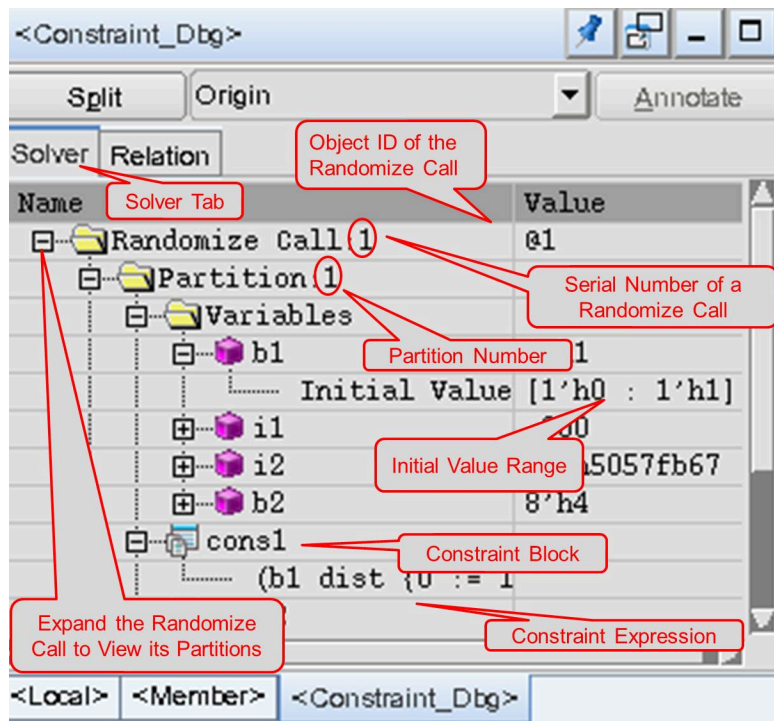
Figure 110 Distribution Re-randomize



Solver View

As described in the Simulation Control Commands section, the Solver tab can be invoked in the *Constraint_Dbg* pane to display the randomize calls. The following figure shows the current Constraint Solver contents with a hierarchical view. A Randomize Call is the root node and it can be expanded to view the partitions of the randomize call.

Figure 111 Solver View in Constraint_Dbg pane



The following data is displayed on the **Solver** tab for the executed randomize call (refer to the following figure):

- The serial number of the randomize call.
- The partition numbers.
- The function call.
- The name and mode (On/Off) of a constraint block.
- Constraint expression.
- The name, mode (On/Off if type is Rand/RandC), value, and initial value range of a variable.
- The name and value of argument of function call.
- If there are any inconsistencies, the name, type, and mode of variables and constraint expressions will be displayed.

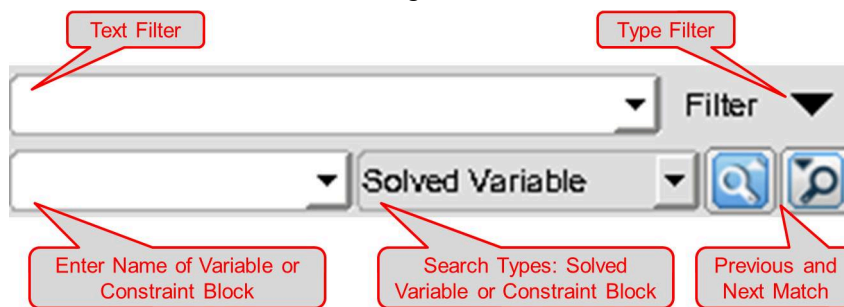
Trace Solver Data

When you want to check the source code of a random variable or constraint block/statement declaration, you can double-click the row in the Solver tab to jump the debug cursor to the line in the *Source Code* pane and highlight it.

Use the following commands in the right-click menu for further tracing of the selected item in the Solver tab.

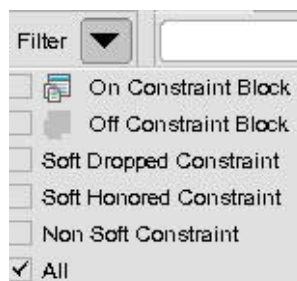
- **Show Navigation Text Field:** Use the Navigation Text field to filter or search a specified string for the variables and constraint blocks in the Solver tab.

Figure 112 Search and Filter Navigator



Input a filtering string to the Text Filter field and press the Enter key to filter the variables, constraint blocks, and constraint statements. You can use the Type Filter menu (as shown in the following figure) to select the filtering type. If a matching name for the specified string does not exist, the contents of the Partition and Variables directories are empty.

Figure 113 Type Filter Menu



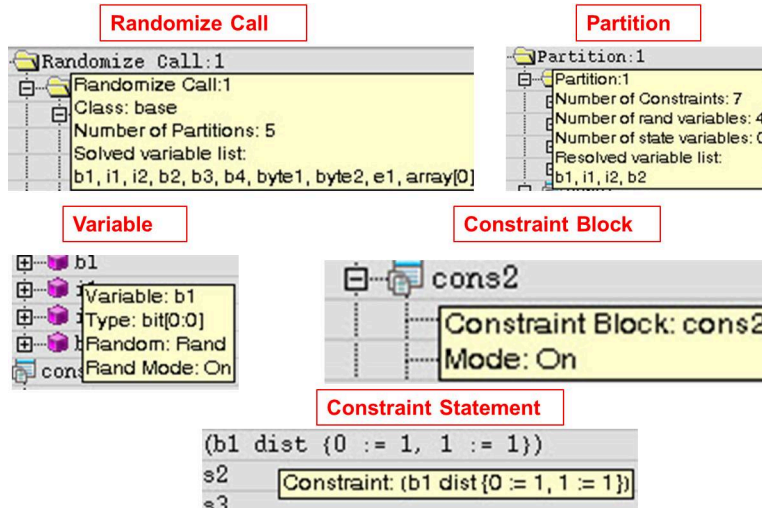
To search the original or filtered Solver data, input a string to the

Solved Variable

field and then click the or icons to search backward or forward. You can search for the solved variables or constraint blocks by selecting the type in the Search Type menu. The tree is expanded to the matched item and the matched item is highlighted.

- **Show Tip:** Move the cursor over items in the Solver tab and hold for a couple of seconds to display a tip window. Different items shows different types of information in the tip. If the cursor is in the Name column, the tip window shows detailed information about the current item (as shown in the following figure). If the cursor is in the Value column, then the tip window shows the contents inside the column for the current item.

Figure 114 Tip Windows with Different Types

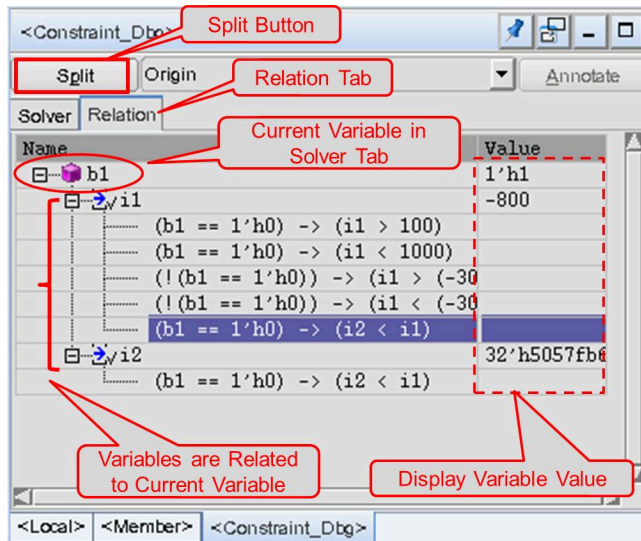


- **Display Source Code in New Tab:** Opens the corresponding file and highlights the line in the Source Code pane. This command works for randomize calls, function calls, constraint blocks, constraint expressions, and variables.
- **Show in Class Browser:** Locates the definition of the selected item in the Class pane. The class and the selected item is highlighted in the Class and Member panes respectively. This command works for constraint blocks and variables.
- **Show in Local View:** Locates the selected item in the Local tab. If the selected item is a variable, use this command to display the selected variable and related variables in the Relation tab. This command works for the root item.
- **Show Relation:** Displays the selected variable and related variables in the Relation tab. This command works for variables.
- **Set Radix:** For variables, change radix is applied on the solved value and also the initial value. For constraint expression, the change radix is applied on all qualified values in the expression. This command works for variables and constraint expressions.
- **Extract Testcases:** From a randomize call, testcases are extracted from all partitions inside the current randomize. From a partition, one testcase is extracted from the current partition. The directory and the filename can be specified. This command works for randomize calls or partitions.

Relation View

As described in the Trace Solver Data section, you can use the Show Relation command to display a Relation tab and the corresponding variable and other variables that are related to this variable is added to this tab (as shown in the following figure).

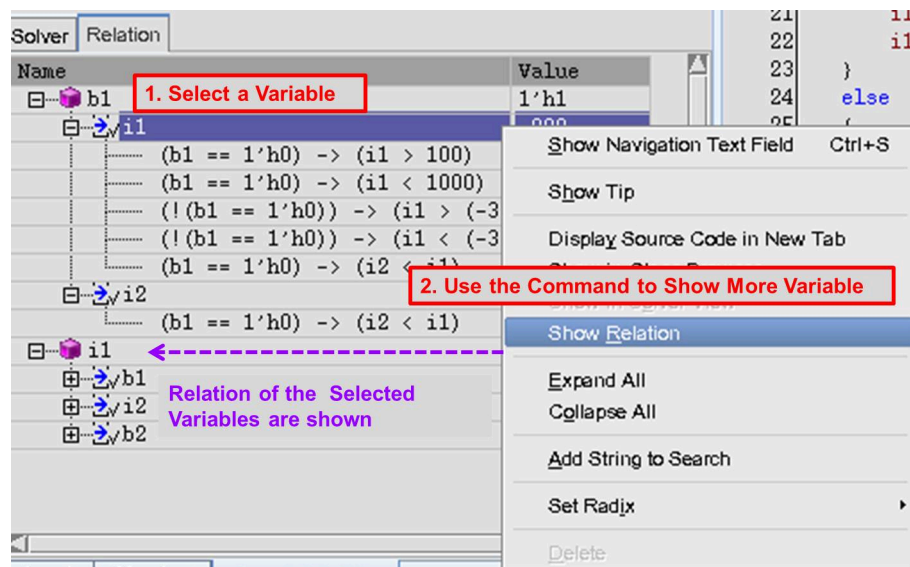
Figure 115 Relation View in Constraint_Dbg pane



More variables can be appended into this tab using following steps (as shown in the following figure):

1. Select a variable from the Relation or Solver tabs.
2. Use the **Show Relation** command in the right-click menu.

Figure 116 Append More Variables in Relation Tab

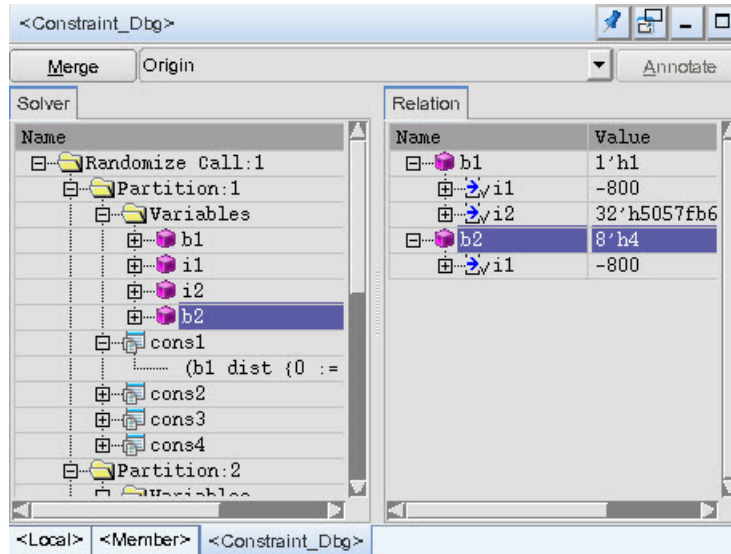


When you want to check the source code of a random variable or constraint statement declaration, you can double-click the row in the **Relation** tab to jump the debug cursor to the line in the *Source Code* pane and highlight it.

The combined tabs in the *Constraint_Dbg* pane for the Solver View and Relation View can be split using the **Split** button in the upper left. As shown in the following figure, the

Solver tab is on the left side and the **Relation** tab is on the right side. You can use the **Merge** button to combine the tabs to the original locations.

Figure 117 Split Solver View and Relation View



Use the following commands in right-click menu for further tracing of the selected variable in the **Relation** tab.

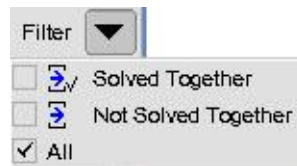
- **Show Navigation Text Field:** Use the *Navigation Text* field to filter the specified string for the second level of variables in the **Relation** tab.

Figure 118 Filter Navigator



Enter a string in the *Text Filter* field (as shown in the figure above) and press the Enter key and then use the Type Filter menu (as shown in the following figure) to select the filtering type.

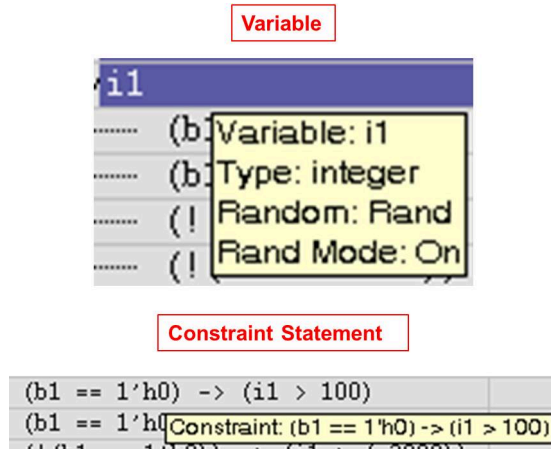
Figure 119 Type Filter Menu



- **Show Tip:** Move the cursor over items in the **Relation** tab and hold for a couple of seconds to display a tip window. If the cursor is in the **Name** column of a variable, the tip window shows the name, type, random type, and random mode of the variable

(as shown in the following figure). If the cursor is in the Name column of a statement, the tip window shows the expression. If the cursor is in the Value column, then the tip window shows the contents inside the column for the current item.

Figure 120 Tip Windows with Different Types

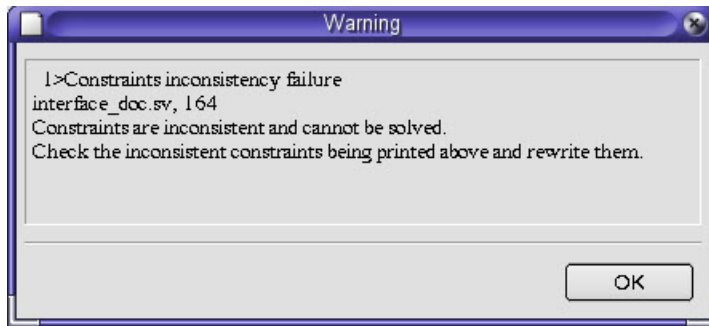


- **Display Source Code in New Tab:** Opens the corresponding file and highlights the line in the *Source Code* pane. This command works for constraint expressions and variables.
- **Show in Class Browser:** Locates the definition of the selected item in the *Class* pane. The class and the selected item is highlighted respectively in the *Class* and *Member* panes. This command works for variables.
- **Show in Solver View:** The selected expression is highlighted in the **Solver** tab. This command works for constraint expressions.
- **Show Relation:** Displays the selected variable and related variables in the **Relation** tab. This command works for variables that are not on the top level.
- **Delete:** Deletes the selected variable and related variables in the **Relation** tab. This command works for root variables.
- **Set Radix:** Refer to the [Trace Solver Data](#) section for the same command and effect.

Inconsistent Constraints

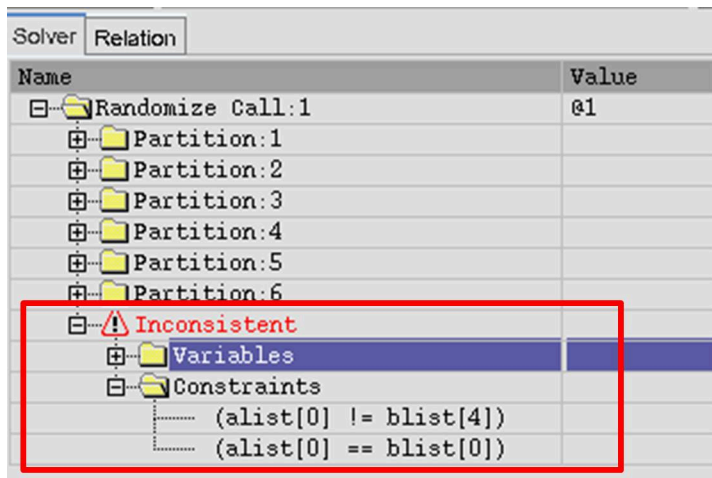
When the **Step in Constraint Solver** command is used and Constraint Solver finds inconsistent constraints, an Error form is displayed to show the inconsistency (refer to the following figure).

Figure 121 Inconsistency Error



Click the **OK** button to go to the Solver tab where the minimal inconsistent constraints are added (as shown in the following figure).

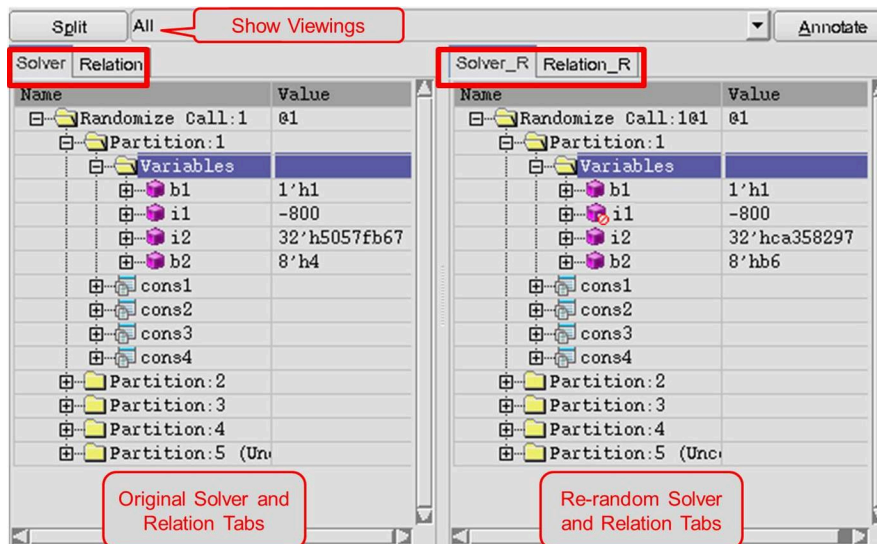
Figure 122 Inconsistency Variables and Constraints



Re-randomize

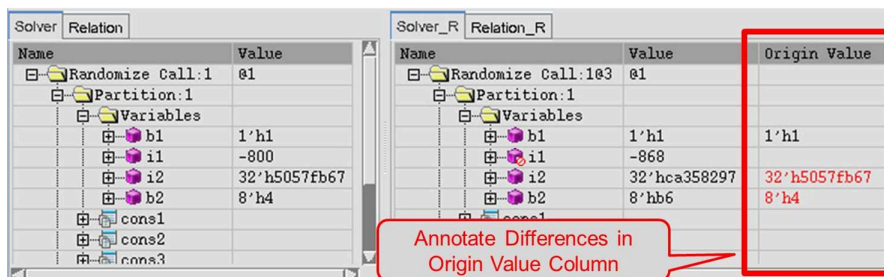
As described in the [Simulation Control Commands](#) section, re-randomization can be done using the **Redo Randomize Call and Step in Constraint Solver** command. Additional Solver and Relation tabs are shown with the Solver_R and Relation_R names (refer to the following figure).

Figure 123 Redo Randomize



You can use the **Annotate** button in the upper right to annotate the solved value from the original randomize to the re-randomize tree. A new column is shown up and the difference is highlighted in red (refer to the following figure).

Figure 124 Annotate Different Solved Value



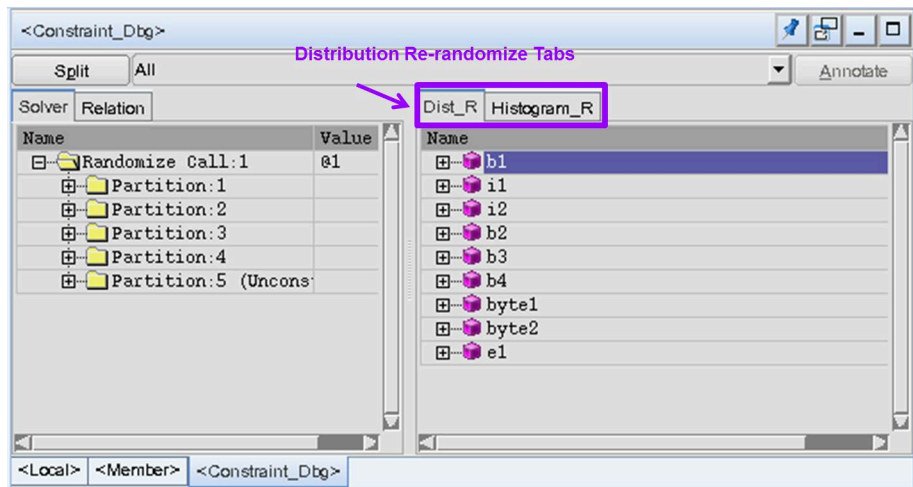
The viewing types of the Constraint_Dbg pane can be selected by the **All** menu. The viewing types are Original, Rerand and All types for showing only the original Solver/Relation tabs, only the Solver_R/Relation_R tabs and both the Solver/Relation and Solver_R/Relation_R tabs.

Distribution Re-randomized Analysis

The Verdi platform allows you to perform re-randomization multiple times after stepping into the Constraint Solver and checking the distribution of the random variable values in a histogram. This feature helps you to understand the constraint behavior and find potential debug issues.

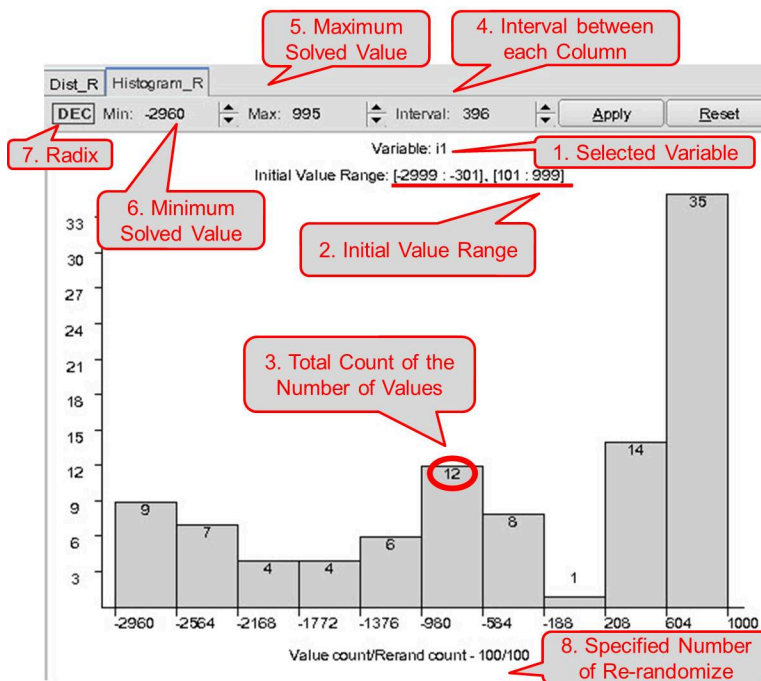
As described in the [Simulation Control Commands](#) section, distribution re-randomization can be done using the **Do Distribution re-randomize and Step in Constraint Solver** command. You can check the distribution of random variable value with the new tabs. After specifying the number of re-randomizations, another set of Dist_R and Histogram_R tabs are shown (as shown in the following figure). All the solved variables in the specified 'N' times randomize-calls are displayed in the Dist_R tab. If the variable is used in any constraints, then the variable becomes expandable. You can expand the variable to check the related constraints.

Figure 125 Distribution Re-randomize



To check the distribution of the selected variable, select a variable and use the **Show Histogram** command in the right-click menu. As shown in the following figure, the distribution of values for the variable is drawn as a histogram in the **Histogram_R** tab.

Figure 126 Distribution Histogram



Refer to the following figure and the following explanation to analyze the histogram.

1. The variable name is displayed on the top, and its initial value is marked below the variable name.
2. The range of the initial value is the union of the initial value ranges from N times (the specified number) randomizations.
3. The number of values during each interval is marked at the top of the bar, if the number is zero, it is highlighted in red.
4. By default, there are 10 columns or less if the interval is "1" already.
5. The Max value is equal to:

$$\text{min_value} + \text{interval} * \text{number_of_columns}$$
6. You can specify the min/max and interval in the respective fields on the top. Click the Apply button, the histogram is re-plotted with the new attributes.
7. The current radix is displayed.

You can use the Reset button to re-plot the default histogram.

Supported Platforms and Simulator

The supported platforms are SOL2, SOL7, LINUX, LINUX64 SUSE32 and SUSE64 and the supported simulator is VCS.

The new features and information described in this document are available with VCS 2013.06 or later versions. Refer to the following table for further features with the supported VCS version.

Supported VCS Versions:

Category	Feature	VCS version
Object Dumping	Dumping object data to FSDB	2013.06-SP1 or later
Rewind	Manually record and rewind simulation state	2013.06 SP1-1 or later 2014.03 or later
	Automatically create checkpoints	2014.12 or later

13

Reverse Debug

The following sections describe the Verdi *Reverse Debug* feature in Interactive Simulation Debug Mode:

- [Introduction](#)
- [Use Model](#)
- [Reverse Debug Commands](#)
- [Checkpoint Distribution](#)
- [Checkpoint Memory Profiling](#)
- [Reverse Debug Support for VHDL](#)
- [Limitations](#)

Introduction

The reverse debug features of the Interactive Simulation Debug solution in Verdi includes capability that supports interactive debugging with running the simulation backwards.

You can start debugging at the symptom of the problem and systematically go back in time along the bug propagation cause-effect chain. Divide-and-conquer debugging method is much more efficient with reverse debugging. For example, if the simulation is stopped before some function call and you are not sure whether the function returns the correct value or not, you can step over this function call and check the returned result. If the result is wrong, you can perform the **Reverse Next** command, step into the function and investigate what causes the wrong result. Without reverse debugging, this would require very costly restart of debugging and playing with breakpoints to reach the same simulation state.

Following are the VCS simulation control commands for reverse executing simulation in Verdi:

- Run/Continue Reverse
- Next Reverse
- Step Reverse

- Step Out Reverse
- Step in Thread Reverse
- Step in Testbench Reverse
- Next in Thread Reverse

You can also easily reverse/advance the simulation to the previous/next value assignment of a signal or variable by invoking a command for the selected one. For a DUT design, this is often achieved by dumping all signals and using the traced information to figure out where to go. For a testbench, this approach does not work. Also, it should work without all signals traced. You can use this new feature to go to the desired assignment of the object.

In addition, Verdi provides the **Undo Last Simulation Control** command to quickly undo the recent simulation execution command(s), such as **Run**, **Next**, and **Step**, etc. The **Redo Last Simulation Control** command is also provided to redo the corresponding command.

The following sections provide a detailed description of these features:

- [Prerequisites](#)
- [Use Model](#)
- [Limitations](#)

Prerequisites

To use these Reverse Debug features in the Interactive Simulation Debug mode, VCS is a prerequisite. You must use the **-debug_access+reverse** VCS elaboration option before the Interactive Simulation Debug mode is enabled.

Use Model

The following sections describe how to enable Reverse Debug features in detail:

- [Before Invoking Verdi Interactive Simulation Debug Mode](#)
- [Enabling Verdi Reverse Interactive Simulation Debug Mode](#)
- [Using Reverse Simulation Control Commands](#)
- [Going to Previous/Next Value Assignment](#)
- [Using Undo and Redo Last Simulation Control Command](#)
- [Going to Next or Previous Message in Interactive Console](#)

- [Going to Next or Previous Function/Task Invocation](#)
- [Going Back When an Object is Created](#)
- [Active Point Behavior](#)
- [Discarding Future Simulator Status](#)
- [Discarding Future Automatically](#)
- [Saving and Restoring Session](#)
- [Specifying Compression Level of a Save Session](#)
- [Adding Virtual Checkpoints to a Session](#)
- [Support for Debug Points](#)

Before Invoking Verdi Interactive Simulation Debug Mode

You must generate the `simv` simulator executable with the **-debug_access+reverse** debug option on the VCS command line. For example:

```
% vcs -sverilog example.sv -debug_access+reverse -kdb=common_elab  
<compilation/elaboration options>
```

Note:

- Refer to the VCS User Guide for details about the **-debug_access+reverse** simulation option.
- If you are using interactive mode but only set `-debug_access` or `-debug_access+all`, (both are without `+reverse`), Verdi Advisor would be turned-on and flash for about 1 minute. You can see the suggestion by clicking the Advisor icon.

Enabling Verdi Reverse Interactive Simulation Debug Mode

You can invoke Verdi Interactive Simulation Debug mode by using the following command:

```
% simv -verdi
```

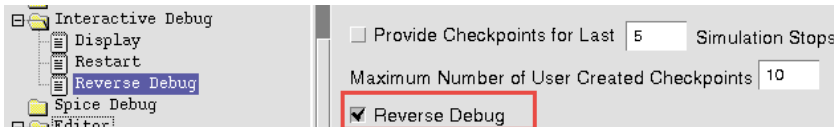
Note:

Refer to the Verdi manual for details about how to enable Verdi Interactive Simulation Debug mode.

Complete the following steps to enable Reverse Debug features to use them in Interactive Simulation Debug mode:

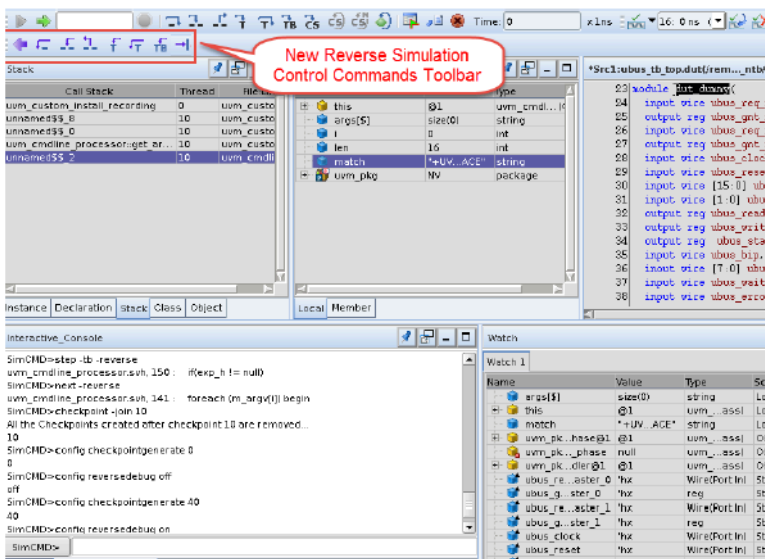
1. Select the **Interactive Debug -> Reverse Debug** page from the *Preferences* form opened by the **Tools -> Preferences** command.
2. Enable the **Reverse Debug** option.

Figure 127 Enable Reverse Debug



3. Click **Apply** or **OK** in the *Preferences* form. As shown in the figure, the new  toolbar for the reverse simulation control commands appears in the Verdi main window.

Figure 128 Reverse Simulation Control Commands Toolbar



Using Reverse Simulation Control Commands

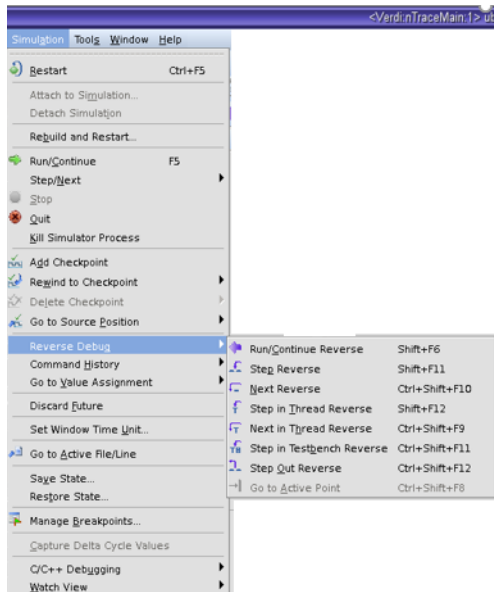
The reverse simulation control commands provide the ability to move to an earlier simulation state from the current interactive simulation debugging. All commands bring the simulation back in time to the completely functional execution state with full visibility. The information of Interactive Simulation Debug frames (for example, the *Local* and *Object* tabs, and so on) is reversed and updated accordingly.

As shown in the following figures, you can use the commands by clicking the command icons or selecting them from the **Simulation** command menu to direct the simulator how to reverse execute the program.

Figure 129 Simulation Control Commands in Toolbar



Figure 130 Simulation Control Commands in Simulation Command Menu

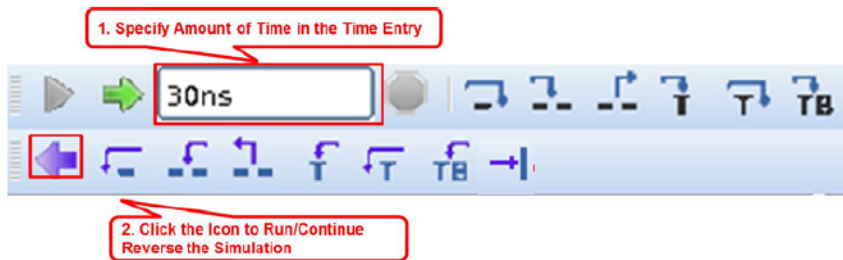


For example, you can reverse the simulation to the previous executed statement using the **Step Reverse** option (by clicking  or selecting the **Simulation -> Step Reverse** option). The *Local* and *Watch* tabs are refreshed accordingly.

Run/Continue Reverse Simulation Control Command

You can specify the amount of time and use the **Run/Continue Reverse** command (by clicking the  icon or selecting the **Simulation -> Run/Continue Reverse** command) to go back in time for the specified amount of time, as the same time entry field used by the **Run/Continue** command (see figure). All the current breakpoints are respected and the simulation stops at the most recent (considered back from the current execution state) breakpoint hit.

Figure 131 Run/Continue Reverse the Simulation



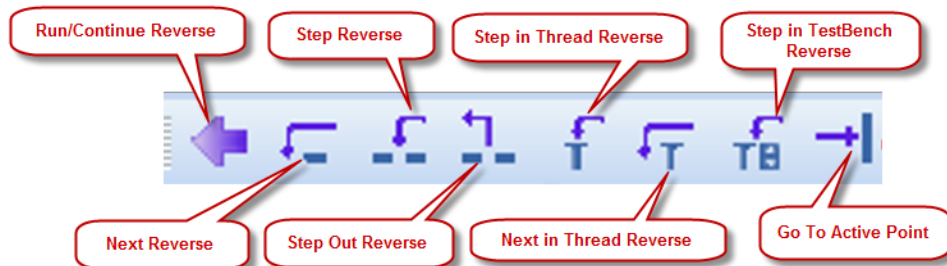
When the time entry field is empty and no breakpoints are hit, the simulation is rewind back to the start. If the reverse debug feature was enabled some time after the simulation started, using the **Run/Continue Reverse** command (as well as other reverse commands) stops at this point instead. In this case, a confirmation dialog box is displayed with the “Rewind to first checkpoint at time <time>?” message.

The runtime of executing the **Run/Continue Reverse** command is dependent on how far back in time it would go. If executing the **Run/Continue Reverse** command takes long time, you can interrupt it by clicking the  **Stop** icon in the toolbar.

Step and Next Reverse Simulation Control Commands

The following reverse commands are available to reverse the simulation by clicking the command icons or selecting them from the **Simulation** command menu:

Figure 132 Simulation Commands and Icons

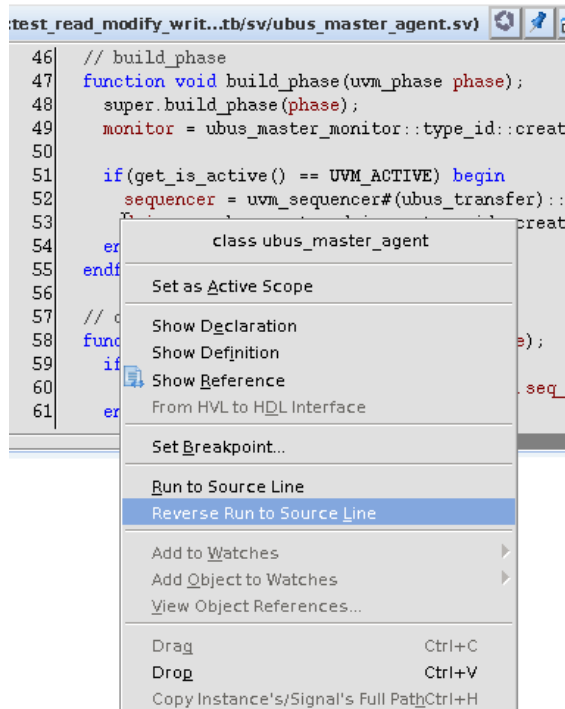


- **Next Reverse:** Goes back one SystemVerilog line which steps over task/function calls. Eventually, it might stop on a breakpoint inside the task/function called at the previous line.
- **Step Reverse:** Goes back one SystemVerilog source code line.
- **Step Out Reverse:** Goes back to the source code line where the current function has been called.
- **Next in Thread Reverse:** Goes back one source code line in the current thread which steps over task/function calls.

- **Step in Thread Reverse:** Goes back one source code line in the current thread.
- **Step in Testbench Reverse:** Goes back one source code line in the testbench code.
- **Go to Active Point:** Quickly goes to the active point.

In the *Source Code* frame, you can use the **Reverse Run to Source Line** right-click option to reverse the simulation to the selected source line.

Figure 133 Reverse Run to Source Line Option



New UCLI Commands

The **-reverse** option is added to the **step**, **next**, and **run** UCLI commands to execute the corresponding reverse execution commands that are available only in Verdi Interactive Simulation Debug mode.

The following table lists the corresponding Verdi simulation control commands.

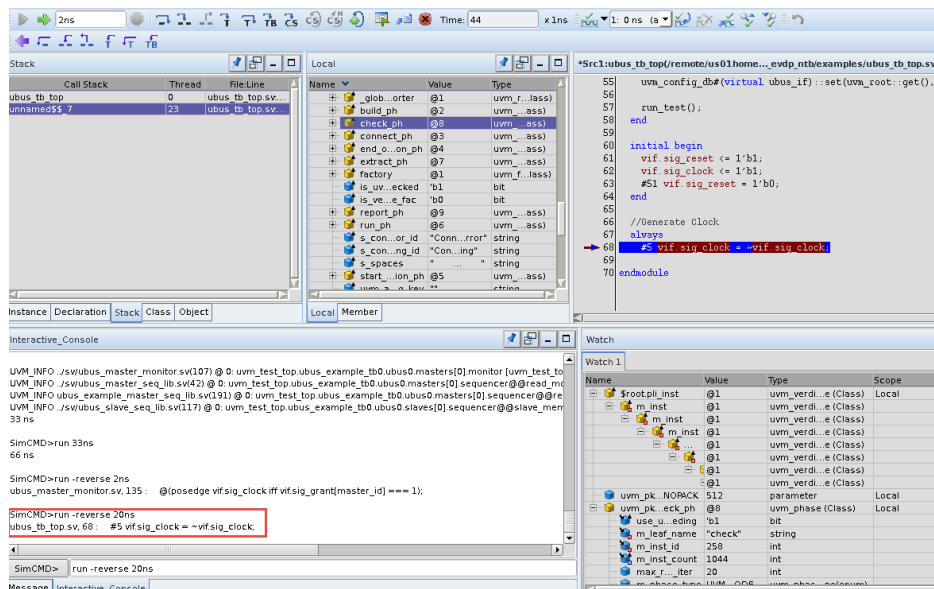
New UCLI Command	Corresponding Simulation Control Command in Verdi
step -reverse	 Step Reverse
step -reverse -thread	 Step in Thread Reverse

step -reverse -tb	 Step in Testbench Reverse
next -reverse	 Next Reverse
next -reverse -thread	 Next in Thread Reverse
next -reverse -end	 Step Out Reverse
run -reverse	 Run/Continue Reverse
run -reverse [time [unit]]	Run/Continue Reverse with the specified time
run -reverse -absolute relative <time>	Run/Continue Reverse with the specified absolute or related time
run -reverse -line <line#> [-file <file>] [-instance <nid>] [-thread <tid>]	Run/Continue Reverse to the specified line in the specified source code file

For example, execute the following command in the Verdi Interactive Console frame:

```
SimCmd> run -reverse 20ns
```

The simulation goes back 20ns and the interactive debugging frames are changed accordingly.

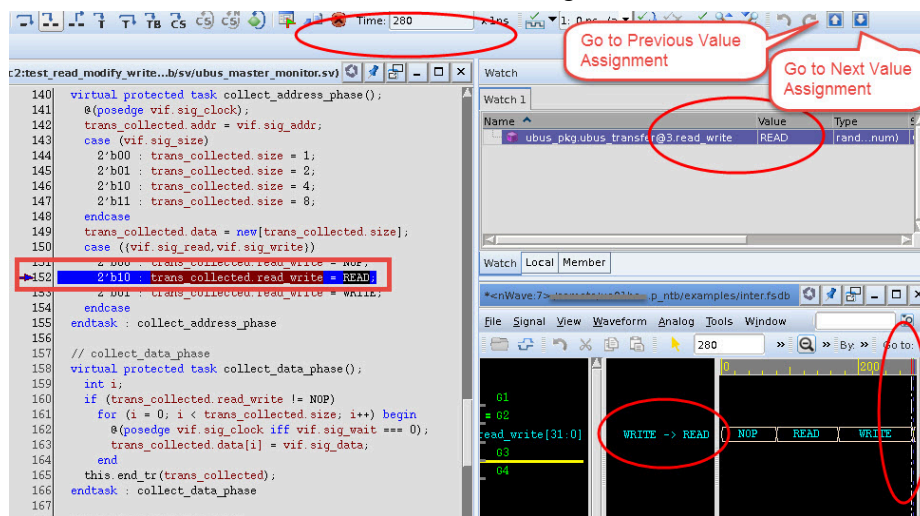


Going to Previous/Next Value Assignment

When you want to find out the previous assignment for a signal or variable, you can select the object in the *Watch*, *Local*, or *Source Code* frames and invoke the **Go to Previous Value Assignment** right-click option or click the  icon to reverse the simulation to the assignment point.

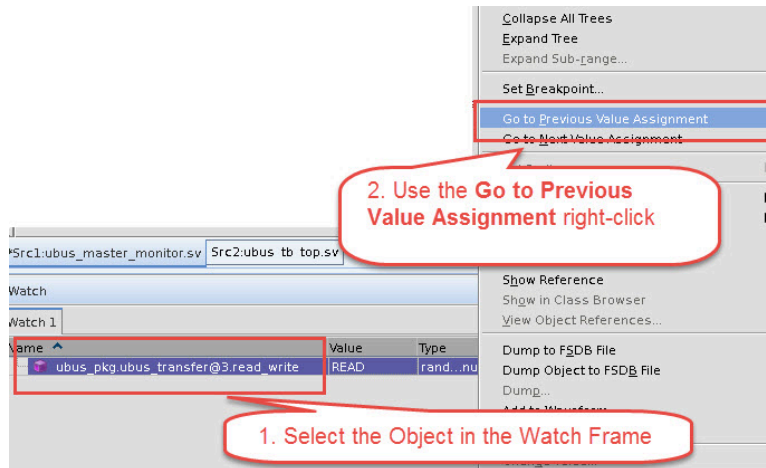
For example, referring to the following figure, at 280 ns simulation time, the value of the **read_write** variable is changed to **READ** from **Write**. You can also see the cursor pointing to the line 152 of the `ubus_master_monitor.sv` file in the source code frame which is the current debug position.

Figure 134 Before "Go to Previous Value Assignment"



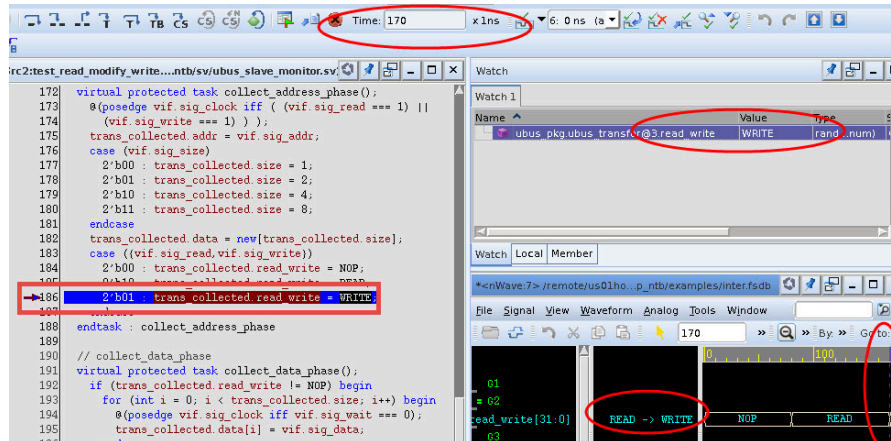
Select the **read_write** variable in the *Watch* frame, then invoke the **Go to Previous Value Assignment** right-click command (refer to the following figure for the use model) or click the  icon.

Figure 135 Invoke "Go to Previous Value Assignment"



Refer to the figure and you can see the simulation time is reversed to 170 ns where the previous assignment of the **read_write** variable happened. The value of the **read_write** variable is changed to Write. You can also see the debug cursor pointing to line 186 of the **ubus_slave_monitor.sv** file in the source code frame which is the current debug position.

Figure 136 After "Go to Previous Value Assignment"

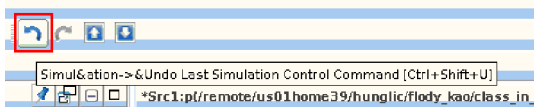


You can also use the **Go to Next Value Assignment** right-click command or click the  command icon to find out the next assignment for the selected signal or variable which advances the simulation to the assignment point.

Using Undo and Redo Last Simulation Control Command

By using the **Undo Last Simulation Control** command, you can quickly undo effect of the most recent simulation execution command(s) (for example, **Run**, **Next**, or **Step**, and so on). For example, after you executed the **Run** command and the simulation stopped at a certain breakpoint. If you want to go back to the simulation state as it was before the **Run** command, use the **Undo Last Simulation Control** command as shown in the following figure:

Figure 137 Undo Last Simulation Control Command



The **Undo Last Simulation Control** command is not fully supported in following situations:

- When command execution is in VHDL code
- When simulation is stopped inside certain breakpoints (for example, constraint solver or assertion breakpoint). In this case, the simulation goes to the most recently executed source code line before the desired location. The **Undo Last Simulation Control** command cannot go back to the desired execution location exactly.

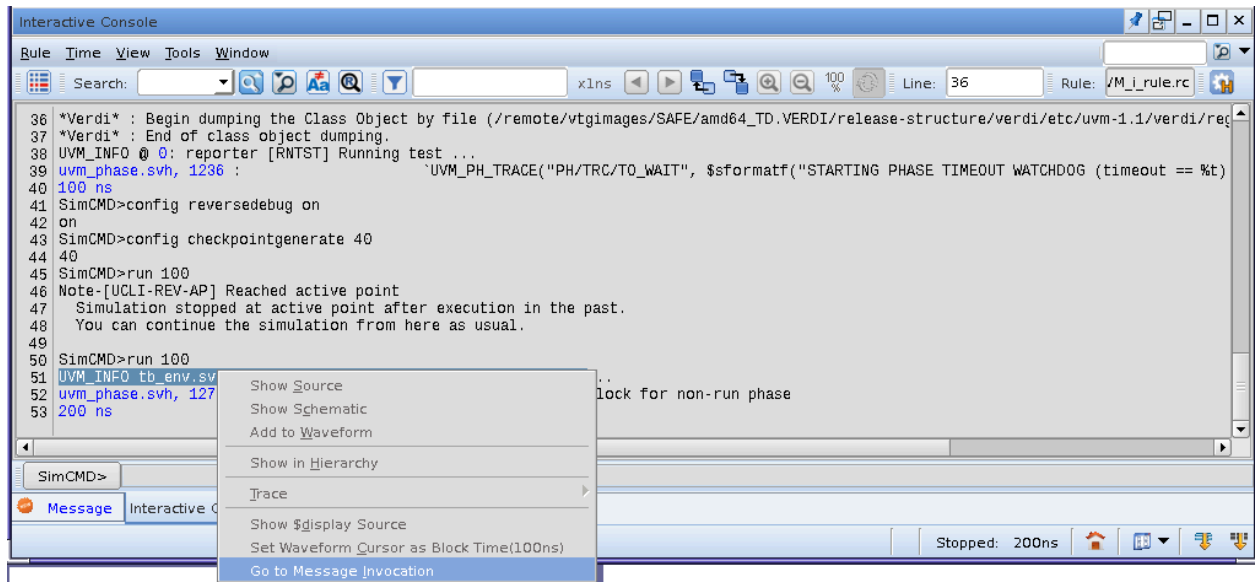
To ensure the **Undo Last Simulation Control** command work in the above situations, enable automatic checkpoint creation at each simulation stop with the following step:

1. Select the **Interactive Debug -> Reverse Debug** page from the *Preferences* form opened by the **Tools -> Preferences** command.

Going to Next or Previous Message in Interactive Console

Verdi allows you to go back/forward when a message is printed in the *Interactive Console* using the **Go to Message Invocation** right-click menu command, as shown in the following figure. This command locates the simulation to the position where the selected message is printed.

Figure 138 Enabling the “Go To Message Invocation” Command



Note:

If there are multiple messages in the same file/line (for example, inside for/while loop), you can only go to the first message invocation.

This command is enabled only when you select a portion of the message that contains file/line and time information.

This option supports only the messages printed by `$display` and UVM/OVM macros like ``uvm_info`, ``uvm_fatal`, ``uvm_warning`, and so on.

Following is the syntax to use the **Go to Message Invocation** command from *Interactive Console*:

```
viaLogViewerRmbInvocation -file <file_name> -line <line_no> [-time
<time>]
```

-file: Specifies file name

-line: Specifies line number

-time: Specifies the time when the message is printed. If you do not specify it, current simulation time is used by default.

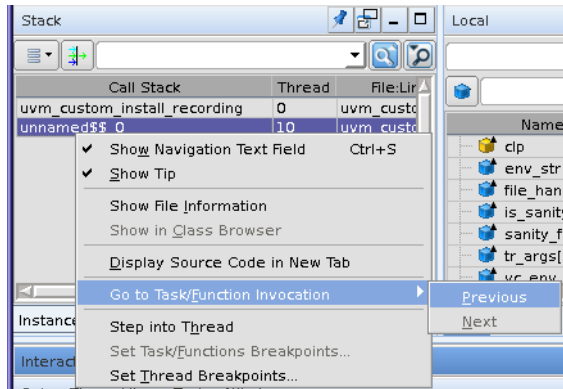
Example:

```
viaLogViewerRmbInvocation -file {../xbus_example_master_seq_lib.sv} -line
{192} -time {1900n}
```

Going to Next or Previous Function/Task Invocation

A new right-click menu command **Go to Task/Function Invocation** with two sub-menu commands **Next** and **Previous** is added to *Stack* frame, *Member* frame, and *Phase pane* (UVM window). The following figure shows this command in the *Stack* frame. This command allows you to go to next or previous function/task invocation, and is enabled only when a valid method is selected.

Figure 139 Go to Task/Function Invocation



Following is the syntax to use the **Go to Task/Function Invocation** command from *Interactive Console*:

From Stack View:

```
srcTBStackGotoInvocation -active -next|-prev
```

-next: Allows you to go to next function/task invocation.

-prev: Allows you to go to previous function/task invocation.

From Phase View:

```
uvmPhaseGoToNextInvocation -phase <name>
uvmPhaseGoToPreviousInvocation -phase <name>
```

-phase: Allows you to specify the selected phase name displayed in the Phase View.

Example:

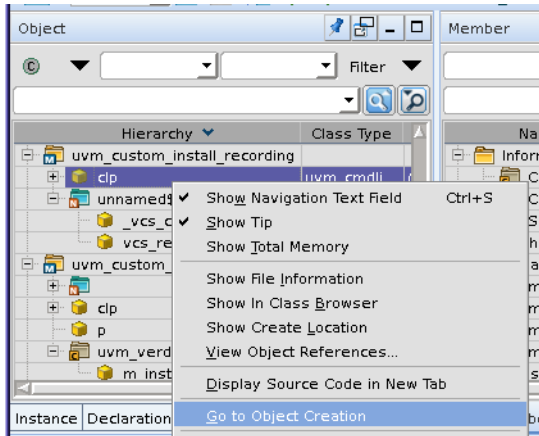
```
uvmPhaseGoToNextInvocation -phase "{build\}"
```

Going Back When an Object is Created

A new right-click menu command **Go to Object Creation** is added to the *Object* frame, *Class* frame, *Member* frame, *Local* frame, and *Watch* frame, as shown in the following

figure. This command allows you to go back when an object is created, and is enabled only when a valid object id is selected.

Figure 140 Go to Object Creation



Following is the syntax to use the **Go to Object Creation** command from *Interactive Console*:

From *Local* frame/*Watch* frame:

```
srcTBGoToCreation -objName <object_name>
```

-objName: Specifies the object name

Example:

```
srcTBGoToCreation -objName "\p::xFool @2"
```

From *Object* frame:

```
srcTBTBOGotoObjectCreation
```

Allows you to go to object creation for selected object id in the *Object* frame.

From *Member* frame:

```
srcTBTBMVGotoObjectCreation
```

Allows you to go to object creation for selected object id in *Member* frame.

Active Point Behavior

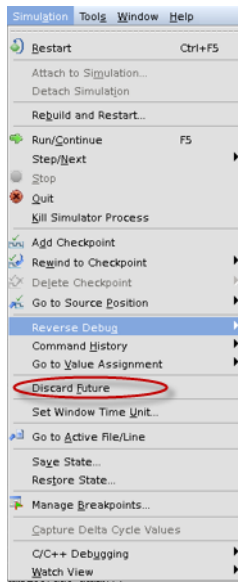
The active point is a checkpoint that is automatically created at the latest available simulation time before going back in time. The active point allows the user to jump back to the latest simulation time from the past.

After the active point is created automatically, whether the user run the simulation without specifying the time or with a specified time that is later than the time of the active point, the simulation stops upon reaching the active point.

Discarding Future Simulator Status

The **Discard Future** command is added to the **Simulation** menu. This command allows you to destroy future simulator status (including FSDB truncation).

Figure 141 *Discard Future*



The following actions issue warning message to remind you of selecting the **Discard Future** command:

- **Drag signals to nWave**
 - If drag-and-drop signals are already dumped, adds them to *nWave*. No message is issued in this case.
 - If drag-and-drop signals are not dumped, a message dialog saying *Can't dump signal 'sig_name', please run 'Discard Future' command and repeat the last operation* is issued.
 - If some drag-and-drop signals are already dumped and some are not, the dumped signals are added, and a message similar to the above mentioned message is issued for the signals that are not dumped.

- Right-click menu commands like **Add to Waveform** or **Add Object to waveform** in different views.
- **File -> Restore Signals** from *nWave*
- For the commands that are issued from *Interactive Console* in past that might change future simulator status, a message is issued in console to ask you to run 'Discard Future' and issue the command again.

Following is the syntax to use the **Discard Future** command from the *Interactive Console*:

```
srcTBDiscardFuture - Discards future simulation status.
```

Discarding Future Automatically

If you do simulation command in past that might change the future simulator status (for example, force, dump, constraint, or assertion), then the following message is displayed:

```
Discard simulation status in future?
```

If you select **Yes**, Verdi automatically allows you to discard future status.

If you select **No**, Verdi cancels your command that can change future simulation status.

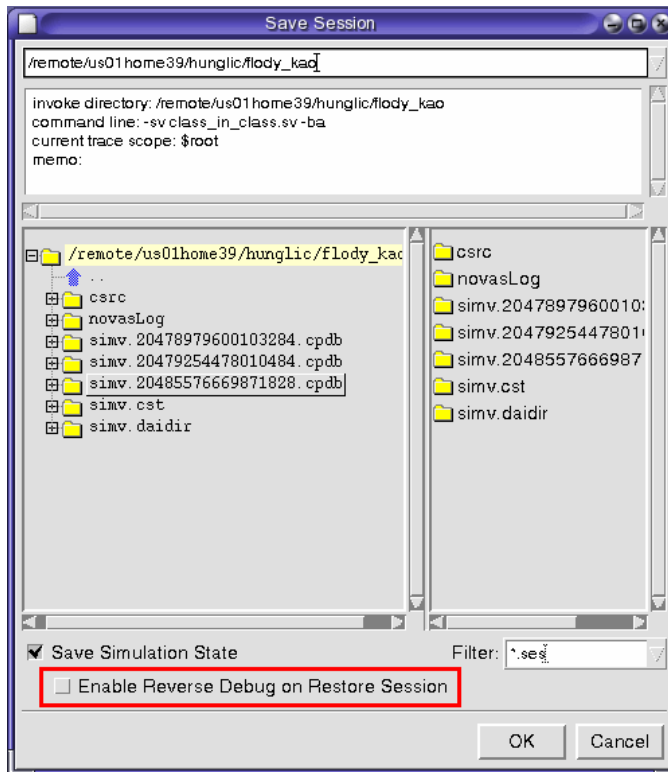
The following right-click menu options allow you to automatically discard future:

- **Dump to FSDB/Dump** in the *Source Code* frame, *Local* frame, *Watch* frame, *Member* frame, and instance tree
- **Force Signal** in the *Source Code* frame
- **Change value** in *Local* frame/*Watch* frame
- **Set Constraint Mode** in *Local* frame/*Watch* frame
- **assert on/off/kill** in the *nWave*, *nTrace* and related dialogs

Saving and Restoring Session

When invoking the **File -> Save Session** command, turn on the **Enable Reverse Debug on Restore Session** option to save all the needed data to enable reverse debug after restoring a session.

Figure 142 Enable Reverse Debug on Restore Session in Save Session Form



Note:

Depending on the simulation memory footprint and runtime length, turn on the **Enable Reverse Debug on Restore Session** option can take significant disk space.

The *Restore Session* form, opened by the **File -> Restore Session** command, allows to restore a saved session. If the session is saved with reverse debug enabled, it will automatically restore all needed data.

After restoring a session, you can use all reverse debug features in the same way as in Interactive debug.

Note:

There might be some initial wait time as data is loaded incrementally as needed from disk.

The VCS version used to save and restore a session must be the same. Mixed simulation versions are not allowed.

UCLI Save and Restore Session

Saving and Restoring

The UCLI checkpoint command supports saving and restoring with checkpoints.

Use the following UCLI command line to save and restore a session:

```
checkpoint
[ -save -session <name> ] |
[ -save_add <checkpoint id> -session <name> ] |
[ -restore -session <name>]
```

Where, *<checkpoint id>* is the checkpoint number, allowing to save an individual checkpoint to a session. To obtain a list of all existing checkpoints and their IDs, execute the `checkpoint -list` command.

The default directory for a saved session is the current directory. All data is located in the *<name>* and *<name>.data* files, and in the *<name>.checkpoints* directory.

To save/restore session to a different directory, specify a relative or an absolute path as a *<name>*.

Example:

```
checkpoint -save -session ./test/session1.cpdb
checkpoint -save -session ~/temp/session1.cpdb
checkpoint -restore -session ./test/session1.cpdb
checkpoint -restore -session ~/temp/session1.cpdb
```

Specifying Compression Level of a Save Session

You can use the following configuration setting to specify compression level when saving checkpoints to a session:

```
config checkpointcompression [high | medium | low]
```

Default is `low`, which means faster runtime. When running in batch mode, if runtime is not important, you can use `high` or `medium` to reduce disk space consumption.

Adding Virtual Checkpoints to a Session

VCS allows you to add virtual checkpoint to a session. You can create a real checkpoint before UVM error and a virtual checkpoint at UVM error, then you can directly rewind to UVM error and start debugging from it. Since it is a virtual checkpoint, VCS rewinds to real checkpoint before UVM error and re-executes to the exact point where UVM error occurred. Virtual checkpoints have little memory overhead at the expense that rewind to them might take some more time.

To add virtual checkpoints to a session, use the following commands in your scripts:

- Add the new `-virtual` option to the `checkpoint -add` command. Following is the syntax:

```
checkpoint -add [<description>] -virtual
```

This command creates a virtual checkpoint even if reverse debug is not enabled and also if there is no checkpoint available before. The only requirement is that step counter should be enabled at this point of time. Step counter is enabled as soon as a first checkpoint is created (or reverse debug is enabled).

- Save virtual checkpoints to the session using `checkpoint -save_add` command.

Example

```
set count 1
run 900ns
# Create a real checkpoint some time before UVM error.
set id [checkpoint -add "before UVM error #${count}"]
checkpoint -save_add $id -session test
checkpoint -kill $id
stop -uvm error
run
# Create a virtual checkpoint at UVM error.
set id [checkpoint -add "UVM error #${count}" -virtual]
checkpoint -save_add $id -session test
checkpoint -kill $id
. . . repeat for other time stamps . . .
```

After restore, you can see the checkpoints in Verdi GUI:

```
1: "before UVM error #1"
2: "UVM error #1"
. . .
```

You can now directly rewind to `UVM Error #1` and start debugging from it. Since it is a virtual checkpoint, the tool rewinds to real checkpoint `before UVM error #1` and re-executes to the exact point where UVM error occurred.

Support for Debug Points

VCS allows you to specify the simulation events (for example, UVM error or fatal messages) as special kind of checkpoints called "debug points" in the simulation. Auto-checkpoint and session save handles the real checkpoints (a special checkpoint created at the current simulation end before executing the reverse command) created before a debug point in a special way to make sure rewind to the debug point is fast after session restore. As a result, the time to start debugging after session restore can be reduced significantly for long running simulations. The debug points are shown in Verdi GUI as usual checkpoints and you can easily rewind to them.

Also, to prioritize debug points in a saved session (and to protect against exceeding disk quota), you can specify maximum disk space to be occupied by the saved session.

The following sections describe the debug points feature in detail:

- [Creating Debug Points](#)
- [Creating Debug Points Automatically](#)
- [Creating User-Defined Debug Points](#)
- [Breakpoints on UVM Error and Fatal Messages](#)
- [Limiting the Size of a Save Session](#)
- [Debug Point Prioritization](#)

Creating Debug Points

To create a new debug point, use the `-debug_point` option with the UCLI `checkpoint` command. Following is the syntax:

```
checkpoint -add "error_type" -debug_point
```

Example:

```
checkpoint -add "UVM Error" -debug_point
```

To allow fast rewind to a debug point, some real checkpoint must be kept close before it. If the real checkpoint is too far away from the debug point, then initial rewind to the debug point takes much time. If it is too close, then the debug range is reduced accordingly. The following new configuration setting allows you to specify how far away (in seconds) a real checkpoint must be kept before debug point:

```
config debugpointinterval 10
```

Default time is 10 seconds, that is, a real checkpoint should be kept at least 10 seconds before debug point.

The checkpoints reserved for the debug points are taken from the checkpoints specified for auto-checkpoint (`config checkpointgenerate`, by default 40). Therefore, it is required to limit the number of debug points. The following new configuration setting allows you to specify the number of debug points (default is 5):

```
config debugpoints 5
```

You cannot use more than $N-2$ debug points, where N is the number of automatic checkpoints.

Creating Debug Points Automatically

VCS automatically creates checkpoints when UVM error or fatal message is produced (when `config uvmerrorcheckpoints on`).

This functionality is enhanced to create checkpoints as “debug points” automatically. The name of the configuration variable is changed to:

```
config uvmdebugpoints on
```

For backward compatibility, old usage is allowed:

```
config uvmerrorcheckpoints on
```

Creating User-Defined Debug Points

The UCLI `stop` command allows you to create debug points using a script file. You can create debug points using the `checkpoint` command in the Tcl script file. Following is the syntax:

```
stop -command {tcl_script} [-continue]
```

Example:

```
stop -command {checkpoint -add "name" -debug_point} [-continue]
```

Breakpoints on UVM Error and Fatal Messages

The UCLI `stop` breakpoint command is enhanced to stop the simulation at UVM error and fatal messages. Following is the syntax:

```
stop -uvm [error | fatal]
```

Limiting the Size of a Save Session

You can specify maximum disk space to be occupied by the saved session to avoid exceeding disk quota. This allows you to prioritize debug points in the saved session.

To enable this feature, a new option `-max_size` is added to the UCLI `checkpoint` command. Following is the syntax:

```
checkpoint [-save | -save_add] -session <name> -max_size <Gb>
```

The `-max_size` option specifies maximum size in Gb which a saved session should not exceed (after the save is complete). A real number is allowed as a size, for example:

```
-max_size 0.5
```

For the `-save_add` option, if the existing session exceeds the specified size, an error message is issued and the checkpoint is not saved.

For the `-save` option, following is the order of saving checkpoints when `-max_size` is provided:

- Save special checkpoints kept before debug points (starting from the first debug point).
- Save checkpoint for the simulation start.
- Save all other checkpoints in usual way starting from the end.

In the above process, if the size exceeds the maximum size, VCS stops saving and issues a warning message.

Debug Point Prioritization

In most cases, the first UVM error/fatal is the starting point for debugging than the following error/fatal messages. That is, when saving session by nightly run in batch mode, more checkpoints should be saved for the first debug point rather than for the following checkpoints. To achieve this, you can use the following script:

```
config endofsim noexit
config reversedebg on
stop -uvm error
stop -uvm fatal
run
stop -delete *
checkpoint -save -session "foo" -max_size 5
run
checkpoint -save -session "foo" -max_size 10
quit
```

The above script uses 10 Gb memory for the complete session, from which 5Gb is used for the first debug point.

To allow this use model, the auto-checkpoint feature is enhanced for the checkpoints already saved to the disk. When the auto-checkpoint feature detects that some checkpoints must be deleted, the checkpoint process is killed (so that it does not consume memory), but the checkpoint is not removed from the disk by the save session commands of the same session.

Reverse Debug Commands

The **Reverse Debug** command includes the following sub-commands: **Run/Continue Reverse**, **Step Reverse**, **Next Reverse**, **Step in Thread Reverse**, **Next in Thread Reverse**, **Step in Testbench Reverse**, and **Step Out Reverse**.

Run/Continue Reverse

Menu Bar:	Simulation -> Reverse Debug -> Run/Continue Reverse
Toolbar Icon:	

This command specifies the amount of time and use the **Run/Continue Reverse** command (by clicking the above icon or selecting the **Simulation -> Run/Continue Reverse** command) to go back in time for the specified amount of time, as the same time entry field used by the **Run/Continue** command (see figure). All the current breakpoints are respected and the simulation stops at the most recent (considered back from the current execution state) breakpoint hit.

When the time entry field is empty and no breakpoints are hit, the simulation is rewind back to the start. If the reverse debug feature was enabled some time after the simulation started, using the **Run/Continue Reverse** command (as well as other reverse commands) stops at this point instead. In this case, a confirmation dialog box is displayed with the *"Rewind to first checkpoint at time <time>?"* message.

The run-time of executing the **Run/Continue Reverse** command is dependent on how far back in time it would go. If executing the **Run/Continue Reverse** command takes long time, you can interrupt it by clicking the  **Stop** icon in the toolbar.

Step Reverse

Menu Bar:	Simulation -> Reverse Debug -> Step Reverse
Toolbar Icon:	

This command goes back one SystemVerilog source code line.

Next Reverse

Menu Bar:	Simulation -> Reverse Debug -> Next Reverse
Toolbar Icon:	

This command goes back one SystemVerilog line which steps over task/function calls. Eventually, it might stop on a breakpoint inside the task/function called at the previous line.

Step in Thread Reverse

Menu Bar:	Simulation -> Reverse Debug -> Step in Thread Reverse
Toolbar Icon:	

This command goes back one source code line in the current thread.

Next in Thread Reverse

Menu Bar:	Simulation -> Reverse Debug -> Next in Thread Reverse
Toolbar Icon:	

This command goes back one source code line in the current thread which steps over task/function calls.

Step in Testbench Reverse

Menu Bar:	Simulation -> Reverse Debug -> Step in Testbench Reverse
Toolbar Icon:	

This command goes back one source code line in the testbench code.

Step out Reverse

Menu Bar:	Simulation -> Reverse Debug -> Step out Reverse
Toolbar Icon:	

This command goes back to the source code line where the current function has been called.

Go to Active Point

Menu Bar:	Simulation -> Reverse Debug -> Go to Active Point
Toolbar Icon:	

This command allows you to quickly go to the active point.

Checkpoint Distribution

The auto-checkpoint feature uses half of the checkpoints for logarithmic distribution (near the current simulation execution location) and other half for linear distribution (checkpoints are distributed regularly over all debug range). This distribution provides optimal reverse debug performance with reasonable memory overhead for RTL design.

However, for gate-level designs checkpoint memory overhead is much higher and linear checkpoints cause much more overhead than logarithmic ones. To keep memory overhead low, you must reduce the number of used checkpoints to 10 or 5. Also, in this case, it is desirable to use only logarithmic checkpoints.

To make auto-checkpoint feature use logarithmic checkpoints only, the following new configuration setting is added to UCLI:

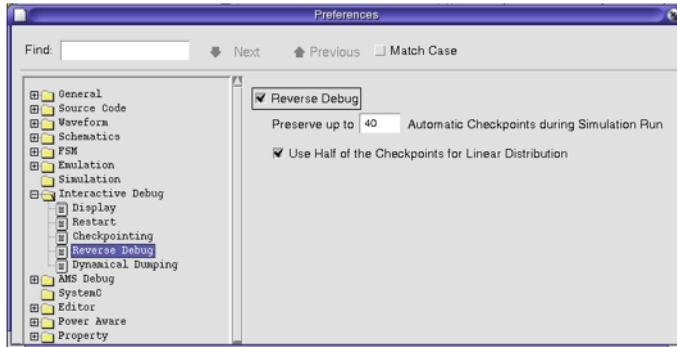
```
config checkpointdistribution [hybrid | log]
```

Default is *hybrid*.

Verdi **Preferences** form for the **Reverse Debug** option is enhanced with the following new option, as shown in the following figure:

Use Half of the Checkpoints for Linear Distribution

This checkbox is checked by default, and is enabled only when the **Reverse Debug** option is enabled. When this option is enabled, Verdi calls `config checkpointdistribution hybrid` UCLI command, and when disabled, Verdi calls `config checkpointdistribution log` UCLI command.



Checkpoint Memory Profiling

VCS allows you to check memory overhead of checkpoints and adjusts the number of checkpoints to be used depending on their runtime memory usage limits. To enable this feature, a new `-memory` option is added to the UCLI `checkpoint` command. Following is the syntax:

```
checkpoint -memory
```

This command reports the current auto-checkpoint overhead for resident memory usage. For example:

```
Checkpoint proportional sizes:
* 1: 777 Mb
* 2: 769 Mb
* 3: 593 Mb
* 4: 573 Mb
* 5: 528 Mb
* 6: 351 Mb
* 7: 312 Mb
* 8: 319 Mb
* static database server: 322 Mb
Active simulation process:
* Proportional size: 394 Mb
* Resident size: 1732 Mb
Total size (active process + checkpoints):
* 4938 Mb (overhead: 3206 Mb, 2.9x)
```

Proportional sizes here reflect memory usage when taking page sharing into account. For example, assume you have 5 checkpoints all sharing 100 Mb memory, and each checkpoint has 50 Mb private memory pages. In this case, each checkpoint has “proportional size” as $50 + 100/5 = 70$ Mb.

The auto-checkpoint overhead is shown relative to resident size of active simulation process. The `checkpoint -memory` command can be used, for example, in the following way:

- Run the simulation to the end with reverse debug enabled.
- Check output of `checkpoint -memory`.
- If memory overhead is too high, reduce the number of checkpoints, restart the simulation, and repeat the above steps until it does not exceed the desired limit.

Reverse Debug Support for VHDL

The following Verdi/UCLI commands are enhanced to work correctly for VHDL source code:

Verdi Commands

- Step Reverse
- Next Reverse
- Run Reverse (This command correctly handles VHDL breakpoints)
- Undo/Redo

UCLI Commands

- `step -reverse`
- `next -reverse`
- `run -reverse` (This command correctly handles VHDL breakpoints)
- Virtual Checkpoints (`config virtualcheckpoints [on|off]`). Virtual checkpoints are created for VHDL locations.

Limitations

The following limitations apply to the Reverse Debug feature:

- Reverse debug is not supported for pure VHDL designs.
- The **Step Out Reverse** command is not supported for VHDL.

Limitations

For Reverse Debug:

- VCS Design Level Parallelism (DLP) is not supported
- VCS Fine-Grained parallelism (FGP) is not supported
- The following actions of PLI code are not supported:
 - IPC communication using sockets, pipes or shared memory
 - Performing the file seeking operations, and then writing at a new position (that is, it is assumed that simulation only appends data to the output files)
- Analog-digital co-simulation (using NanoSim) is not supported
- The Reverse debug commands are not supported for pure VHDL source code.
- When the design is compiled with the **-simprofile** option for simulation profiling, reverse debug will not be possible.

For **Go to Previous/Next Assignment** feature:

- Several object types do not allow the setting of a value change breakpoint. When the breakpoint cannot be set, an error message is generated and the command has no effect.

14

Tracing Testbench Driver in Temporal Flow View

The following sections describe this feature in Interactive Simulation Debug Mode:

- [Introduction](#)
- [Viewing Testbench Frame Node on TFV](#)
- [Viewing Signal Information when Tracing on Existing Path](#)
- [Property Window for Testbench Node/Folder](#)
- [Limitations](#)

Introduction

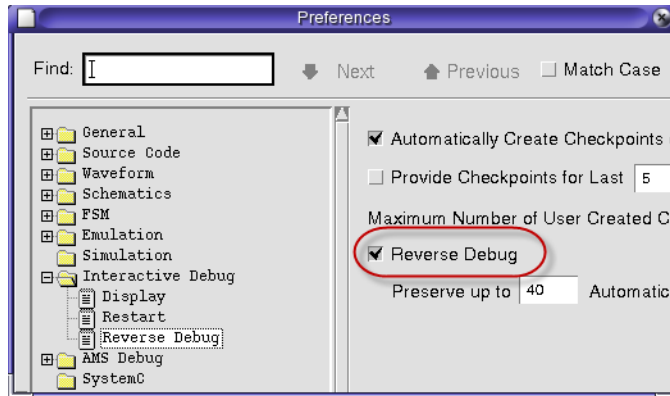
Verdi provides a new tracing mechanism to trace the driver of value change on the testbench scope using the **Trace on TFV** command icon  in the toolbar.

The **Trace on TFV** command records the tracing results and displays them in the *Temporal Flow View* frame. You can use the tracing map in the *Temporal Flow View* frame to figure out the relation of signal driver/load.

Perform the following steps to enable this feature:

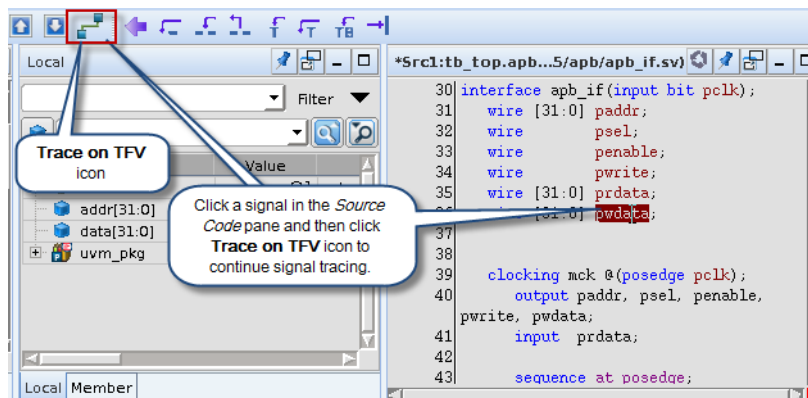
1. Compile your design with the `-debug_access+reverse` option. This option enables reverse debug capabilities.
2. In the Verdi GUI, select the **Reverse Debug** option in the Preferences dialog box, as shown in the following figure:

Figure 143 Preferences Dialog



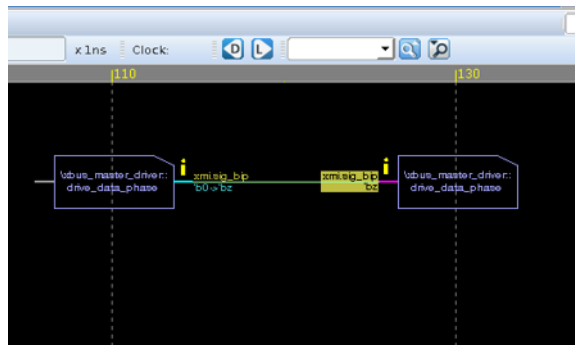
3. To perform signal tracing on the testbench scope, use either of the following methods:
 - a. Select a signal in the *Source Code* pane, as shown in Figure 144.

Figure 144 Signal Tracing



Click the **Trace on TFF** icon in the toolbar. The trace path is added to the *Temporal Flow View* as shown in Figure 145.

Figure 145 Tracing Testbench Driver in Temporal Flow View



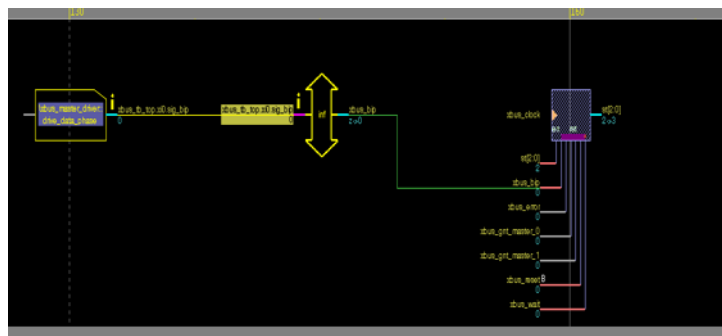
The **i** icon on the trace path indicates the path traced with a single signal and **i+** icon indicates the path traced with multiple signals.

After first tracing, the driver statements are highlighted in the *Source Code* pane.

- b. Double-click on the hardware boundary signal.

Verdi displays the trace path connected to the testbench signal as shown in the following figure:

Figure 146 Trace Path Connected to the Testbench Signal in the Temporal Flow View



Viewing Testbench Frame Node on TFB

The following topics describe this section in detail:

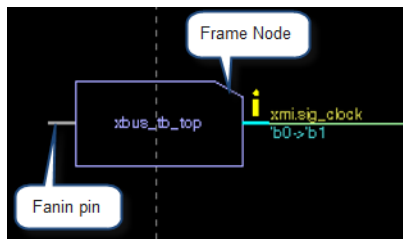
- [Frame Node](#)
- [Stack Node](#)
- [Interface Node](#)
- [Trimmed label](#)

- [Right-Click Options](#)
- [Tracing to Same Stack Node at the Same Time](#)

Frame Node

A signal is represented by a frame node as shown in the following figure. An empty fanin pin is added to the frame node if it is end of the trace path.

Figure 147 Frame Node



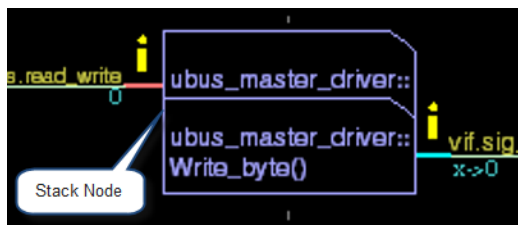
You can click a fanin pin and then click **Trace on TFV** icon in the toolbar to continue signal tracing.

If you double-click a frame node, *Source Code* pane shows the execution point of the active frame.

Stack Node

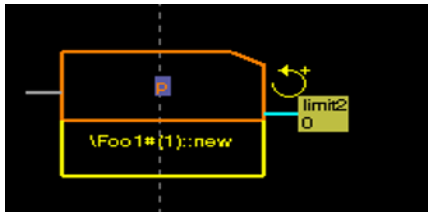
If multiple frames with the same time are very close to each other in the same stack, they are merged into a stack node as shown in the following figure:

Figure 148 Stack Node



If you double-click a frame within a stack node, the *Source Code* pane shows the execution point of the frame. The clicked frame is highlighted in orange color as shown in the following figure, and indicates the current rewind frame.

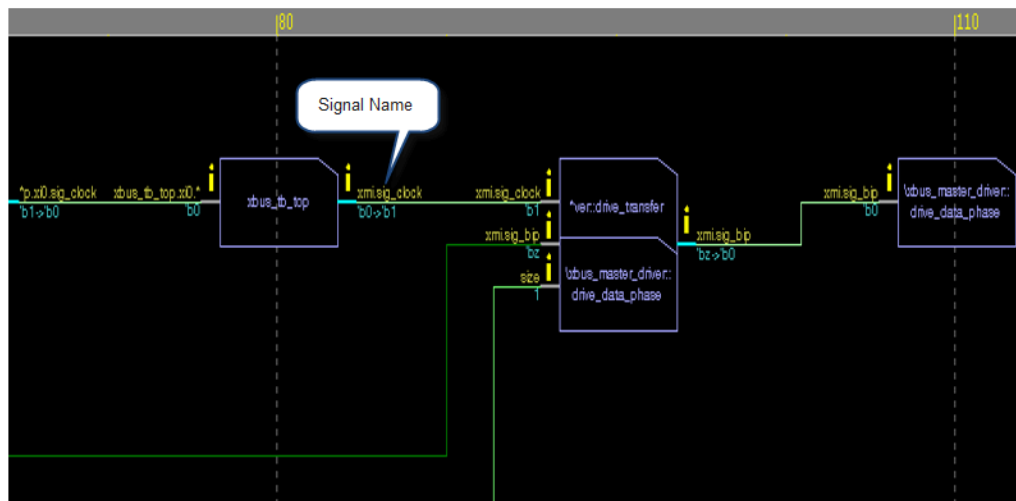
Figure 149 Frame Highlighted in Orange Color



If you execute new trace or any commands that change the simulation execution point, the highlight color will be canceled.

The selected signal name is displayed as a pin name on the frame node. The connection connects from start node to end node as shown in [Figure 150](#).

Figure 150 Signal Name



If you double-click on normal frame node, Verdi rewinds to the latest trace point of the frame.

If you double-click a testbench signal pin, the source statement is highlighted in the *Source Code* pane.

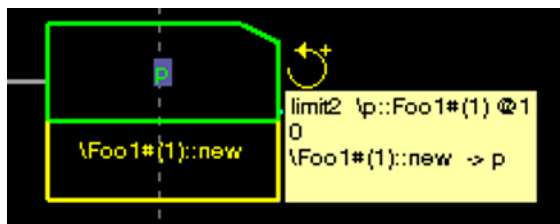
Hovering the mouse pointer on a frame node highlights the frame and displays its full name as shown in the following figure:

Figure 151 Viewing Frame Name



Hovering the mouse pointer on a trace path frame or a rewind frame shows its information in a tooltip as shown in the following figure:

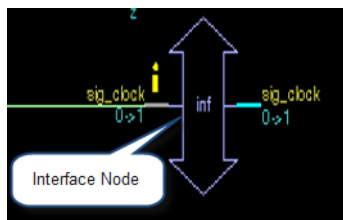
Figure 152 Viewing Frame Information in a Tooltip



Interface Node

If the trace path traces through a hardware interface, an interface node is added to the trace path, as shown in the following figure:

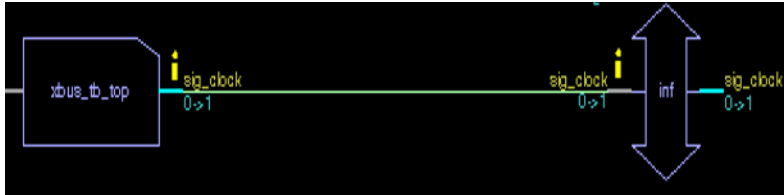
Figure 153 Interface Node



An interface node indicates that the tracing happens through the hardware interface signal.

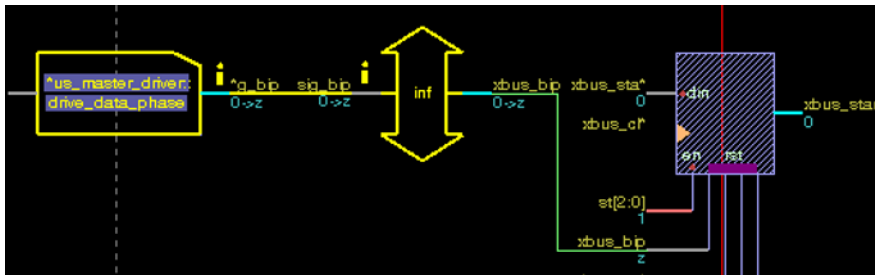
If tracing happens directly from the interface signal, the start node is the interface node, as shown in the following figure:

Figure 154 Tracing Through Interface Signal



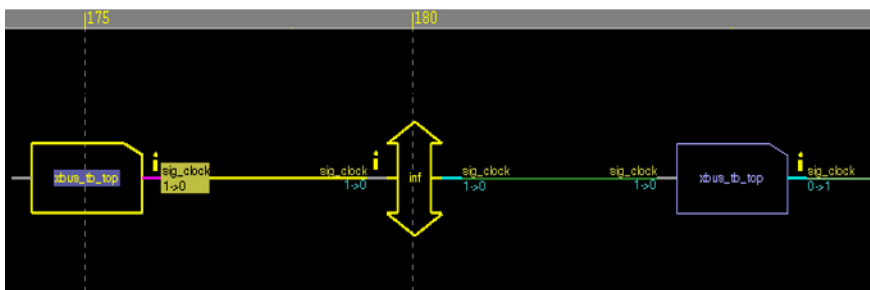
If tracing happens from the DUT cell, the trace path traces through the interface node and connects to end frame, as shown in the following figure:

Figure 155 Tracing Through DUT



If tracing of the interface signal happens from testbench frame, the trace path traces through the interface cell, as shown in the following figure:

Figure 156 Tracing from Testbench



The fanout signal connects to the start frame and fanin signal connects to the end frame.

If you double click on an interface node, Verdi does not go back to the execution point. You can use the property window (see [Property Window for Testbench Node/Folder](#)) to go back to the related execution point of an interface cell.

Trimmed label

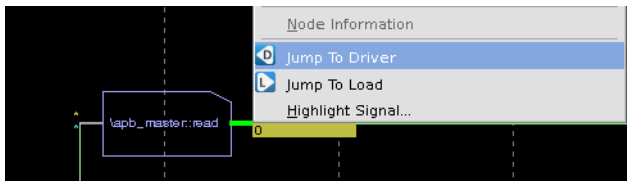
If the signal name is longer than the label space, Verdi trims off extra name and replaces it with "^".

For a testbench signal, Verdi trims the signal from start to end. If you enable Show Hierarchy name, Verdi trims the hierarchy name first.

Right-Click Options

You can right-click a signal on the frame node and select the **Jump To Driver** or **Jump to Load** right-click option, as shown in the following figure, to jump to the respective driver/load in the *Source Code* pane.

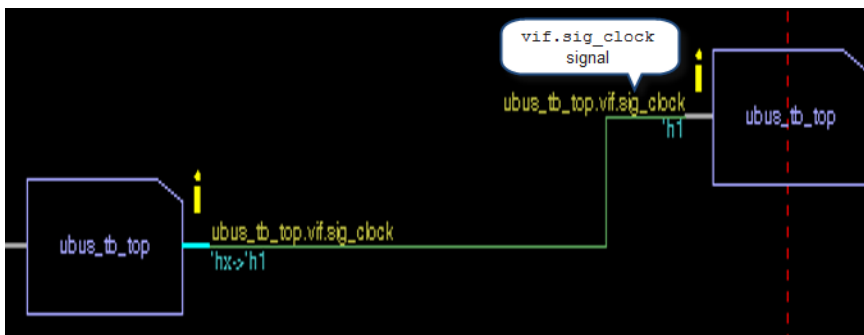
Figure 157 Right-click Options



Viewing Signal Information when Tracing on Existing Path

When you trace on the existing trace path, Verdi updates the trace information on the signal label. Therefore, you can check the trace information on the TFV trace path. Verdi adds the  icon on the trace path to indicate the path traced with a single signal and adds the  icon to indicate the path traced with multiple signals. For example, when you trace the `vif.sig_clock` signal, Verdi creates a trace path as shown in the following figure:

Figure 158 Tracing on Existing Trace Path



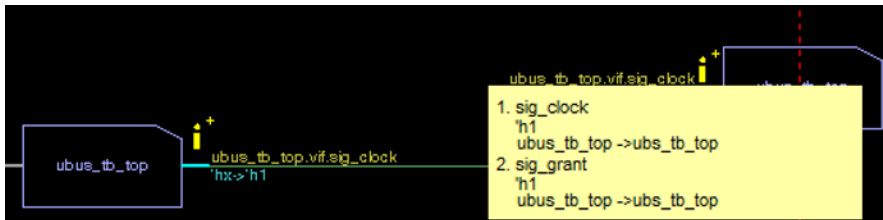
When you go back to the first frame and trace another signal `vif.sig_grant`, as the trace path is same as `vif.sig_clock`, Verdi combines these two tracing results and shows the  icon above the signal pin as shown in the following figure:

Figure 159 Tracing on Existing Trace Path



Hovering the mouse pointer on the  icon displays related traced signal information in a tooltip as shown in the following figure:

Figure 160 Viewing Traced Signal Information in a Tooltip

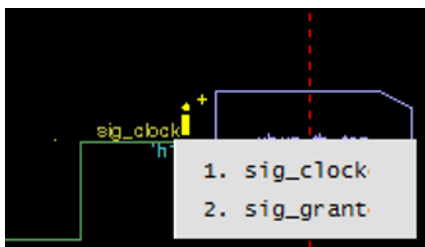


Following is the tooltip format:

```
#. signal name object name
value
start frame -> end frame
```

When you right click on the  icon, Verdi creates a menu list with related signals as shown in the following figure:

Figure 161 RMB Menu



You can highlight the related trace path by clicking on a signal in the menu list. Verdi highlights the trace path and displays related signal name on the path as shown in the following figure:

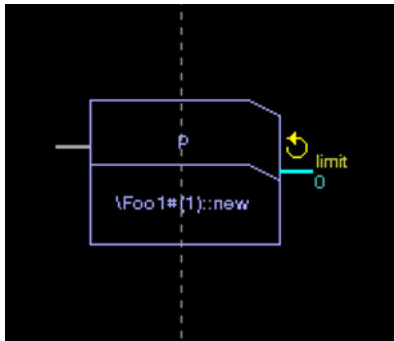
Figure 162 Highlight Related Trace Path



Tracing to Same Stack Node at the Same Time

When you trace back to same stack at the same time, Verdi creates a dotted frame node connected to the trace start frame node, as shown in the following figure, to indicate the trace is back to same stack at the same simulation time.

Figure 163 Dotted Frame Node



For example, tracing signal `limit` on frame `Foo1#(1)::new` at time 0 traces back to frame `p` on same stack at time 0.

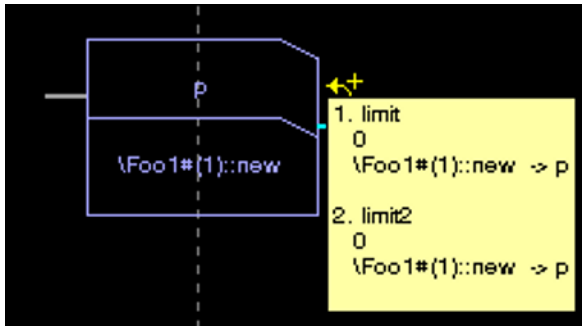
Figure 164 Tracing Signal Limit

Call Stack	Thread	File:Line
p	0	class_in_class.sv:35
Foo1#(1)::new	2	class_in_class.sv:70

The loop  icon indicates the presence of a single loop path in the node. The multiple loop  icon indicates the presence of multiple trace path loops on the same stack node.

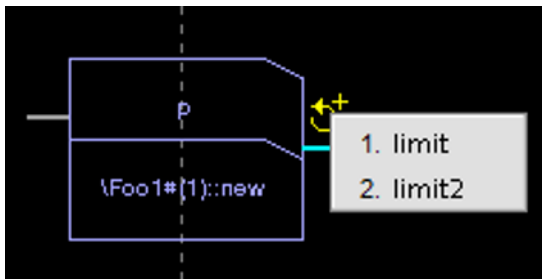
Hovering the mouse pointer on the /  icon shows related connection information in a tooltip as shown in the following figure:

Figure 165 Viewing Connection Information in a Tooltip



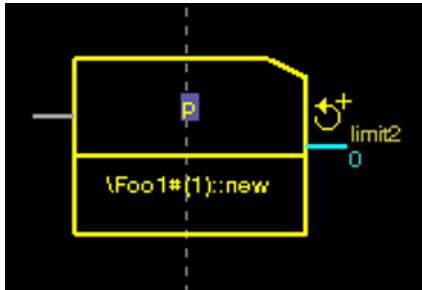
When you right click on the /  icon, Verdi creates a menu list with related signals as shown in the following figure:

Figure 166 RMB Menu



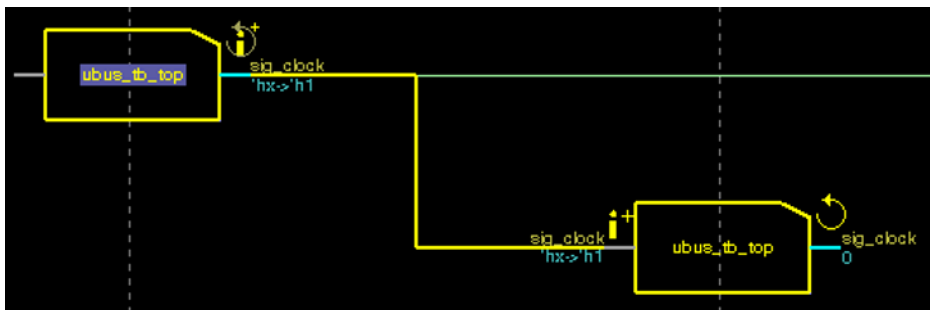
You can highlight the related trace path by clicking on a signal in the menu list. Verdi highlights the related frame and changes the fanout signal and value as shown in the following figure:

Figure 167 Highlight Related Trace Path



Verdi adds the  icon to the trace path, as shown in the following figure, if the stack node is also the end node of the trace path.

Figure 168 Highlight Related Trace Path

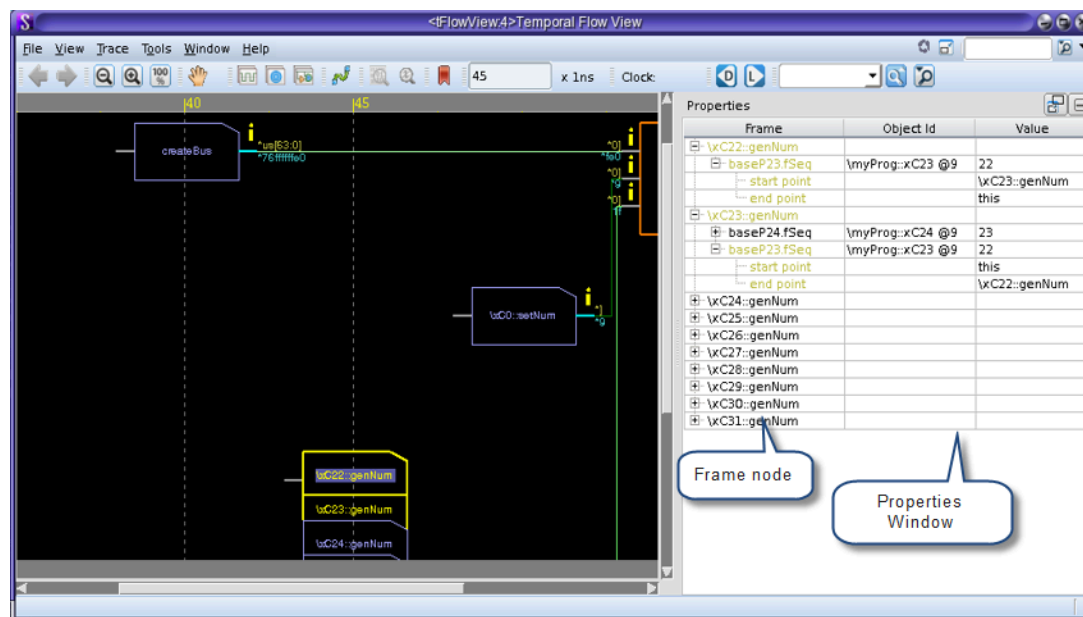


Property Window for Testbench Node/Folder

Starting with this release, you can use the *Properties* window in the *Temporal Flow View* frame to view detailed information of the testbench trace.

In the *Temporal Flow View* frame, select **View -> Properties** to open the *Properties* window as shown in the following figure:

Figure 169 Properties Window



You can also use the keyboard shortcut key **Ctrl+P** to turn on/off the *Properties* window.

The *Properties* window allows you to highlight and rewind check point by double-clicking a frame item in the hierarchical tree list.

When you select a testbench folder on the TFV, the *Properties* window displays all the related frames on the tree list. The first level tree node is the frame node and the related traced signals are the sub-tree nodes.

For each traced signal, a tree node contains two sub-tree nodes, one is the start check point of the trace and another is the end check point of the trace.

To select a fanin or fanout signal, the *Properties* window lists all traced signals as the first level tree node, and also contains two fixed sub-tree nodes (start point and end point for user to rewind check point) for each node.

The structure of the hierarchical tree list is as follows:

For the Testbench folder selected on TFV:

First level: frame node

Second level: trace signal node

Third level: two fixed entries - start point and end point

For the Testbench Node selected on TFV:

First level: trace signal node

Second level: two fixed entries - start point and end point

You can double click on a frame node to expand it, and then you can double click on a signal node to highlight the traced path. You can double click on a start or end check point item to rewind to check point.

The coloring of the property tree is same as the coloring of the testbench folder on the TFV.

Limitations

Following is the limitation of this feature:

- Session save/restore is not supported for Testbench path on TFV.

15

Appendix A: SystemVerilog Example

This appendix provides a method to use an UVM example with the VCS simulator to practice the features of *Interactive Simulation Debug* mode in the Verdi platform. This example is the UVM `test_read_modify_write` example with the Verdi modified libraries. Change directory to the `$VERDI_HOME/share/vmlib/uvm/uvm/lib/examples/integrated/ubus_evdp_ntb/examples` directory, follow the steps to set the environment variables, and specify the compile and simulation options.

1. Set the environment variables as described in the [Set Environment Variables](#) section:

```
# Set VCS environment
> setenv FSDB_ENV_DUMP_SEQ_NUM 1
> setenv FSDB_ENV_MAX_GLITCH_NUM 0
# Set for the Save Session and Save State feature
> setenv VCS_ENABLE_ASLR_SUPPORT 1

# Set FSDB dumping environment
> setenv VERDI_HOME <VERDI_INST_DIR>
> setenv tab $VERDI_HOME/share/PLI/VCS/LINUXAMD64/novas.tab
> setenv pli $VERDI_HOME/share/PLI/VCS/LINUXAMD64/pli.a
> setenv LD_LIBRARY_PATH
$VERDI_HOME/share/PLI/VCS/LINUXAMD64:$LD_LIBRARY_PATH
```

2. Specify the `VCS_UVM_HOME` environment variable before compilation to identify the path to the UVM 1.1d library to VCS simulator.

```
> setenv VCS_UVM_HOME $VERDI_HOME/share/vmlib/uvm/uvm-1.1d/lib/src
```

3. Use the following command to compile the design:

```
> vcs -full64 -debug_access+all -sverilog -timescale=1ns/1ns +acc +vpi
+incdir+$VCS_UVM_HOME $VCS_UVM_HOME/uvm.sv -P $tab $pli -ntb_opts uvm
+incdir+../sv ubus_tb_top.sv
```

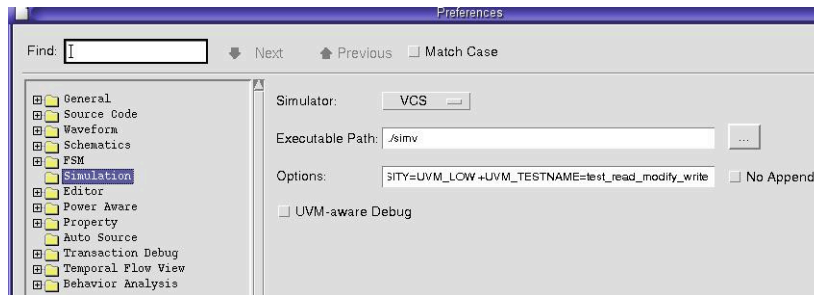
4. Invoke the Verdi platform and import your UVM testbench:

```
> verdi +UVM_TESTNAME=test_read_modify_write -ntb_opts uvm -sv ./
ubus_tb_top.sv +incdir+../sv
```

5. Click the **Work Modes** button to go to next page and click the **Go to Work** button to start with the main page of the Verdi platform.

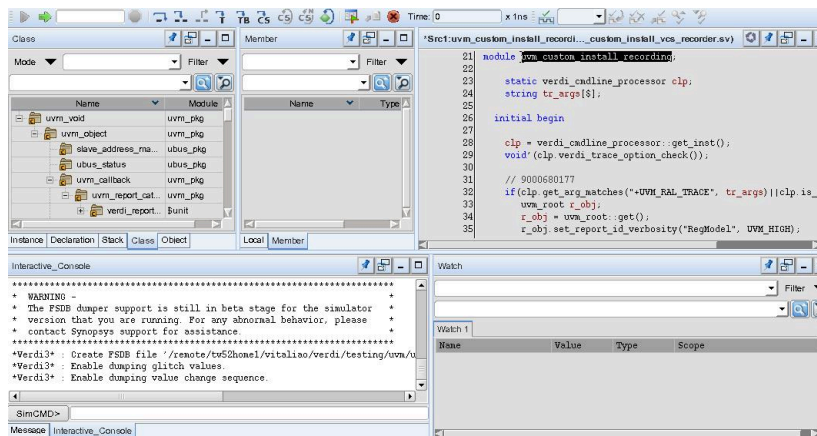
- Use the **Tool -> Preferences** command to open the *Preferences* form and select the **Simulation** page. Add the `+UVM_VERBOSITY=UVM_LOW` and `+UVM_TESTNAME=test_read_modify_write` UVM configurations in the **Options** field to set up the simulation (refer to the following figure). Click the **OK** button after setting done.

Figure 170 Set Simulation Options in Preferences Form



- Use the **Tool -> Run Simulation** command to enable Interactive Simulation Debug mode. As shown in the following figure, you can start to simulate this example in Interactive Simulation Debug mode.

Figure 171 Enabled Interactive Simulation Debug Mode with UVM Example



16

Appendix B: VHDL Support

The following sections describe the VHDL supported data types and the differences between VHDL and Verilog/SystemVerilog in the Verdi Interactive Simulation Debug mode.

Data Types

The supported data types of VHDL are ENUMERATION, INTEGER, REAL, PHYSICAL, ARRAY, RECORD, ACCESS, BOOLEAN, BIT, TIME, CHARACTER, STRING, BIT_VECTOR, STD_LOGIC and STD_LOGIC_VECTOR.

Differences between VHDL and Verilog

Local Tab

- Double-clicking the node in the Local tab highlights the definition of the node.
- The nodes with BIT_VECTOR or STD_LOGIC_VECTOR data types cannot be expanded.
- When the simulation stops into VHDL scope (for example, VHDL process), but not VHDL function/procedure, the Local tab is empty.
- The array display uses "()" to indicate each dimension instead of "[]".

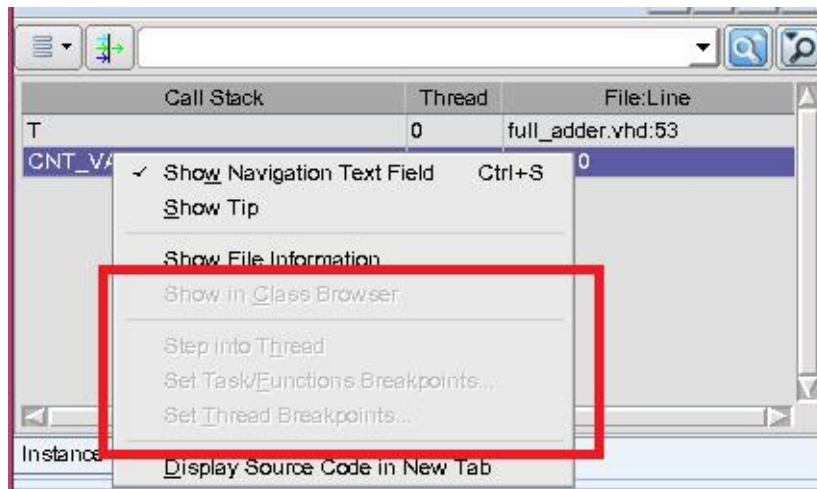
Figure 172 Array Display

Name	Value	Type
VAR_ARRAY		CC
VAR_ARRAY(5)		BB
VAR_ARRAY(5)(3)		AA
VAR_ARRAY(5)(3)(1)	FALSE	BOOL
VAR_ARRAY(5)(3)(2)	FALSE	BOOL
VAR_ARRAY(5)(4)		AA
VAR_ARRAY(5)(4)(1)	FALSE	BOOL
VAR_ARRAY(5)(4)(2)	FALSE	BOOL
VAR_ARRAY(6)		BB
VAR_ARRAY(6)(3)		AA
VAR_ARRAY(6)(3)(1)	FALSE	BOOL
VAR_ARRAY(6)(3)(2)	FALSE	BOOL
VAR_ARRAY(6)(4)		AA
VAR_ARRAY(6)(4)(1)	FALSE	BOOL
VAR_ARRAY(6)(4)(2)	FALSE	BOOL
VAR_ARY		DD
VAR_HEAD		ACCESS
VAR_INLINE	null	ACCESS
VAR_SIGNED_VAR	UU	STDLOGICVF
VAR_STRING	SR	STRING

Stack Tab

- When the simulation stops into VHDL scope (for example, VHDL process), but not VHDL function/procedure, the Stack tab is empty.
- The Thread mode is disabled and the unnamed scope cannot be hidden for VHDL scopes. The following figure also shows the disabled commands.

Figure 173 Disabled Commands for VHDL



Watch Tab

- When the simulation stops into VHDL sub-program, the variables declared inside a VHDL function/procedure can be added to the Watch tab by dragging the variable from the Local tab or *Source Code* pane. The content of the variables are based on the active scope.
- When the simulation stops into VHDL scope (for example, VHDL process), but not a VHDL function/procedure, the VHDL signals or variables can be added to the Watch tab by dragging the variable from the *Source Code* pane and dropping to the Watch tab. The content of the variables are their runtime value accordingly and based on the global paths that point to the instances.
- The array display is the same as in the Local tab.

Dumping

When the simulation stops into a function/procedure, the variables declared inside a VHDL sub-program cannot be dumped to the FSDB file or be added to the *nWave* window.

17

Appendix C: Tcl Dumping Command

The following are the supported dumping commands in the UCLI command line (refer to the *FSDB Dumping Commands* section of the *Linking Novas Files with Simulators and Enabling FSDB Dumping manual* (`$VERDI_HOME/doc/linking_dumping.pdf` for detailed usage):

- `fsdbDumpvars`
- `fsdbDumpvarToFile`
- `fsdbDumpflush`

18

Appendix D: Known Problems and Limitations

- The Verdi Interactive Simulation Debug Solution only works with VCS versions 2013.06 or later. If other versions of VCS are used, the Verdi platform exits Interactive Simulation Debug directly.
- Using Verdi Interactive Simulation Debug mode and Verdi assertion tools (for example, Evaluator, Statistics, Analyzer and Add Temporary Assertions) simultaneously is not supported.
- With VCS versions prior to 2013.06-SP1-1, the rewind and checkpoint features cannot be used.
- The first created checkpoint (with checkpoint id '1') cannot be removed since the simulator limitation.
- Mailboxes, semaphore, constraint, dynamic array, associative array and queues are not supported for dumping. The dynamic and automatic variables outside of a class are not supported for dumping.
- The object member data with class type is not expandable in the waveform. You must manually trigger dumping, and then add the object (member of parent object) by object ID to the waveform as first level item.
- The inter.fsdb FSDB file generated in Interactive Simulation Debug mode is not supported by FSDB utilities and virtual FSDB files.
- The value change of a runtime signal cannot be updated in-time when the simulation time goes to the next one. You must reload the FSDB file for the correct value.
- The following are limitations regarding save and restore functions:
 - The checkpoints cannot be restored currently.
 - The status of the Watch pane cannot be restored.
- Annotation values are not supported for the following data types or coding styles:
 - Modport signals
 - Partial bit(s) selection, for example, `Array[i]`, `Array[CONST]`

- Structure element, for example, `Array[i].a`, `func().b`
- Property signals
- The radix setting in the *Source Code* pane and in the **Local/Watch** tabs are independent.